

Shell 编程是 Linux/UNIX 环境下必备的技能，核心是通过脚本自动化执行命令、处理文本、管理系统。本文基于 **Bash (Bourne-Again Shell)** (Linux 默认 Shell) 编写

一、基础入门：Shell 脚本核心概念

1.1 编程介绍

1.1.1 编程语言初始

1.1.1.1 基础知识

1.1.1.1.1 核心知识点

1. 编程的本质：

编程是将人类解决问题的逻辑，转化为计算机能理解并执行的**指令集合**。计算机只能识别二进制 (0/1)，编程语言作为“翻译器”，让人类用接近自然语言的方式编写指令，再通过解释器 / 编译器转化为机器码。

2. Shell 编程的定位：

Shell 编程是面向 **Linux/UNIX 系统管理、自动化操作** 的脚本编程，核心价值是“串联系统命令”——无需编写复杂逻辑，即可快速实现文件处理、进程管理、批量任务自动化。

3. 计算机执行程序的基本逻辑：

- 输入：接收用户 / 文件 / 其他程序的指令 / 数据；
- 处理：CPU 执行指令（计算、判断、循环等）；
- 输出：返回结果（屏幕打印、文件写入、状态码等）；

Shell 脚本完全遵循此逻辑，比如 `ls -l` 是“输入指令”，CPU 执行文件遍历逻辑，最终在终端输出文件列表。

4. Shell 与操作系统的关系：

Shell 是用户与操作系统内核 (Kernel) 之间的“中间层”：用户编写 Shell 脚本 → Shell 解释器 (如 Bash) 解析指令 → 调用内核接口执行 (如读写文件、创建进程) → 返回结果给用户。

1.1.1.1.2 配套示例 (理解执行逻辑)

```
1 #!/bin/bash
2 # 演示 Shell 脚本的“输入-处理-输出”逻辑
3 # 输入：定义变量接收用户输入
4 read -p "请输入一个数字： " num
5
6 # 处理：判断数字奇偶性
7 if (( num % 2 == 0 )); then
8     result="偶数"
9 else
10    result="奇数"
11 fi
12
13 # 输出：打印处理结果
```

14 echo "你输入的数字 \$num 是 \$result"

1.1.1.2 编程语言

1.1.1.2.1 核心知识点

1. 编程语言的本质：

是一套“语法规则 + 语义定义”的集合，用于描述“计算机该做什么、怎么做”。不同编程语言的核心差异在于**设计目标**（如 C 追求效率、Python 追求简洁、Shell 追求系统命令联动）。

2. Shell 语言的核心特点：

特点	说明
解释型	无需编译，由 Bash 逐行解析执行（对比 C 编译型语言）
弱类型	无需声明变量类型（如 <code>a=10</code> 是数值， <code>a="abc"</code> 是字符串）
命令导向	核心是调用 Linux 系统命令（ls、grep、sed 等），而非原生语法
轻量高效	适合短脚本、自动化任务，不适合复杂业务逻辑（如大型应用开发）

3. Shell 与其他主流语言的对比

语言	类型	适用场景	对比 Shell
C	编译型 / 低级语言	操作系统、底层组件	效率极高，但开发复杂；Shell 依赖 C 编写的内核
Python	解释型 / 高级语言	数据分析、Web 开发	语法更优雅，生态更全；Shell 处理系统命令更直接
Java	编译 + 解释型	大型应用、企业级开发	跨平台性强；Shell 无跨平台优势（仅适配 U

4. Shell 的版本与选型：

- 基础 Shell (sh)：UNIX 原生 Shell，语法简单但功能少；
- Bash (Bourne-Again Shell)：Linux 默认 Shell，兼容 sh，扩展了变量、循环、函数等功能（本文核心）；
- Zsh/Fish：增强型 Shell，交互更友好（如自动补全），但脚本兼容性不如 Bash。

1.1.1.2.2 配套示例（Shell 与 Python 实现相同逻辑）

```
1 # Shell 版本：计算1-10的和
2 #!/bin/bash
3 sum=0
4 for ((i=1; i<=10; i++)); do
5     sum=$((sum + i))
6 done
7 echo "Shell计算1-10的和: $sum"
```

```
1 # Python 版本：计算1-10的和（对比参考）
2 sum = 0
3 for i in range(1, 11):
4     sum += i
5 print(f"Python计算1-10的和: {sum}")
```

1.1.1.3 编程语言类型

1.1.1.3.1 核心知识点

编程语言可按 **执行方式**、**抽象层次**、**用途** 分为不同类型，Shell 属于“解释型、高级脚本语言、系统级语言”，以下是核心分类及 Shell 的定位：

1.1.1.3.1.1 按执行方式分类（最核心）

类型	定义	执行流程	代表语言	Shell 归属
编译型	编写完成后需通过编译器转化为机器码（二进制文件），运行时直接执行机器码	源码 → 编译 → 可执行文件 → 运行	C、C++、Go	× 不属于
解释型	无需编译，由解释器逐行解析源码并执行	源码 → 解释器逐行执行	Shell、Python、JavaScript	✓ 典型解释型

关键差异：

- 编译型：一次编译，多次运行（效率高，如 `gcc test.c -o test` 生成可执行文件 `test`）；
- 解释型：每次运行都需解释器（灵活，修改源码后直接运行，无需重新编译）。

1.1.1.3.1.2 按抽象层次分类

类型	定义	特点	代表语言	Shell 归属
低级语言	贴近机器码，直接操作硬件	效率极高，语法晦涩	机器语言、汇编语言	× 不属于
高级语言	贴近人类自然语言，屏蔽硬件细节	易读易写，开发效率高	Shell、Python、Java	✓ 高级语言

1.1.1.3.1.3 按用途分类

类型	定义	代表语言	Shell 归属
系统级语言	面向操作系统管理、硬件交互	C、Shell	✅ 核心用途是系统管理
应用级语言	面向业务开发（Web、APP、数据分析）	Python、Java、JavaScript	× 不适合复杂应用开发
脚本语言	面向自动化任务、批量处理	Shell、Perl、Python	✅ 典型脚本语言

1.1.1.3.2 配套示例（编译型 vs 解释型 对比演示）

步骤 1：编译型语言（C）示例

```
1 // test.c (C语言源码)
2 #include <stdio.h>
3 int main() {
4     int sum = 0;
5     for (int i=1; i<=10; i++) {
6         sum += i;
7     }
8     printf("C语言计算1-10的和: %d\n", sum);
9     return 0;
10 }
```

```
1 # 1. 编译：将C源码转化为可执行文件
2 gcc test.c -o test
3
4 # 2. 运行可执行文件（无需编译器，直接执行）
5 ./test
6
7 # 3. 再次运行（无需重新编译）
8 ./test
```

步骤 2：解释型语言（Shell）示例

```
1 # test.sh (Shell源码)
2 #!/bin/bash
3 sum=0
4 for ((i=1; i<=10; i++)); do
5     sum=$((sum + i))
6 done
7 echo "Shell计算1-10的和: $sum"
```

```
1 # 1. 运行（无需编译，Bash解释器逐行执行）
2 bash test.sh
3
4 # 2. 修改源码（比如计算1-20的和），直接重新运行
5 sed -i 's/10/20/' test.sh
6 bash test.sh
```

步骤 3：核心差异总结

```
1 #!/bin/bash
2 # 对比编译型和解释型的执行效率
3 echo "=== 编译型（C）执行1000次 ==="
4 time for ((i=1; i<=1000; i++)); do ./test; done > /dev/null
5
6 echo -e "\n=== 解释型（Shell）执行1000次 ==="
7 time for ((i=1; i<=1000; i++)); do bash test.sh; done > /dev/null
8
9 # 输出结论
10 echo -e "\n结论：编译型语言执行效率更高，但解释型语言（Shell）开发/修改更灵活，适合快速自动化场景"
```

1.1.2 编程语言逻辑

1.1.2.1 语言分类

语言分类

```
1 低级编程语言：
2  机器：
3    二进制的0和1的序列，称为机器指令。
4    一般人看不懂
5  汇编：
6    用一些助记符号替代机器指令，称为汇编语言。
7    一般人看不懂，但是能够有感觉
8
9  高级编程语言：
10   编译：
11     借助于专属编译器将一些高级语言编译成机器代码文件，然后再交给程序去执行。
12     如：C, C++ 等
13   解释：
14     将高级语言的代码逐行加载到解释器环境，逐行解释为机器代码，然后再交给程序去执行。
15     如：shell, python, php, JavaScript等
```

1.1.2.2 编程逻辑

```
1 | shell 脚本 = Linux 命令 + 逻辑
2 | 编程语言的目的是通过风格化的编程思路将代码写出来后，实现项目功能的。为了实现功能，我们
  | 通过在代码层面通过一些代码逻辑来实现：
3 |
4 | 顺序执行 - 程序按从上到下顺序执行
5 |
6 | 选择执行 - 程序执行过程中，根据条件选择不同的顺序执行
7 |
8 | 循环执行 - 程序执行过程中，根据条件重复执行代码
```

1.2 shell基础

1.2.1 shell简介

1.2.1.1 运维是干什么的

```
1 | 运维是什么？
2 | 所谓的运维，其实就是公司的内部项目当中的一个技术岗位而已，它主要做的是项目的维护性工作。它所涉及的内容范围非常多。
3 |
4 | 以xx项目为例：
5 |
6 | 规划：我们需要多少资源来支持项目的运行
7 | 管理：项目运行过程中的所有内容都管理起来
8 | 流程规范：所有操作都形成制度，提高工作效率
9 | 平台：大幅度提高工作效率
10 | 监控：实时查看项目运行状态指标
11 | 告警：状态指标异常，告知工作人员处理
12 | 安全：网站运营安全措施
13 | 优化：保证用户访问网站体验很好
14 | 考核：权责分配，保证利益
15 |
16 | 自动化运维：就是将图里面所有的工作都使用自动化的方式来实现。
17 | 实现自动化的方式很多，常见的方式：工具和脚本。
18 | 工作中常见的脚本有哪些呢？
19 |     shell脚本 和 其他开发语言脚本
```

1.2.1.2 shell语言

```
1 | 在计算机科学中，Shell 就是一个命令解释器。
2 |
3 | shell 是位于操作系统和应用程序之间，是他们二者最主要的接口，shell 负责把应用程序的输入命令信息解释给操作系统，将操作系统指令处理后的结果解释给应用程序。
4 |
5 | 一句话，shell 就是在操作系统和应用程序之间的一个命令翻译工具。
```

1.2.2 shell 脚本实践

1.2.2.1 脚本基础

shell 脚本

```
1 当可执行的 Linux 命令或语句不在命令行状态下执行，而是通过一个文件执行时，我们称文件为
   shell 脚本。
2
3 重复化、复杂化的工作，通过把工作的命令写成脚本，以后仅仅需要执行脚本就能完成这些工作。
4     自动化分析处理
5     自动化备份
6     自动化批量部署安装
7     等等...
```

1.2.2.2 脚本操作

脚本创建

```
1 脚本创建工具：
2 创建脚本的常见编辑器是 vi/vim。
3
4 脚本命名
5 shell脚本的命名简单来说就是要有意义，方便我们通过脚本名，来知道这个文件是干什么用的。
6
7 脚本内容：
8 各种可以执行的命令
```

1.2.2.3 脚本实践

```
1 15:14:38 root@redis01:~# cat net_info.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: net_info.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 获取ip地址信息
12 ifconfig ens33 | grep -w inet | awk '{print $2}' | xargs echo "IP: "
13 # 获取掩码地址信息
14 ifconfig ens33 | grep -w inet | awk '{print $4}' | xargs echo "NetMask: "
```

```

15 # 获取广播地址信息
16 ifconfig ens33 | grep -w inet | awk '{print $6}' | xargs echo "Broadcast: "
17 # 获取 MAC 地址信息
18 ifconfig ens33 | grep ether | awk '{print $2}' | xargs echo "MAC Address: "
19
20 # 执行脚本
21 15:14:40 root@redis01:~# bash net_info.sh
22 IP: 192.168.121.131
23 NetMask: 255.255.255.0
24 Broadcast: 192.168.121.255
25 MAC Address: 00:0c:29:b4:8d:6a
26 15:14:43 root@redis01:~#

```

1.2.3 脚本注释

注释是 Shell 脚本中对代码逻辑、功能的说明，解释器会忽略注释内容，核心作用是提升脚本可读性、便于维护。Shell 本身仅原生支持**单行注释**，多行注释需通过特殊语法模拟

1.2.3.1 单行注释（核心语法）

1.2.3.1.1 知识点

1. **基本语法**：以 `#` 开头，`#` 后的所有内容均为注释（行内注释需注意位置）；
2. **注意事项**：
 - `#` 必须是行首（或行内非字符串区域），若在字符串（单 / 双引号）内，`#` 会被视为普通字符；
 - `#` 与注释内容间建议加空格（规范，非强制）；
 - 脚本首行的 `#!/bin/bash` 不是注释（Shebang，指定解释器），但 `#!` 仅在第一行有效，第二行及以后的 `#!` 会被视为注释。

1.2.3.1.2 配套代码示例（single_comment.sh）

```

1 #!/bin/bash
2 # 这是单行注释：脚本功能 - 演示单行注释的各种写法
3 # 作者：xxx
4 # 创建时间：2025-12-04
5
6 name="Shell" # 行内注释：定义变量（注意#前加空格，规范写法）
7 echo "Hello $name" # 输出变量值
8
9 # 错误示例：字符串内的#不是注释
10 str="注释测试：# 这不是注释"
11 echo $str # 输出：注释测试：# 这不是注释
12
13 #! 第二行的#!会被视为注释
14 # 注释可以写TODO：TODO 后续优化变量命名规范

```


1.2.3.2 多行注释（模拟实现）

Shell 无原生多行注释语法，需通过 **空命令** `:` + **Here Document (HereDoc)** 模拟，以下是 3 种常用方式（推荐第一种）。

1.2.3.2.1 方式 1: `:<<标记符 注释内容 标记符`（最常用）

1.2.3.2.1.1 知识点

- `:` 是 Shell 空命令（执行后无操作，返回值 0）；
- `<<标记符` 表示开始读取内容，直到遇到相同的 `标记符` 结束（标记符可自定义，如 EOF、!、_ 等）；
- 注释内容中可包含任意字符（包括 #、特殊符号），标记符必须成对且结束标记符需单独一行（无空格）。

```
1 15:24:50 root@redis01:~# :
2 15:24:52 root@redis01:~# echo $?
3 0c
```

1.2.3.2.1.2 配套代码示例 (multi_comment1.sh)

```
1 #!/bin/bash
2 # 演示多行注释（推荐方式）
3 :<<EOF
4 这是多行注释块：
5 1. 脚本用途：批量处理日志文件
6 2. 核心逻辑：
7     - 遍历 /var/log 下的 .log 文件
8     - 压缩7天前的日志
9     - 删除30天前的日志
10 3. 注意事项：执行前需确认日志目录权限
11 EOF
12
13 # 业务代码
14 echo "开始处理日志..."
```

1.2.3.2.2 方式 2: `: 加单行注释堆叠`（简单但不优雅）

1.2.3.2.2.1 知识点

- 每行都以 `#` 开头，适合短注释块；
- 优点：简单直观；缺点：注释 / 取消注释需逐行操作。

1.2.3.2.2.2 配套代码示例 (multi_comment2.sh)

```

1 #!/bin/bash
2 # 多行注释（堆叠式）
3 # 功能：计算1-100的和
4 # 实现逻辑：
5 # 1. 初始化sum=0
6 # 2. for循环遍历1-100
7 # 3. 累加求和并输出
8
9 sum=0
10 for ((i=1; i<=100; i++)); do
11     sum=$((sum + i))
12 done
13 echo "1-100的和: $sum"

```

1.2.3.2.3 方式 3：其他标记符变体（灵活适配场景）

1.2.3.2.3.1 知识点

- 标记符可自定义（如 `!`、`_`、`COMMENT`），避免与注释内容冲突；
- 结束标记符必须**顶行写**（前后无空格、无缩进）。

1.2.3.2.3.2 配套代码示例（multi_comment3.sh）

```

1 #!/bin/bash
2 # 自定义标记符的多行注释
3 :<<!
4 注释内容包含EOF字符：比如"备份文件到 /tmp/EOF/"
5 使用!作为标记符，避免冲突
6 !
7
8 :<<_
9 这是另一个多行注释块
10 标记符为下划线_
11 _
12
13 echo "自定义标记符演示完成"

```

1.2.3.2.4 多行注释注意事项

1. 结束标记符必须单独一行，且**无任何空格 / 缩进**（如下写法错误）：

```

1 :<<EOF
2 错误示例
3 EOF # 缩进导致注释未结束，脚本报错

```

2. 多行注释不能嵌套（如下写法错误）：

```
1      :<<EOF
2      外层注释
3      :<<!
4      内层注释（错误：嵌套不支持）
5      !
6      EOF
```

3. 注释块内避免出现与结束标记符完全一致的内容（会提前终止注释）。

1.2.3.4 注释的最佳实践（规范）

1.2.3.4.1 必加注释的场景

- 脚本开头：注明功能、作者、创建时间、依赖、执行权限；
- 核心逻辑：复杂判断 / 循环、特殊算法的说明；
- 特殊参数：脚本参数的含义、默认值；
- 风险操作：删除 / 修改文件、系统配置变更的提示；
- TODO/FIXME：待优化、待修复的问题。

1.2.3.4.2 规范注释示例（standard_comment.sh）

```
1  #!/bin/bash
2  #####
3  # 脚本名称: log_clean.sh
4  # 功能描述: 自动清理/压缩系统日志，避免磁盘空间耗尽
5  # 作者: xxx
6  # 创建时间: 2025-12-04
7  # 依赖环境: bash 4.0+、gzip、find
8  # 执行权限: 需root用户（或sudo权限）
9  # 使用方式: bash log_clean.sh [日志目录] [保留天数]
10 #          示例: bash log_clean.sh /var/log 30
11 #####
12
13 # 定义默认参数
14 LOG_DIR=${1:-/var/log} # 参数1: 日志目录，默认/var/log
15 KEEP_DAYS=${2:-30}     # 参数2: 保留天数，默认30天
16
17 # 校验参数合法性（核心逻辑注释）
18 if ! [[ $KEEP_DAYS =~ ^[0-9]+$ ]]; then
19     echo "错误: 保留天数必须是数字!"
20     exit 1
21 fi
22
23 # 压缩7天前的日志（风险操作: 需确认文件存在）
24 find $LOG_DIR -name "*.log" -mtime +7 -exec gzip {} \;
25
26 # FIXME: 后续需添加日志压缩后的备份逻辑
```

```
27 | # TODO: 增加邮件通知功能，清理完成后发送报告
28
29 echo "日志清理完成！保留最近${KEEP_DAYS}天的日志"
```

1.2.3.5 注释常见误区

1. 注释与代码脱节：代码修改后未同步更新注释（导致注释误导）；
2. 过度注释：简单逻辑（如 `i=1`）无需注释，避免冗余；
3. 注释内容模糊：如仅写“处理数据”，应明确“处理 /var/log 下的日志数据，过滤空行”；
4. 行内注释滥用：行尾仅注释关键逻辑，避免每行都加行内注释。

1.2.4 脚本执行方式

1.2.4.1 核心概念铺垫

在讲解执行方式前，先明确两个关键概念：

1. **子 Shell**：执行脚本时新建的 Shell 进程，子 Shell 的环境变量 / 修改不会同步到父 Shell；
var="string" 父 shell
var 子 shell
var="aaaa" 子 shell 中修改，父 shell 是否回收到影响
2. **执行权限**：脚本文件需具备 `x`（执行）权限（`chmod +x`），部分方式可绕过；
3. **当前 Shell 环境**：终端当前的进程环境（如已定义的变量、别名等）。

1.2.4.2 方式一：直接执行（赋予执行权限后运行）

1.2.4.2.1 核心语法

```
1 | # 步骤1：赋予执行权限（仅需执行一次）
2 | chmod +x 脚本名.sh
3 |
4 | # 步骤2：执行（需指定路径，相对/绝对均可）
5 | ./脚本名.sh # 相对路径（当前目录）
6 | /绝对路径/脚本名.sh # 绝对路径（如 /root/scripts/test.sh）
```

1.2.4.2.2 案例

```
1 | 15:57:37 root@redis01:~# cat test01.sh
2 | #
   | =====
   | ====
3 | # 脚本基础信息
4 | # filename: test01.sh
5 | # name: xuruizhao
6 | # email: xuruizhao00@163.com
7 | # v: LnxGuru
8 | # GitHub: xuruizhao00-sys
```

```

9  #
=====
====
10 #!/bin/bash
11 # 演示直接执行的特点
12 export TEST_VAR="直接执行-子Shell变量"
13 echo "脚本内的TEST_VAR: $TEST_VAR"
14 echo "当前Shell进程ID (PID): $BASHPID" # $$表示当前进程ID
15
16 15:57:39 root@redis01:~# chmod +x test01.sh
17 15:57:54 root@redis01:~# ls -l test01.sh
18 -rwxr-xr-x 1 root root 486 Dec  4 15:57 test01.sh
19
20 # 直接执行（相对路径）
21 # 输出示例：
22 # 脚本内的TEST_VAR: 直接执行-子Shell变量
23 # 当前Shell进程ID (PID): 12345（随机数，与终端PID不同）
24 15:57:59 root@redis01:~# ./test01.sh
25 脚本内的TEST_VAR: 直接执行-子Shell变量
26 当前Shell进程ID (PID): 17051
27
28 # 验证是否修改当前 Shell 环境
29 # 输出：终端的TEST_VAR: （空，子 Shell 变量不回传）
30 15:58:06 root@redis01:~# echo "终端的TEST_VAR: $TEST_VAR"
31 终端的TEST_VAR:
32 15:58:42 root@redis01:~#
33

```

1.2.4.2.3 适用场景

- 生产环境中独立运行的脚本（如自动化部署、日志清理、定时任务核心脚本）；
- 脚本需要作为可执行程序独立分发 / 运行；
- 希望脚本在子 Shell 中运行，避免污染当前终端环境（如修改环境变量、定义函数）。

1.2.4.2.4 注意事项

- 必须指定路径（`./` 或绝对路径），否则系统会在 `$PATH` 中查找，若当前目录不在 `$PATH` 则报错（如直接输 `exec_direct.sh` 会提示“命令未找到”）；
- 脚本首行 `#!/bin/bash`（Shebang）必须正确，否则可能执行失败（如指定不存在的解释器 `#!/bin/xxx`）；
- 执行权限仅需赋予一次，后续可重复执行。

1.2.4.3 方式二：通过解释器直接执行（bash/sh 脚本.sh）

1.2.4.3.1 核心语法

```

1 bash 脚本名.sh [参数] # 使用Bash解释器（推荐）
2 sh 脚本名.sh [参数] # 使用sh解释器（兼容模式）
3 zsh 脚本名.sh [参数] # 其他Shell解释器（如zsh/ksh）

```

1.2.4.3.2 配套案例

```
1 16:06:05 root@redis01:~# cat test02.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: test02.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 演示解释器执行的特点
12 echo "使用的解释器: $SHELL"
13 echo "脚本参数1: $1" # 接收命令行参数
14 export INTERP_VAR="解释器执行-子Shell变量"
15 echo "脚本内的INTERP_VAR: $INTERP_VAR"
16
17
18 # 1. 无需赋予执行权限, 直接用bash执行 (传参)
19 bash exec_interpreter.sh 测试参数
20 # 输出示例:
21 # 使用的解释器: /bin/bash
22 # 脚本参数1: 测试参数
23 # 脚本内的INTERP_VAR: 解释器执行-子Shell变量
24
25 # 2. 验证当前Shell环境 (变量未同步)
26 echo "终端的INTERP_VAR: $INTERP_VAR"
27 # 输出: 终端的INTERP_VAR: (空)
28
29 # 3. 用sh执行 (兼容模式, 注意Bash扩展语法可能报错)
30 sh exec_interpreter.sh
31 # 若脚本含[[ ]], C风格for循环等Bash扩展语法, sh执行会报错
```

1.2.4.3.3 适用场景

- 脚本未赋予执行权限 (如临时测试、只读文件、共享目录下的脚本) ;
- 需要指定特定解释器运行 (如用 `sh` 兼容模式验证脚本跨 Shell 兼容性) ;
- 开发 / 调试阶段快速测试脚本, 无需频繁修改权限。

1.2.4.3.4 注意事项

- 无需执行权限, 但脚本文件需有**读权限** (`r`) (如 `chmod 400 脚本.sh` 仍可执行) ;
- 若脚本首行 Shebang 与执行时指定的解释器冲突, 以执行时指定的为准 (如 Shebang 是 `#!/bin/sh`, 但用 `bash` 执行则用 Bash) ;

- `sh` 是 Bash 的兼容模式，部分 Bash 扩展语法（如 `[[ ]]`、`for ((i=1; i<=10; i++))`）在 `sh` 下会报错。

1.2.4.3 方式三：source / 点命令执行（source 脚本.sh/. 脚本.sh）

1.2.4.3.1 核心语法

```
1 source 脚本名.sh [参数]    # 方式1（直观）
2 . 脚本名.sh [参数]         # 方式2（点+空格+脚本名，空格不可省略）
```

1.2.4.3.2 配套案例

```
1 16:09:50 root@redis01:~# cat test03.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: test03.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 演示source执行的特点
12 export SOURCE_VAR="source执行-当前Shell变量"
13 echo "脚本内的SOURCE_VAR: $SOURCE_VAR"
14 echo "当前Shell进程ID (PID): $$" # 与终端PID一致
15
16 # 定义一个函数（仅当前Shell有效）
17 hello() {
18     echo "Hello from source script! 参数: $1"
19 }
20
21
22 # 1. 无需执行权限，source执行（传参）
23 source test03.sh 测试参数
24 # 输出示例：
25 # 脚本内的SOURCE_VAR: source执行-当前Shell变量
26 # 当前Shell进程ID (PID): 6789（与终端PID相同）
27
28 # 2. 验证当前Shell环境（变量已生效）
29 echo "终端的SOURCE_VAR: $SOURCE_VAR"
30 # 输出：终端的SOURCE_VAR: source执行-当前Shell变量
31
32 # 3. 调用脚本内定义的函数（已生效）
33 hello 世界
34 # 输出：Hello from source script! 参数: 世界
```

```
35
36 # 4. 用点命令执行（等价source）
37 . exec_source.sh
```

1.2.4.3.3 适用场景

- 需要修改当前 Shell 环境的场景（如定义环境变量、别名、函数）；
- 加载系统 / 项目配置文件（如 `source ~/.bashrc`、`source /etc/profile`）；
- 脚本中的变量 / 函数需要在终端后续操作中使用（如项目启动前的环境配置）；
- 批量设置环境变量（如 `export PATH=/usr/local/bin:$PATH`）。

1.2.4.3.4 注意事项

- 不创建子 Shell，脚本的所有修改（变量、函数、别名）都会直接影响当前终端；
- 无需执行权限，但需读权限；
- 脚本中的 `exit` 命令会直接退出当前终端（而非仅退出脚本），需慎用；
- 路径规则：若脚本不在当前目录，需指定相对 / 绝对路径（如 `source ../config.sh`）

1.2.4.4 方式四：重定向执行（sh < 脚本.sh）

1.2.4.4.1 核心语法

```
1 bash < 脚本名.sh [参数] # Bash解释器
2 sh < 脚本名.sh [参数]   # sh解释器
3
4 # 扩展：远程脚本管道执行（最常用场景）
5 curl -s http://example.com/remote.sh | bash
```

1.2.4.4.2 配套案例

```
1 #!/bin/bash
2 # 演示重定向执行的特点
3 echo "重定向执行-脚本参数1: $1" # 注意：参数无法传递！
4 export REDIRECT_VAR="重定向执行-子Shell变量"
5 echo "脚本内的REDIRECT_VAR: $REDIRECT_VAR"
6
7
8 # 1. 重定向执行（尝试传参，无效）
9 bash < exec_redirect.sh 测试参数
10 # 输出：
11 # 重定向执行-脚本参数1: （空，参数无法传递）
12 # 脚本内的REDIRECT_VAR: 重定向执行-子Shell变量
13
14 # 2. 验证当前环境（变量未同步）
15 echo "终端的REDIRECT_VAR: $REDIRECT_VAR"
16 # 输出：终端的REDIRECT_VAR: （空）
17
18 # 3. 远程脚本管道执行（示例，仅演示语法）
```



```
19 # curl -s https://raw.githubusercontent.com/xxx/test.sh | bash
```

1.2.4.4.3 适用场景

- 脚本文件来自标准输入（如远程下载后直接执行、管道传输的脚本内容）；
- 临时执行无本地文件的脚本（如快速测试网上的脚本片段）；
- 自动化部署中“一键执行”远程脚本（如 `curl | bash`）。

1.2.4.4.4 注意事项

- 无法向脚本传递命令行参数（`$1`、`$2` 等为空）；
- 创建子 Shell，不修改当前环境；
- 脚本首行的 Shebang 会被忽略（解释器由执行时指定的 `bash/sh` 决定）；
- 风险提示：远程脚本管道执行（`curl | bash`）需确认脚本来源安全，避免执行恶意代码

1.2.4.5 方式五：作为函数/别名执行

1.2.4.5.1 核心语法

```
1 # 方式1：将脚本内容封装为函数（当前Shell生效）
2 function 函数名() {
3     # 原脚本核心逻辑
4 }
5
6 # 方式2：别名执行（简化长命令）
7 alias 别名="bash /绝对路径/脚本名.sh [固定参数]"
```

1.2.4.5.2 配套案例

```
1 #!/bin/bash
2 # 定义函数（替代独立脚本文件）
3 log_clean() {
4     # 核心逻辑：清理7天前的日志文件
5     find /var/log -name "*.log" -mtime +7 -delete
6     echo "[$(date +%Y-%m-%d %H:%M:%S)] 日志清理完成！"
7 }
8
9 # 执行函数（当前Shell生效）
10 log_clean
11
12 # 后续终端可重复调用
13 # log_clean
```

```
1 # 1. 定义临时别名（终端关闭后失效）
2 alias clean_log="bash /root/scripts/log_clean.sh"
3
4 # 2. 执行别名
5 clean_log
6
7 # 3. 永久生效：将别名写入 ~/.bashrc
8 echo "alias clean_log='bash /root/scripts/log_clean.sh'" >> ~/.bashrc
9 source ~/.bashrc # 立即生效（无需重启终端）
```

1.2.4.5.3 适用场景

- 脚本逻辑需要频繁复用（如日常运维命令：日志清理、服务重启）；
- 简化长路径 / 复杂参数的脚本执行（如 `alias deploy='bash /data/project/deploy.sh prod'`）；
- 脚本逻辑较短，无需单独文件存储（如几行的小工具）。

1.2.4.5.4 注意事项

- 函数执行在当前 Shell 环境（类似 `source`），变量 / 修改会影响当前终端；
- 别名本质是“解释器执行”的封装，创建子 Shell，不修改当前环境；
- 永久别名需写入 `~/.bashrc`（普通用户）或 `/etc/bashrc`（全局），并 `source` 生效；
- 别名若与系统命令重名，需用 `\命令名` 执行原命令（如 `\ls` 跳过 `ls` 别名）。

1.2.4.6 方式六：定时执行 (crontab)

1.2.4.6.1 核心语法

```
1 # 步骤1: 编辑crontab定时任务
2 crontab -e
3
4 # 步骤2: 添加定时规则（分 时 日 月 周 执行命令）
5 */30 * * * * bash /绝对路径/脚本名.sh > /tmp/脚本日志.log 2>&1
```

1.2.4.7 各执行方式对比 & 选型建议

执行方式	是否需执行权限	是否创建子 Shell	是否修改当前环境	能否传参	核心适用场景
直接执行（./ 脚本.sh）	是	是	否	能	独立运行的生产脚本
解释器执行（bash 脚本）	否	是	否	能	临时测试、指定解释器
source / 点命令	否	否	是	能	加载配置、修改当前环境

执行方式	是否需执行权限	是否创建子 Shell	是否修改当前环境	能否传参	核心适用场景
重定向执行 (bash < 脚本)	否	是	否	否	远程脚本、标准输入执行
函数 / 别名执行	无 (无文件)	函数: 否 / 别名: 是	函数: 是 / 别名: 否	能	频繁复用、简化命令
crontab 定时执行	建议是	是	否	能	周期性后台任务
选型核心原则					

1. 若需修改当前环境 (变量 / 函数 / 别名) → 选 `source` / 函数执行;
2. 若独立运行、不污染环境 → 选 直接执行 / 解释器执行;
3. 若远程 / 无文件执行 → 选 重定向执行 (`curl | bash`);
4. 若周期性任务 → 选 `crontab` 定时执行;
5. 若频繁复用 → 选 函数 / 别名执行。

1.2.5 脚本调试方式

Shell 脚本调试的核心目标是定位**语法错误**、**逻辑错误**、**变量值异常**、**命令执行失败**等问题。根据脚本复杂度和调试需求, 可分为「解释器级调试 (bash 内置参数)」 「脚本内精细化调试」 「实战技巧 / 第三方工具」 三类

1.2.5.1 核心概念

1. **语法错误**: 违反 Shell 语法规则 (如少写 `done`、括号不匹配), 脚本直接无法执行;
2. **逻辑错误**: 语法合法但结果不符合预期 (如循环次数错误、变量值为空);
3. **调试输出**: 通过打印命令、变量、执行流程, 还原脚本运行过程;
4. **静态检查**: 不执行脚本, 提前检测语法 / 规范 / 潜在问题。

1.2.5.2 方式 1: bash 内置调试参数 (最常用)

bash 提供了一系列内置参数, 可直接通过命令行传递, 快速开启调试模式, 无需修改脚本内容。

参数	核心作用	语法示例	适用场景
<code>-n</code>	检查语法错误, 不执行脚本	<code>bash -n script.sh</code>	快速验证脚本语法合法性 (如写完脚本第一步检查)
<code>-x</code>	跟踪执行过程, 打印每一步命令 (展开变量)	<code>bash -x script.sh</code>	定位逻辑错误 (如变量值异常、循环 / 判断逻辑错误)
<code>-v</code>	打印脚本原始内容 + 执行过程 (不展开变量)	<code>bash -v script.sh</code>	确认脚本是否被正确读取 (如路径 / 编码问题)

参数	核心作用	语法示例	适用场景
<code>-e</code>	任意命令执行失败（返回值≠0）立即退出脚本	<code>bash -e script.sh</code>	避免错误扩散（如某步命令失败后，脚本仍继续执行导致更多问题）
<code>-u</code>	使用 未定义变量 时立即报错	<code>bash -u script.sh</code>	定位因变量未定义导致变量为空的错误
<code>-o xtrace</code>	等价 <code>-x</code> ，可结合其他参数（如 <code>-o errexit</code> 等价 <code>-e</code> ）	<code>bash -o xtrace -o errexit script.sh</code>	多参数组合调试

1.2.5.2.1 基础参数案例（单参数调试）

1.2.5.2.1.1 -n 检查语法错误☆☆☆

```
1 16:31:30 root@redis01:~# cat test1.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: test1.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 语法错误: for循环缺少done
12 sum=0
13 for ((i=1; i<=10; i++)) {
14     sum=$((sum + i))
15 # 缺少done
16 echo "1-10的和: $sum"
17
18 16:31:31 root@redis01:~# bash -n test1.sh
19 # 输出报错（定位语法错误行）:
20 test1.sh: line 17: syntax error: unexpected end of file
21 16:31:36 root@redis01:~#
22
```

1.2.5.2.1.2 -x 跟踪执行过程☆☆☆

```
1 16:33:30 root@redis01:~# cat debug.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: debug.sh
```

```
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
=====
====
10 #!/bin/bash
11 # 计算1-10的偶数和
12 sum=0
13 for ((i=1; i<=10; i++)); do
14     if ((i % 2 == 0)); then
15         sum=$((sum + i))
16     fi
17 done
18 echo "1-10偶数和: $sum"
19
20 # 执行调试 (-x 会打印每一步执行的命令和变量值):
21 16:33:33 root@redis01:~# bash -x debug.sh
22 + sum=0
23 + (( i=1 ))
24 + (( i<=10 ))
25 + (( i % 2 == 0 ))
26 + (( i++ ))
27 + (( i<=10 ))
28 + (( i % 2 == 0 ))
29 + sum=2
30 + (( i++ ))
31 + (( i<=10 ))
32 + (( i % 2 == 0 ))
33 + (( i++ ))
34 + (( i<=10 ))
35 + (( i % 2 == 0 ))
36 + sum=6
37 + (( i++ ))
38 + (( i<=10 ))
39 + (( i % 2 == 0 ))
40 + (( i++ ))
41 + (( i<=10 ))
42 + (( i % 2 == 0 ))
43 + sum=12
44 + (( i++ ))
45 + (( i<=10 ))
46 + (( i % 2 == 0 ))
47 + (( i++ ))
48 + (( i<=10 ))
49 + (( i % 2 == 0 ))
50 + sum=20
51 + (( i++ ))
52 + (( i<=10 ))
```

```

53 + (( i % 2 == 0 ))
54 + (( i++ ))
55 + (( i<=10 ))
56 + (( i % 2 == 0 ))
57 + sum=30
58 + (( i++ ))
59 + (( i<=10 ))
60 + echo 1-10偶数和: 30
61 1-10偶数和: 30
62

```

关键：通过 `-x` 可清晰看到循环次数、变量 `i` 和 `sum` 的变化，快速定位逻辑错误（如循环条件写错、判断条件错误）。

1.2.5.2.1.3 -e 命令失败立即退出

```

1 16:35:13 root@redis01:~# cat debug.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: debug.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 演示-e参数: 执行错误命令后立即退出
12 ls /xxx # 不存在的目录, 执行失败
13 echo "这行不会执行" # 因-e参数, 上一步失败后脚本直接退出
14 16:35:14 root@redis01:~# bash -e debug.sh
15 # 输出:
16 # ls: 无法访问 '/xxx': 没有那个文件或目录
17 # (无后续echo输出, 脚本已退出)
18 ls: cannot access '/xxx': No such file or directory
19 16:35:18 root@redis01:~#

```

1.2.5.2.1.4 -u 未定义变量报错

```

1 16:36:38 root@redis01:~# cat debug.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: debug.sh

```

```

5  # name: xuruizhao
6  # email: xuruizhao00@163.com
7  # v: LnxGuru
8  # GitHub: xuruizhao00-sys
9  #
   =====
   ====
10 #!/bin/bash
11 # 演示-u参数: 使用未定义变量
12 echo "未定义变量: $undefined_var"
13 echo "这行不会执行"
14
15 16:36:39 root@redis01:~# bash -u debug.sh
16 debug.sh: line 11: undefined_var: unbound variable
17 16:36:44 root@redis01:~#
18

```

情况	<code>bash -u</code> 能报错吗?
未定义变量	✅ 会报错
已定义但为空变量	❌ 不会报错

1.2.5.2.2 组合参数调试

多个参数组合可覆盖更多调试场景，比如：

```

1  # 组合: 语法检查(-n) + 执行失败退出(-e) + 未定义变量报错(-u) + 跟踪执行(-x)
2  bash -n -e -u -x script.sh
3
4  # 等价简写 (-o 方式)
5  bash -o errexit -o nounset -o xtrace script.sh

```

1.2.5.3 方式 2：脚本内精细化调试 (set 指令)

当脚本较长时，全局 `-x` 会输出大量日志，可通过 `set` 指令在脚本内**局部开启 / 关闭调试**，只调试关键代码段。

1.2.5.3.1 核心语法

```

1  set -x  # 开启调试 (等价 bash -x)
2  set +x  # 关闭调试
3  set -e  # 开启“命令失败立即退出”
4  set +e  # 关闭“命令失败立即退出”
5  set -u  # 开启“未定义变量报错”
6  set +u  # 关闭“未定义变量报错”

```

1.2.5.3.2 配套案例

```
1 16:40:24 root@redis01:~# cat debug.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: debug.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 脚本内局部调试示例
12 echo "这部分不调试"
13
14 # 开启调试（只调试循环部分）
15 set -x
16 sum=0
17 for ((i=1; i<=5; i++)); do
18     sum=$((sum + i))
19 done
20 set +x # 关闭调试
21
22 echo "调试结束，总和: $sum"
23 echo "这部分也不调试"
24
25 16:40:25 root@redis01:~# bash debug.sh
26 这部分不调试
27 + sum=0
28 + (( i=1 ))
29 + (( i<=5 ))
30 + sum=1
31 + (( i++ ))
32 + (( i<=5 ))
33 + sum=3
34 + (( i++ ))
35 + (( i<=5 ))
36 + sum=6
37 + (( i++ ))
38 + (( i<=5 ))
39 + sum=10
40 + (( i++ ))
41 + (( i<=5 ))
42 + sum=15
43 + (( i++ ))
44 + (( i<=5 ))
```



```
45 + set +x
46 调试结束，总和：15
47 这部分也不调试
48
```

1.2.5.3.3 进阶：自定义调试输出格式

通过 `PS4` 变量可自定义 `-x` 的调试前缀，显示行号、函数名等，便于定位问题：

```
1 16:41:46 root@redis01:~# cat debug.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: debug.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 自定义调试输出格式：显示行号+命令
12 export PS4='[行号:$LINENO] '
13 set -x
14
15 sum=0
16 for ((i=1; i<=3; i++)); do
17     sum=$((sum + i))
18 done
19 set +x
20
21 echo "总和：$sum"
22 16:41:48 root@redis01:~# bash debug.sh
23 [行号:14] sum=0
24 [行号:15] (( i=1 ))
25 [行号:15] (( i<=3 ))
26 [行号:16] sum=1
27 [行号:15] (( i++ ))
28 [行号:15] (( i<=3 ))
29 [行号:16] sum=3
30 [行号:15] (( i++ ))
31 [行号:15] (( i<=3 ))
32 [行号:16] sum=6
33 [行号:15] (( i++ ))
34 [行号:15] (( i<=3 ))
35 [行号:18] set +x
36 总和：6
```

1.2.5.4 方式 3：实战调试技巧（低成本高效）

对于复杂脚本，结合以下技巧可快速定位问题，无需依赖复杂工具。

1.2.5.4.1 关键位置打印变量 / 状态

在脚本的判断、循环、函数前后，手动添加 `echo` / `printf` 打印变量值、命令返回值，是最基础且有效的调试方式。

```

1  16:44:03 root@redis01:~# cat debug.sh
2  #
   =====
   ====
3  # 脚本基础信息
4  # filename: debug.sh
5  # name: xuruizhao
6  # email: xuruizhao00@163.com
7  # v: LnxGuru
8  # GitHub: xuruizhao00-sys
9  #
   =====
   ====
10 #!/bin/bash
11 # 手动打印变量调试
12 read -p "请输入数字: " num
13
14 # 打印输入的变量值（验证是否正确接收）
15 echo "【调试】输入的num值: $num, 类型: $(file -b <(echo $num))"
16
17 # 校验是否为数字
18 if ! [[ $num =~ ^[0-9]+$ ]]; then
19     echo "错误: 输入不是数字"
20     exit 1
21 fi
22
23 # 打印计算过程
24 sum=$((num * 2))
25 echo "【调试】num*2的结果: $sum"
26
27 echo "最终结果: $sum"
28 16:44:06 root@redis01:~# bash debug.sh
29 请输入数字: 4
30 【调试】输入的num值: 4, 类型: symbolic link to pipe:[96197]
31 【调试】num*2的结果: 8
32 最终结果: 8
33 16:44:10 root@redis01:~#
  
```

1.2.5.4.2 捕获错误信号 trap

通过 `trap` 捕获脚本执行中的错误（`ERR` 信号），自动打印出错行、变量值，无需手动加 `echo`。

1.2.5.4.2.1 trap 测试

```
1 16:51:41 root@redis01:~# cat debug.sh
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: debug.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 #!/bin/bash
11 # 捕获ERR信号，打印出错信息
12 trap 'echo "【错误】行号$LINENO执行失败！当前变量: num=$num"; exit 1' ERR
13
14 num="abc" # 非数字，导致后续计算失败
15 ((sum = num * 2)) # 这行会触发ERR信号
16 echo "不会执行到这里"
17
18 16:51:42 root@redis01:~# bash debug.sh
19 【错误】行号14执行失败！当前变量: num=abc
20 16:51:43 root@redis01:~#
```

1.2.5.4.2.2 ERR 信号触发规则

`ERR` 信号仅在“简单命令”执行失败（返回值 $\neq 0$ ）时触发，而 `${(...)}` 属于「参数扩展」（不是独立的“简单命令”），其内部语法错误不会改变整个命令的返回码，也不会触发 `ERR`。

语法类型	是否为“简单命令”	错误是否返回非 0	是否触发 ERR
<code>\${(num*2)}</code> （算术扩展）	否（参数扩展）	否（脚本继续执行）	× 不触发
<code>let num*2</code> （let 命令）	是（独立命令）	是（返回非 0）	✅ 触发
<code>((num*2))</code> （算术命令）	是（独立命令）	是（返回非 0）	✅ 触发

```
1 16:55:57 root@redis01:~# cat debug.sh
```

```

2  #
   =====
   ====
3  # 脚本基础信息
4  # filename: debug.sh
5  # name: xuruizhao
6  # email: xuruizhao00@163.com
7  # v: LnxGuru
8  # GitHub: xuruizhao00-sys
9  #
   =====
   ====
10 #!/bin/bash
11 num="a"
12 # 1. 算术扩展：错误但返回码为0
13 sum=$((num*2))
14 echo $?
15
16 # 2. let 命令：错误且返回码非0
17 let sum=sum*2
18 echo $?
19
20 # 3. 算术命令：错误且返回码非0
21 ((sum = sum * 2))
22 echo $?
23 16:55:59 root@redis01:~# bash debug.sh
24 0
25 1
26 1
27

```

1.2.5.4.2.3 ERR 信号的其他触发注意事项

即使使用了“简单命令”，以下场景也可能导致 `ERR` 信号不触发，需额外注意：

- 错误命令在 `if/while/until` 条件中
- 未开启 `set -o errexit` (或 `set -e`)
 - `ERR` 信号的触发不依赖 `set -e`，但 `set -e` 会让脚本在命令失败时立即退出，与 `trap ERR` 结合更稳妥
- 管道命令的失败（默认只检测最后一个命令）
 - 管道中前面的命令失败，默认不会触发 `ERR`（如 `ls /xxx | grep a`，`ls` 失败但 `grep` 成功，整体返回码为 0）：

1.2.5.4.3 重定向调试输出到文件

当 `-x` 输出过多时，可将调试日志重定向到文件，避免终端刷屏，便于后续分析。

```

1 # 方式1: 命令行重定向
2 bash -x debug_x.sh > debug.log 2>&1
3
4 # 方式2: 脚本内重定向 (仅调试输出到文件)
5 #!/bin/bash
6 exec 2> debug.log # 将标准错误 (-x 的调试输出) 重定向到文件
7 set -x
8 sum=0
9 for ((i=1; i<=10; i++)); do
10     sum=$((sum + i))
11 done
12 set +x
13 echo "总和: $sum"

```

1.2.5.5 方式 4: 第三方工具调试 (复杂脚本)

对于逻辑复杂的脚本 (如多函数、多分支、嵌套循环), 可使用专业调试工具。

1.2.5.5.1 shellcheck: 静态语法 / 规范检查 (推荐)

1.2.5.5.1.1 作用

检测脚本中的语法错误、潜在问题 (如未定义变量、无用代码、不符合规范的写法), 给出修复建议。

1.2.5.5.1.2 安装 (Linux)

```

1 # CentOS/RHEL
2 # 安装 EPEL 源 (Rocky 8/9 通用)
3 dnf install epel-release -y
4 # 更新 dnf 缓存 (确保能识别新仓库的包)
5 dnf makecache
6 dnf install shellcheck -y
7
8 # 如果是 rocky8+ 系统在安装时可能会出现如下报错
9 Last metadata expiration check: 0:00:53 ago on Thu 11 Dec 2025 11:33:10 AM
10 CST. Error: Problem: conflicting requests - nothing provides libffi.so.6()
11 (64bit) needed by ShellCheck-0.6.0-3.el8.x86_64 from epel (try to add '--
12 skip-broken' to skip uninstallable packages or '--nobest' to use not only
13 best candidate packages)
14
15 这是由于 libffi.so.6()(64bit) 依赖缺失, 是因为 Rocky Linux 8 后期版本 / 9 版本
16 中, 系统默认的 libffi 库版本升级为 libffi.so.7 (或更高), 但 EPEL 仓库中适配 el8
17 的 ShellCheck 0.6.0 版本仍依赖旧版的 libffi.so.6, 导致依赖冲突
18
19 # 我们可以直接在 GitHub 中下载预编译的二进制包
20 scversion="stable" # or "v0.4.7", or "latest"
21 wget -qO-
22 "https://github.com/koalaman/shellcheck/releases/download/${scversion?}/sh
23 ellcheck-${scversion?}.linux.x86_64.tar.xz" | tar -xJv

```

```

15 cp "shellcheck-`${scversion}`/shellcheck" /usr/bin/
16 shellcheck --version
17
18 # Ubuntu/Debian
19 apt update
20 apt install -y shellcheck
21
22 # macOS
23 brew install shellcheck

```

1.2.5.5.1.3 案例（检测脚本问题）

准备有问题的脚本

```

1 #!/bin/bash
2 # 存在的问题：未定义变量、无用的cd命令、双引号缺失
3 cd /tmp
4 echo $undefined_var
5 ls $HOME/*.log # 无双引号，若文件名含空格会出错
6
7 shellcheck bad_script.sh
8 # 输出示例（给出问题和修复建议）：
9 # In bad_script.sh line 5:
10 # echo $undefined_var
11 #      ^-----^ SC2154: undefined_var is referenced but not assigned.
12 #
13 # In bad_script.sh line 4:
14 # cd /tmp
15 # ^-- SC2164: Use 'cd ... || exit' or 'cd ... || return' in case cd fails.
16 #
17 # In bad_script.sh line 6:
18 # ls $HOME/*.log
19 #      ^-----^ SC2086: Double quote to prevent globbing and word
    splitting.

```

1.2.5.6 总结

调试场景	推荐方式	核心优势
快速检查语法错误	<code>bash -n + shellcheck</code>	不执行脚本，快速定位语法问题
定位逻辑错误（变量 / 循环 / 判断）	<code>bash -x + 脚本内 set -x/+x</code>	跟踪执行过程，还原变量变化
避免错误扩散	<code>bash -e + trap ERR</code>	出错立即退出，自动打印错误信息
变量相关问题	<code>bash -u + 手动 echo 变量</code>	定位未定义 / 空变量导致的问题

调试场景	推荐方式	核心优势
复杂脚本（多函数 / 分支）	<code>bashdb</code> 断点调试	精细化控制执行流程，查看调用栈
规范 / 潜在问题	<code>shellcheck</code>	给出修复建议，提升脚本健壮性

1.2.5.7 调试最佳实践

1. **先静态后动态**：先用 `bash -n` + `shellcheck` 检查语法 / 规范，再用 `-x` 跟踪执行；
2. **局部调试**：长脚本用 `set -x/+x` 只调试关键段，避免日志冗余；
3. **关键节点打印**：在判断、循环、函数前后打印变量 / 状态，快速定位问题；
4. **重定向调试日志**：复杂脚本将 `-x` 输出重定向到文件，便于复盘；
5. **避免调试残留**：调试完成后，删除 / 注释掉脚本内的 `set -x`、`echo` 等调试代码；
6. **结合返回值**：通过 `echo $?` 打印命令返回值，定位命令执行失败的原因（如 `ls /xxx; echo $?`）。

1.2.6 脚本开发规范

工业级标准：可维护 + 可扩展 + 安全

1.2.6.1 基础规范

1.2.6.1.1 Shebang 规范

- **规则：**
 1. Shebang (`#!/bin/bash`) 必须是脚本**第一行**，行首无任何空格 / 制表符 / 空行 / 注释；
 2. 明确指定解释器（优先用 `#!/bin/bash`，而非 `#!/bin/sh`，避免兼容问题）；
 3. 禁止在 Shebang 行尾加注释（ShellCheck 会报 SC1015 警告）。

1.2.6.1.2 文件名规范

- **强制规则：**
 1. 小写字母 + 下划线 (`_`) 组合，禁止大写 / 空格 / 特殊字符（如 `-` 尽量避免，易与参数混淆）；
 2. 后缀统一为 `.sh`（便于识别，非强制但推荐）；
 1. windows.txt.md.py
 2. linux.xxx demo01.py bash demo01.py
 3. 命名体现脚本功能（如 `log_clean.sh` 而非 `test1.sh`）；
 4. 批量脚本按模块前缀分类（如 `db_backup_mysql.sh`、`db_backup_postgres.sh`）。
- **正确示例：** `system_monitor.sh`、`file_batch_rename.sh`
- **错误示例：** `SystemMonitor.sh`、`文件备份.sh`、`test.sh`

1.2.6.1.3 文件权限规范

- **强制规则：**

1. 执行脚本权限设为 `755` (`chmod 755 script.sh`)，普通脚本文件设为 `644`；
2. 禁止设置 `777` 权限（任意用户可修改，极高安全风险）；
3. 脚本所属用户 / 组明确（生产环境建议归 `root` 或专用运维用户）。

```
1 | chmod 755 /usr/local/bin/log_clean.sh
2 | chown root:root /usr/local/bin/log_clean.sh
```

1.2.6.1.4 编码与换行规范

- **强制规则：**

1. 编码统一为 `UTF-8`（避免中文注释乱码）；
2. 换行符用 Unix 格式（`LF`），禁止 Windows 格式（`CRLF`，会导致 `\r` 语法错误）；
3. 脚本末尾保留一个空行（符合 POSIX 规范，避免部分工具解析异常）。

```
1 | # 检查换行符（CRLF会显示^M）
2 | cat -v script.sh
3 | # 修正为LF换行符
4 | dos2unix script.sh
```

1.2.6.2 文件结构规范（模块化，易维护）

规范的脚本需按「固定顺序」组织内容，便于快速定位逻辑。推荐结构如下：

```
1 | #!/bin/bash
2 | #####
3 | # 1. 头部注释（脚本元信息）
4 | #####
5 | # 脚本名称: log_clean.sh
6 | # 功能描述: 自动清理/压缩系统日志，监控磁盘使用率
7 | # 作者: xxx
8 | # 创建时间: 2025-12-05
9 | # 版本: v1.2
10 | # 依赖环境: bash 4.0+、gzip、find、df
11 | # 执行权限: 需root/sudo权限
12 | # 使用方式: bash log_clean.sh [日志目录] [保留天数]
13 | #           示例: bash log_clean.sh /var/log 30
14 | # 注意事项: 执行前需确认日志目录无重要临时文件
15 | #####
16 |
17 | #####
18 | #####
```



```

18 # 2. 环境检查（依赖/权限/系统）
19 #####
20 # 检查解释器版本
21 if [[ ${BASH_VERSION%%.*} -lt 4 ]]; then
22     echo "错误：需要bash 4.0及以上版本"
23     exit 1
24 fi
25
26 # 检查执行用户
27 if [[ $UID -ne 0 ]]; then
28     echo "错误：需root用户执行"
29     exit 1
30 fi
31
32 # 检查依赖命令
33 dep_commands=("gzip" "find" "df")
34 for cmd in "${dep_commands[@]"; do
35     if ! command -v $cmd &>/dev/null; then
36         echo "错误：缺少依赖命令 $cmd，请先安装"
37         exit 1
38     fi
39 done
40
41 #####
42 # 3. 全局变量定义（大写+下划线，便于区分）
43 #####
44 # 默认参数
45 LOG_DIR=${1:-/var/log}
46 KEEP_DAYS=${2:-30}
47 LOG_FILE="/var/log/log_clean.log"
48 TIMESTAMP=$(date +%Y-%m-%d %H:%M:%S)
49
50 #####
51 # 4. 函数定义（单一职责，命名见名知意）
52 #####
53 # 日志输出函数
54 log_info() {
55     echo "[${TIMESTAMP}] [INFO] $1" >> $LOG_FILE
56     echo "[${TIMESTAMP}] [INFO] $1"
57 }
58
59 log_error() {
60     echo "[${TIMESTAMP}] [ERROR] $1" >> $LOG_FILE
61     echo "[${TIMESTAMP}] [ERROR] $1"
62     exit 1

```

```

63 }
64
65 # 清理日志函数
66 clean_old_logs() {
67     log_info "开始清理日志：目录=$LOG_DIR，保留天数=$KEEP_DAYS"
68     # 压缩7天前的日志
69     find $LOG_DIR -name "*.log" -type f -mtime +7 -exec gzip {} \;
70     if [[ $? -ne 0 ]]; then
71         log_error "压缩日志失败"
72     fi
73     # 删除30天前的压缩日志
74     find $LOG_DIR -name "*.log.gz" -type f -mtime +$KEEP_DAYS -delete
75     if [[ $? -ne 0 ]]; then
76         log_error "删除日志失败"
77     fi
78     log_info "日志清理完成"
79 }
80
81 # 磁盘监控函数
82 check_disk_usage() {
83     disk_usage=$(df -h $LOG_DIR | grep -v Filesystem | awk '{print $5}' |
84 sed 's/%//g')
85     if [[ $disk_usage -gt 80 ]]; then
86         log_error "磁盘使用率超标：$disk_usage%（阈值80%）"
87     fi
88     log_info "磁盘使用率：$disk_usage%"
89 }
90 #####
91 # 5. 主逻辑（调用函数，简洁清晰）
92 #####
93 main() {
94     # 校验参数合法性
95     if ! [[ $KEEP_DAYS =~ ^[0-9]+$ ]]; then
96         log_error "保留天数必须是数字：$KEEP_DAYS"
97     fi
98     if ! [[ -d $LOG_DIR ]]; then
99         log_error "日志目录不存在：$LOG_DIR"
100     fi
101
102     # 执行核心逻辑
103     check_disk_usage
104     clean_old_logs
105
106     log_info "脚本执行完成"
107     exit 0
108 }
109

```

```

110 # 调用主函数
111 main
112
113 #####
114 # 6. 收尾清理（可选，如临时文件删除）
115 #####
116 # trap 'rm -f /tmp/tmp.log' EXIT

```

1. 头部注释：必须包含「功能、作者、版本、依赖、使用方式」，便于他人快速理解；
2. 环境检查：先校验依赖 / 权限 / 参数，避免脚本执行中报错；
3. 变量 / 函数：全局变量大写（区分局部变量），函数单一职责（如 `clean_old_logs` 只做日志清理）；
4. 主逻辑：通过 `main` 函数统一调用，避免代码平铺直叙。

1.2.6.3 编码规范（统一语法，降低出错率）

1.2.6.3.1 变量规范

- **命名：**
 - 全局变量：大写 + 下划线（如 `LOG_DIR`）；
 - 局部变量：小写 + 下划线（如 `disk_usage`）；
 - 禁止单字符变量（除循环变量 `i/j` 外），禁止拼音 / 无意义命名（如 `a=10`）。
- **使用：**
 - 变量引用必须加 `${}`（避免歧义，如 `${name}s` 而非 `$names`）；
 - 字符串变量必须用双引号包裹（避免空格 / 通配符解析错误，如 `"$LOG_DIR"`）；
 - 禁止使用未定义变量（开启 `set -u` 强制检查）。

```

1 # 正确示例
2 user_name="张三"
3 echo "欢迎 ${user_name} 使用脚本" # 加{}，双引号包裹
4
5
6 # 错误示例
7 username=张三
8 echo 欢迎 $username使用脚本 # 无{}，易歧义；无双引号，空格会截断

```

1.2.6.3.2 注释规范

- **强制规则：**
 1. 脚本头部必须有元信息注释（见文件结构示例）；
 2. 函数 / 核心逻辑必须加注释（说明功能，而非重复代码）；
 3. 特殊逻辑（如正则、复杂判断）加行内注释；
 4. 禁止冗余注释（如 `i=1 # 定义变量i`）；
 5. 多行注释用 `:<<EOF ... EOF`，禁止逐行 `#` 堆叠（长注释块）。

```

1 # 计算磁盘使用率（核心逻辑：过滤df输出，提取百分比并去掉%）
2 disk_usage=$(df -h $LOG_DIR | grep -v Filesystem | awk '{print $5}' | sed
  's/%//g')

```

1.2.6.3.3 缩进与格式

• 强制规则：

1. 缩进用 4 个空格（禁止 Tab，避免不同编辑器显示不一致）；
2. 条件 / 循环语句的 `then/do` 单独一行（或加空格分隔）；
3. 管道命令多行书写时，`|` 放在行尾，便于阅读；
4. 长命令用 `\` 换行（每行不超过 80 字符）。

```

1 # 条件语句（then单独一行，缩进4空格）
2 if [[ $disk_usage -gt 80 ]]; then
3     log_error "磁盘使用率超标"
4 fi
5
6 # 管道命令多行书写
7 find $LOG_DIR \
8     -name "*.log" \
9     -type f \
10    -mtime +7 \
11    -exec gzip {} \;

```

1.2.6.3.4 条件 / 循环规范

• 条件判断：

1. 优先用 `[[ ]]` 代替 `[ ]`（支持正则、`&&/||`，无需转义）；
2. 数值判断用 `(( ))`（如 `((i > 10))`），禁止用 `>/<`（易被解析为重定向）；
3. 文件判断优先用 `-f/-d/-e` 等，避免直接 `ls` 判断。

• 循环语句：

1. 遍历数组用 `for item in "${arr[@]}"`（双引号包裹，避免空格分割）；
2. 禁止无限循环无退出条件（必须有 `break` / 超时控制）。

```

1 # 条件判断
2 if [[ $user_name =~ ^[a-zA-Z0-9_]+$ ]]; then # 正则匹配
3     echo "用户名合法"
4 fi
5
6 # 数值判断
7 if (( count >= 10 )); then
8     break
9 fi
10
11 # 数组遍历
12 fruits=("苹果" "香蕉" "橙子")

```

```

13 for fruit in "${fruits[@]"}; do
14     echo $fruit
15 done

```

1.2.6.3.5 算术运算规范

- 优先用 `$(( ))` (如 `$((a + b))`) 或 `let` (如 `let sum+=1`) ;
- 浮点数运算用 `bc/awk` (Shell 不原生支持) ;
- 运算前必须校验变量是否为数值 (避免 `aa.sh` 这类非数值导致语法错误) 。

```

1 num=10
2 sum=$((num * 2)) # 整数运算
3 float_sum=$(echo "scale=2; $num * 3.14" | bc) # 浮点数运算 (保留2位

```

1.2.6.4 安全规范

1.2.6.4.1 权限控制

- 禁止用 `root` 执行非必要脚本 (最小权限原则) ;
- 脚本中涉及文件修改 / 删除时, 校验文件路径 (防止路径遍历, 如 `../` 注入) ;
- 禁止硬编码密码 / 密钥 (通过环境变量 / 配置文件传入, 且配置文件权限设为 `600`) 。

1.2.6.4.2 输入校验

- 所有外部输入 (参数、用户输入、环境变量) 必须校验合法性:
 - 路径参数校验: `[[ $path =~ ^/[a-zA-Z0-9_/>+$ ]]` ;
 - 数值参数校验: `[[ $num =~ ^[0-9/>+$ ]]` ;
 - 禁止输入包含 `;/|/&` 等特殊字符 (避免命令注入) 。

```

1 # 校验路径参数, 防止路径遍历
2 if [[ $LOG_DIR =~ \.\. ]]; then
3     log_error "非法路径: 包含../"
4 fi

```

1.2.6.4.3 危险命令管控

- 禁止使用 `rm -rf /*` (必须指定具体目录, 如 `rm -rf $LOG_DIR/*.log.gz`) ;
- `rm` 命令前先校验目录是否存在 (避免误删) ;
- 禁止使用 `eval` (易导致命令注入, 如 `eval "$user_input"`) ;
- 管道执行远程脚本 (`curl | bash`) 必须校验来源 (如 MD5 校验) 。

1.2.6.4.4 错误处理

- 开启 `set -e` (命令失败立即退出)、`set -u` (未定义变量报错)、`set -o pipefail` (管道错误检测) ;
- 用 `trap` 捕获 `ERR/INT/EXIT` 信号 (如捕获 Ctrl+C, 清理临时文件) ;
- 所有命令执行后检查返回码 (`$?`) , 失败时输出日志并退出。

```

1 # 开启安全参数
2 set -euo pipefail
3
4 # 捕获信号：退出时清理临时文件
5 trap 'rm -f /tmp/tmp_log*; log_info "脚本被中断，清理临时文件"' INT EXIT

```

1.2.6.4.5 输出重定向

- 禁止将错误输出重定向到 `/dev/null`（除非明确确认无需关注）；
- 脚本执行日志必须落地（便于问题排查）；
- 敏感操作（如删除文件）执行前输出提示（或加 `-f` 参数确认）

二、shell 变量

2.1 变量基础

2.1.1 变量场景

2.1.1.1 数据存储

```

1 所谓的数据存储，我们从三方面来理解这句话：
2
3 1、数据保存到哪里 -- 各种媒介，CPU、内存、磁盘、磁带、网盘...
4
5 2、数据保存的效果 -- 完整、安全、有效
6
7 3、数据保存的单元 -- 存储空间
8
9 数据的存储空间默认不是一个整体，而是由一个个的存储单元组成，每一个存储单元都有一个唯一的
  整数编号，我们称这个编号为：地址
10
11 存储单元的作用：存储数据+读写数据
12
13 存储空间大小：1字节(B) = 8bit == 00000000
14
15 地址特点：十六进制，例如：0x20000000

```

2.1.1.2 存储样式

对于数据的存储来说，主要有两种样式：物理地址和逻辑地址。

```

1 物理地址：内存或硬盘中真正存储数据的位置，也就是说通过磁盘设备查找的位置
2
3 逻辑地址：用于查找物理地址的存储块地址叫逻辑地址。程序中用的地址一般都是逻辑地址
4
5 逻辑地址包括两部分：起始值(十六进制)+偏移量(十六进制)
6
7 数据表的描述主要是逻辑地址，因为程序一般存储的是逻辑地址。

```

数据一旦存储下来就不再发生变化了，而程序中可能会在很多场景中用到同一个数据，就会出现两个问题：

```
1 1 物理地址人听得懂，机器看不懂
2
3 - 所以用逻辑地址找物理地址
4
5 2 软件可以通过逻辑地址找到数据地址，但是软件不懂场景
6
7 - 所以人用逻辑地址的别名来代指向同一个xx地址
8
9 应用程序中为了 多场景应用这个逻辑地址的别名，本质上就是“变量”。
```

2.1.1.3 变量本质

变量的本质其实就是 通过一个名称帮助程序快速找到内存中具体数据的地址。

- 变量说白了就是指向xx值。
变量存在的意义是，脚本文件执行的时候，能够让脚本逻辑不变化的前提下，可以根据实际的数据进行相同逻辑的处理，从而满足当前业务需求

2.1.1.4 编程语言中的变量使用

```
1 静态编译语言：
2
3 使用变量前，先声明变量类型，之后类型不能改变，在编译时检查。
4
5 如： java, c
6
7 动态编译语言：
8
9 不用事先声明，可随时改变类型。
10
11 如： bash, Python
12
13
14 强类型语言：
15
16 不同类型数据操作，必须经过强制转换才同一类型才能运算。
17
18 如java , c# , python
19
20 示例：
21
22     print('shuji' + 10) 提示出错，不会自动转换类型
23
24     print('shuji' + str(10)) 结果为magedu10，需要显示转换类型
25
```

```
26 弱类型语言：
27
28 语言的运行时会隐式做数据类型转换。无须指定类型，默认均为字符型；
29
30 参与运算会自动进行隐式类型转换；变量无须事先定义可直接调用。
31
32 如：bash ， php, javascript
33
34 示例：
35
36 echo 'aaa'+222
```

2.1.2 变量定义

Shell 脚本中的变量是存储数据的容器，用于在脚本运行过程中传递和操作信息。与编译型语言不同，Shell 中的变量**无需显式声明类型**，其值始终为字符串（即使看起来像数字），但可参与算术运算。

2.1.2.1 变量定义

在 Shell 中，变量通过 `变量名=值` 的形式定义。**注意：等号两侧不能有空格**，否则会被解释为命令。

✅ 2.1.2.1.1 正确定义方式

```
1 name="Alice"
2 age=25
3 site=github.com
```

✗ 2.1.2.1.2 错误示例（会导致语法错误）

```
1 name = "Alice"
2 # 错误：等号两边有空格，Shell 会尝试执行名为 "name" 的命令
```

🔍 2.1.2.1.3 使用变量

使用 `$变量名` 或 `${变量名}` 引用变量值。推荐在复杂上下文中使用花括号以避免歧义。

```
1 echo ${name}
2 echo $age
```

💡 提示：`${}` 在拼接字符串时特别有用：

```
1 file="log"
2 echo "${file}_2025.txt" # 输出：log_2025.txt
```


2.1.2.1.4 只读变量（常量）

```
1 readonly PI=3.14159
2 # PI=3.14 # 执行会报错: readonly variable
```

使用 `readonly` 声明变量后，其值不可修改。

2.1.2.1.5 删除变量

使用 `unset` 可删除变量（不能删除 `readonly` 变量）。

```
1 unset age
2 echo $age # 输出为空
```

2.1.2.2 变量命名规范

2.1.2.2.1 变量命名规则

Shell 变量名的字符和格式有严格限制，是定义变量的第一要求：

规则	说明	正确示例	错误示例	错误原因
字符限制	仅允许「字母（大小写）、数字、下划线（_）」	<code>name</code> 、 <code>USER_AGE</code> 、 <code>var123</code>	<code>user-name</code> 、 <code>var@123</code>	含非法字符（-、@）
开头限制	不能以数字开头	<code>var1</code> 、 <code>_var</code>	<code>1var</code> 、 <code>123_name</code>	数字开头，Shell 解析为命令 / 数值
长度限制	无官方长度限制（建议≤64 字符）	<code>LOG_DIR</code> 、 <code>file_count</code>	超长变量名（如 <code>a1234567890123456789012345678901234567890123456789012345678901234567890</code> ）	可读性差，无语法错误但不推荐
关键字禁止	不能使用 Shell 关键字（如 <code>if/for/while/echo</code> 等）	<code>for_var</code>	<code>for</code> 、 <code>i</code>	

```
1 #!/bin/bash
2 # 正确命名
3 name="张三"
4 USER_AGE=25
5 var123="test"
6 _var="下划线开头"
7
8 # 错误命名（取消注释查看报错）
9 # user-name="李四" # 含-，报错：-bash: user-name=李四：未找到命令
10 # 1var="数字开头" # 报错：-bash: 1var=数字开头：未找到命令
11 # for="关键字" # 报错：-bash: for=关键字：语法错误
```

2.1.2.2.2 赋值语法规则

变量赋值的核心是 `变量名=值`，等号的处理直接决定是否报错

规则	说明	正确示例	错误示例	错误原因
等号无空格	<code>=</code> 前后不能有空格（空格会被 Shell 解析为命令分隔）	<code>age=20</code>	<code>age = 20</code>	Shell 把 <code>age</code> 当作命令， <code>=</code> 当作参数，报错「age: 未找到命令」

禁止多变量连续赋值	不推荐 <code>a=b=c=10</code> （虽能执行，但易引发逻辑问题）	<code>a=10; b=10; c=10</code>	<code>a=b=c=10</code>	实际是 <code>c=10</code> ， <code>b=c</code> ， <code>a=b</code> ，最终 <code>a</code> 值为 <code>c</code> （非 10）
空值赋值	允许定义空变量	<code>empty var=</code> 、 <code>empty var=""</code>	-	空变量可后续赋值，引用时需注意（开启 <code>set</code>

规则	说明	正确示例	错误示例	错误原因
赋值与引用分离	赋值时变量名不加\$，引用时必须加\$（或\${}） 取消变量定义时也不需要加\$	num=10; echo \$num	\$num=10	赋值时加\$会解析为“未定义变量”，报错 [=10: 未找到命令]

```
1 #!/bin/bash
2 # 正确赋值
3 age=20
4 empty_var="" # 空值赋值
5 echo "age=$age, empty_var=$empty_var"
6
7 # 错误赋值（取消注释查看报错）
8 # age = 20 # 等号有空格，报错
9 # $num=10 # 赋值加$，报错
10 # a=b=c=10 # 连续赋值（逻辑错误）
11 # echo "a=$a, b=$b, c=$c" # 输出a=c, b=c, c=10
```

2.1.2.2.3 变量值的处理规则

规则	说明	正确示例	错误示例	错误原因
含空格 / 通配符需加引号	值含空格、*、?、[]等，必须用单 / 双引号包裹	path="/tmp/test dir"、 pattern="*.sh"	path=/tmp/test dir	Shell 把 dir 当作独立命令，报错 [dir: 未找到命令]
单 / 双引号区别	双引号解析变量，单引号原样输出（无变量解析）	name="张三"; str="hello \$name"	str='hello \$name'	单引号内 \$name 不解析，输出 hello \$name（非语法错误，按需选择）
特殊字符转义	若不用引号，需用 \ 转义特殊字符（\$、&、空格等）	str=hello\ \$world	str=hello \$world	\$world 被解析为未定义变量，输出 hello

规则	说明	正确示例	错误示例	错误原因
多行值处理	变量值需换行时，用 <code>\n</code> 转义或 HereDoc， Shell 默认忽略换行	<code>multi_line="line1\nline2"</code>	<code>multi_line="line1 line2"</code>	直接换行 赋值会报错，需转义或 HereDoc

```

1 #!/bin/bash
2 # 正确：值含空格加双引号
3 path="/tmp/test dir"
4 echo "path=$path"
5
6 # 正确：特殊字符转义
7 str=hello\ $world
8 echo "str=$str" # 输出hello $world
9
10 # 错误：值含空格不加引号（取消注释报错）
11 # path=/tmp/test dir # 报错：dir: 未找到命令
12
13 # 多行值处理
14 multi_line="line1\nline2"
15 echo -e "$multi_line" # -e解析\n，输出两行

```

2.1.2.3 特殊类型变量的定义要求

除基础变量外，只读、环境、数组、局部变量的定义有额外规则：

2.1.2.3.1 只读变量 (readonly/declare -r)

- 规则 1：定义后**不可修改、不可撤销** (unset 无效) ；
- 规则 2：可直接定义 `readonly 变量名=值`，或先定义再标记 `var=10; readonly var`；
- 规则 3：禁止重复定义只读变量（重复定义会报错）。

```

1 #!/bin/bash
2 # 正确定义只读变量
3 readonly PI=3.14
4 # 或
5 var=10
6 declare -r var # 等价readonly
7
8 # 错误操作（取消注释报错）
9 # PI=3.1415 # 只读变量无法修改，报错
10 # unset PI # 只读变量无法撤销，报错
11 # readonly PI=3.14 # 重复定义，报错

```

2.1.2.3.2 环境变量 (export)

- 规则 1: 需用 `export` 声明, 才能被子 Shell 继承 (未 `export` 仅当前 Shell 有效);
- 规则 2: 可「先定义后 `export`」或「定义 + `export` 一步到位」;
- 规则 3: 环境变量名建议大写 (规范), 如 `export APP_VERSION="2.0"`。

```
1 #!/bin/bash
2 # 正确定义环境变量
3 app_name="myapp"
4 export app_name # 方式1
5
6 export app_version="2.0" # 方式2 (推荐)
7
8 # 验证: 子Shell可访问环境变量
9 bash -c 'echo "子Shell中app_name=$app_name"' # 输出myapp
```

2.1.2.3.3 数组变量

(1) 索引数组 (默认, bash 所有版本支持)

- 规则 1: 用 `()` 包裹值, 值之间用空格分隔 (索引从 0 开始);
- 规则 2: 禁止值含空格不加引号 (会被解析为多个元素);
- 规则 3: 追加元素用 `数组名[${#数组名[@}]=新值`。

(2) 关联数组 (bash 4.0+ 支持)

- 规则 1: 必须先声明 `declare -A 数组名`, 再赋值 (键值对);
- 规则 2: 键名可含数字 / 字符串, 无需连续;
- 规则 3: CentOS 6 等低版本 bash 不支持, 需升级。

```
1 #!/bin/bash
2 # 索引数组 (正确)
3 fruits=("苹果" "香蕉" "橙子") # 值含空格加引号
4 echo "fruits[1]=${fruits[1]}" # 输出香蕉
5
6 # 关联数组 (正确)
7 declare -A user_info
8 user_info["name"]="张三"
9 user_info["age"]=25
10
11 # 错误: 关联数组未声明 (取消注释报错)
12 # user_info2["name"]="李四" # bash4+报错, 低版本解析为索引数组
```

2.1.2.3.3 局部变量

- 规则 1: 仅能在**函数内**定义, 用 `local 变量名=值`;
- 规则 2: 局部变量仅函数内生效, 函数外无法访问;

- 规则 3: `local` 必须放在函数内第一行（赋值前），否则先赋值为全局变量，再标记局部（逻辑错误）。

```
1 #!/bin/bash
2 # 正确定义局部变量
3 func() {
4     local local_var="局部变量" # 函数内第一行定义
5     echo "函数内: $local_var"
6 }
7
8 func
9 echo "函数外: $local_var" # 输出空（无法访问）
10
11 # 错误: local位置错误（取消注释验证）
12 # func2() {
13 #     var="全局变量"
14 #     local var="局部变量" # 先赋值全局，再标记局部
15 #     echo "函数内: $var" # 输出局部变量，但全局var已被赋值
16 # }
17 # func2
18 # echo "函数外: $var" # 输出全局变量（逻辑错误）
```

2.1.2.4 变量定义的禁止行为（常见陷阱）

禁止行为	示例	后果
变量名含空格 / 特殊符号	<code>user name="张三"、 var@123=test</code>	语法错误，报错「未找到命令」
赋值时变量名加\$	<code>\$num=10</code>	解析为“未定义变量”，报错「=10: 未找到命令」
只读变量重复定义 / 修改	<code>readonly PI=3.14; PI=3.1415</code>	报错「只读变量无法修改」
函数外使用 <code>local</code>	<code>local var=10</code>	语法错误，报错「local: 仅在函数内有效」
未转义的\$直接赋值	<code>str=hello\$world</code>	<code>\$world</code> 被解析为未定义变量，最终 <code>str=hello</code>
数组元素含空格不加引号	<code>arr=(test dir)</code>	被解析为两个元素（test、dir），而非一个元素（test dir）

2.1.2.5 变量定义的最佳实践（规范层面）

1. 命名规范：

- 全局变量 / 环境变量：大写 + 下划线（如 `LOG_DIR`、`APP_VERSION`）；
- 局部变量 / 函数内变量：小写 + 下划线（如 `file_count`、`user_name`）；
- 数组变量：复数名词（如 `fruits`、`users`）；
- 禁止单字符变量（循环变量 `i/j` 除外）、禁止拼音 / 无意义命名（如 `a=10`）。

2. 安全规范：

- 所有外部输入的变量（参数、用户输入）必须校验合法性（数值、路径等）；
- 路径变量校验是否含 `../`（防止路径遍历）；
- 开启 `set -u`（未定义变量报错），提前暴露未定义变量问题。

3. 可读性规范：

- 变量名见名知意（如 `LOG_DIR` 而非 `dir`）；
- 复杂值赋值时换行（如数组、多行字符串），提升可读性；
- 变量定义集中放在脚本开头（全局变量）或函数开头（局部变量）。

2.1.3 变量分类

Shell 变量的分类可以从**作用域 / 生命周期**、**定义主体**、**数据类型**等维度划分，其中**按作用域分类**是最常用、最核心的方式。

2.1.3.1 按作用域 / 生命周期分类（核心）

2.1.3.1.1 局部变量 (Local Variable)

- **定义**：仅在**当前 shell 进程**或**函数内部**有效，子进程 / 子 shell 无法访问，退出当前 shell / 函数后立即销毁。
- **声明方式**：普通 shell 中直接赋值（默认当前 shell 局部）；函数内需用 `local` 关键字显式声明（否则会升级为当前 shell 的全局变量）。

```
1 15:33:45 root@redis01:~# cat local_variable.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: local_variable.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 普通 shell 的局部变量，只对当前 shell 有效
16 var="local variable"
17 echo ${var}
18
19 # 函数内部的局部变量（仅在函数内部有效）
20 test_func(){
21     local func_inner_var="func_inner"
22     echo ${func_inner_var}
23 }
```

```

22 test_func
23
24 # 无输出，函数外不可见
25 echo ${func_inner_var}
26
27 15:33:46 root@redis01:~# bash local_variable.sh
28 local variable
29 func_inner

```

- **查看方式：** `set` 命令（列出所有局部变量 + 环境变量）。

2.1.3.1.2 环境变量 (Environment Variable)

- **定义：** 全局有效，当前 shell 及所有子 shell / 子进程（如 bash、python 脚本、其他命令）均可访问。
- **声明方式：** 通过 `export` 关键字导出（临时生效），或写入配置文件持久化。

```

1 15:44:47 root@redis01:~# cat enviroment_variable.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: enviroment_variable.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 临时导出环境变量
16 export USER_NAME="zhangsan"
17 echo ${USER_NAME}
18
19 # 子 shell 中访问
20 /bin/bash -c 'echo ${USER_NAME}'
21
22 15:44:49 root@redis01:~# bash enviroment_variable.sh
23 zhangsan
24 zhangsan
25 15:44:51 root@redis01:~#

```

常用系统内置环境变量：

- `PATH`：命令搜索路径（执行 `ls` 时系统查找的目录）；
- `HOME`：当前用户主目录（如 `/home/zhangsan`）；
- `SHELL`：当前使用的 shell（如 `/bin/bash`）；

- **LANG**：语言环境（如 `zh_CN.UTF-8`）；
 - **PWD**：当前工作目录（等价于 `pwd` 命令）
 - **查看方式**：`env/printenv/export` 命令。
- 持久化**：临时 `export` 仅当前 shell 有效，需写入配置文件：
- 当前用户：`~/.bashrc` 或 `~/.bash_profile`；
 - 所有用户：`/etc/profile` 或 `/etc/environment`；
 - 生效：`source ~/.bashrc`（立即生效）或重启终端。

2.1.3.1.3 位置参数变量

- **定义**：shell 预定义的、用于接收**传递给脚本 / 函数的参数**的变量。

变量	含义说明
<code>\$0</code>	表示脚本本身的名称（若通过软链接执行， <code>\$0</code> 显示软链接名）
<code>\$1~\$9</code>	第 1 到第 9 个位置参数（传入的第 1 个参数是 <code>\$1</code> ，第 2 个是 <code>\$2</code> ，依此类推）
<code>\${10}</code>	第 10 及以上位置参数（必须用花括号包裹，否则 <code>\$10</code> 会被解析为 <code>\$1 + 0</code> ）
<code>\$#</code>	传入的位置参数的 总个数
<code>\$*</code>	所有位置参数的集合（加引号时 <code>"\$"</code> 会将所有参数拼接为一个字符串）
<code>@</code>	所有位置参数的集合（加引号时 <code>"@"</code> 会保留每个参数的独立性，推荐优先用）
<code>?</code>	上一个命令 / 脚本的退出状态码（0 = 成功，非 0 = 失败，常用来判断参数处理结果）

2.1.3.1.3.1 示例

```

1  15:50:38 root@redis01:~# cat test.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: test.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 # 打印脚本名称
12 echo "脚本名称: $0"
13
14 # 打印前3个位置参数
15 echo "第1个参数: $1"
16 echo "第2个参数: $2"
```

```

17 echo "第3个参数: $3"
18
19 # 打印参数总个数
20 echo "参数总个数: $#"

```

2.1.3.1.3.2 *和@ 的核心区别 (重点)

表面上 `$*` 和 `$@` 输出相同, 但**加引号后** 行为完全不同:

- `"$*"`: 将所有参数拼接成**一个字符串** (用系统默认分隔符 `IFS`, 默认是空格);
- `"$@"`: 将每个参数视为**独立的个体** (保留参数边界, 适合循环处理)。

```

1 15:51:33 root@redis01:~# cat test.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo "=== 循环 \$* (加引号) ==="
16 for arg in "$*"; do
17     echo "参数: $arg"
18 done
19
20 echo -e "\n=== 循环 \$@ (加引号) ==="
21 for arg in "$@"; do

```

```

18     echo "参数: $arg"
19 done
20
21 15:51:34 root@redis01:~# bash test.sh 苹果 "香蕉 葡萄" 橙子
22 === 循环 $* (加引号) ===
23 参数: 苹果 香蕉 葡萄 橙子
24
25 === 循环 $@ (加引号) ===
26 参数: 苹果
27 参数: 香蕉 葡萄
28 参数: 橙子

```

2.1.3.1.3.3 处理超过 9 个参数

\$10 会被 Shell 解析为 $\$1 + 0$ ，因此第 10 个及以上参数必须用 `${数字}` 形式：

```

1 15:53:08 root@redis01:~# cat test01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo "第10个参数: ${10}"
16 echo "第11个参数: ${11}"
17
18 15:53:11 root@redis01:~# bash test01.sh 1 2 3 4 5 6 7 8 9 10 11
19 第10个参数: 10
20 第11个参数: 11
21 15:53:18 root@redis01:~#

```

2.1.3.2 按照定义主体分

2.1.3.2.1 用户自定义变量

用户手动创建的变量，用于临时存储数据

```

1 age=25
2 name="Tom"
3 arr=(a b c)
4 export str2="s2"

```

2.1.3.2.2 系统内置变量

shell / 操作系统预定义的变量，用于存储系统配置、环境信息，比如：

- `PATH`、`HOME`、`SHELL`（环境变量）；
- `$?` 、 `$$` （特殊变量）。

2.1.3.3 按数据类型分类（Shell 弱类型，无严格类型区分）

2.1.3.3.1 字符串变量（最常用）

- **单引号**：纯字符串，不解析变量 / 转义符（强引用）：

```
1 15:56:15 root@redis01:~# name="zhangsan"
2 15:56:21 root@redis01:~# name_ex='xixi and $name'
3 15:56:48 root@redis01:~# echo $name_ex
4 xixi and $name
5 15:56:54 root@redis01:~#
```

- **双引号**：解析变量（弱引用）

```
1 15:56:54 root@redis01:~# name="zhangsan"
2 15:57:41 root@redis01:~# name_ex="xixi and $name"
3 15:57:46 root@redis01:~# echo $name_ex
4 xixi and zhangsan
5 15:57:47 root@redis01:~#
```

- **无引号**：简单字符串，含空格 / 特殊字符需加引号：

```
1 15:57:47 root@redis01:~# str=hello
2 15:58:26 root@redis01:~# str=hello world
3 Command 'world' not found, but can be installed with:
4 snap install world
5 15:58:32 root@redis01:~#
```

2.1.3.3.2 数值变量

本质是字符串，需通过 `let`、`(( ))`、`expr` 等工具进行数值运算：

```
1 num=10
2 let num=num+5 # num=15
3 echo $((num*2)) # 输出: 30
4 expr $num + 3 # 输出: 18
```

2.1.3.3 数组变量

索引从 0 开始，可存储多个字符串：

```
1 arr=(apple banana orange)
2 echo ${arr[0]}      # 输出: apple (第1个元素)
3 echo ${arr[@]}      # 输出: apple banana orange (所有元素)
4 echo ${#arr[@]}     # 输出: 3 (数组长度)
```

2.1.4 变量基本操作

2.1.4.1 变量查看

在 Shell 中，查看变量的目的是确认变量的**值**、**属性**（如类型、只读、环境变量等）或**所有已定义变量**，常用方法如下：

方法	用途	适用场景
<code>echo \$变量名</code>	查看单个变量的值	快速验证单个变量赋值结果
<code>printf "%s\n" \$变量名</code>	格式化查看变量值（避免特殊字符问题）	变量值含空格 / 特殊符号时
<code>set</code>	查看所有变量（包括本地 / 环境 / 函数）	排查所有已定义变量
<code>env / export</code>	查看环境变量（全局可访问）	确认环境变量是否生效
<code>declare -p 变量名</code>	查看变量的 值 + 属性 （类型 / 只读等）	调试变量属性问题

```
1 16:06:12 root@redis01:~# cat show_variable.sh
2 #!/bin/bash
3 #
4 =====
5 # 脚本基础信息
6 # filename: show_variable.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 # 先定义测试变量
14 name="张三"
15 age=25
16 export PATH=$PATH:/tmp # 环境变量示例
17
18 # 1. 基础查看: echo
19 echo "echo 查看单个变量: "
```

```
18 echo "name = $name"
19 echo "age = $age"
20 echo "-----"
21
22 # 2. 格式化查看: printf (处理含空格/特殊字符的变量)
23 address="北京市 朝阳区"
24 echo -e "\nprintf 格式化查看: "
25 printf "address = %s\n" "$address"
26 echo "-----"
27
28 # 3. set 查看所有变量 (本地+环境+函数)
29 # 注: 输出内容较多, 可结合 grep 过滤指定变量
30 echo -e "\nset 过滤查看 name 变量: "
31 set | grep "^name="
32 echo "-----"
33
34 # 4. env/export 查看环境变量
35 echo -e "\nexport 查看 PATH 环境变量 (截取末尾): "
36 export | grep "^PATH=" | awk -F'/' {print $1 "/tmp..."}'
37 echo "-----"
38
39 # 5. declare -p 查看变量值+属性 (最推荐的调试方式)
40 echo -e "\ndeclare -p 查看变量属性: "
41 declare -p name age address
42
43 16:06:17 root@redis01:~# bash show_variable.sh
44 echo 查看单个变量:
45 name = 张三
46 age = 25
47 -----
48
49 printf 格式化查看:
50 address = 北京市 朝阳区
51 -----
52
53 set 过滤查看 name 变量:
54 name=张三
55 -----
56
57 export 查看 PATH 环境变量 (截取末尾):
58 -----
59
60 declare -p 查看变量属性:
61 declare -- name="张三"
62 declare -- age="25"
63 declare -- address="北京市 朝阳区"
```

2.1.4.2 变量定义

Shell 变量默认是**字符串类型**，定义的核心规则是「变量名 = 值」，无类型限制（除非用 `declare` 声明）。

- 1. 变量名：只能包含**字母、数字、下划线**，且**不能以数字开头**（区分大小写）；
- 2. 赋值符号 `=` 两边**不能有空格**（空格会被 Shell 解析为命令分隔符）；
- 3. 变量值含**空格 / 特殊字符**时，需用单引号 `' '` 或双引号 `" "` 包裹；
- 4. 引用变量：用 `$变量名` 或 `${变量名}`（推荐后者，避免变量名拼接歧义）。

赋值方式 (4 种常用)

赋值方式	语法示例	说明
直接赋值	<code>var=value</code>	最简单的赋值，值为固定字符串
命令替换赋值	<code>var=\$(命令)</code> 或 <code>var= 命令</code>	变量值为命令的执行结果
算术赋值	<code>var=\$((算术表达式))</code>	整数运算（仅支持整数，浮点数需用 <code>bc</code> ）
交互赋值	<code>read [-p 提示语] var</code>	从用户输入获取变量值

```
1 16:08:26 root@redis01:~# cat define_variable.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: define_variable.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 #!/bin/bash
16 # 1. 直接赋值（核心规则演示）
17 var1=hello          # 正确：无空格
18 # var2 = world      # 错误：=两边有空格（运行会报错）
19 var3="hello world"   # 含空格，双引号包裹
20 var4='hello $var1'   # 单引号：原样输出（不解析变量）
21 var5="hello $var1"   # 双引号：解析变量
22
23 echo "1. 直接赋值： "
24 echo "var1 = $var1"
25 echo "var3 = $var3"
26 echo "var4 = $var4"  # 输出: hello $var1
27 echo "var5 = $var5"  # 输出: hello hello
```

```

24 echo "-----"
25
26 # 2. 命令替换赋值 ($( ) 推荐, `` 兼容旧版本)
27 current_dir=$(pwd)      # 获取当前目录
28 file_count=$(ls | wc -l) # 统计当前目录文件数
29 echo -e "\n2. 命令替换赋值: "
30 echo "当前目录: $current_dir"
31 echo "当前目录文件数: $file_count"
32 echo "-----"
33
34 # 3. 算术赋值 ($( ( )) 仅支持整数)
35 a=10
36 b=3
37 sum=$((a + b))
38 sub=$((a - b))
39 mul=$((a * b))
40 div=$((a / b)) # 整数除法, 结果取整
41 mod=$((a % b))
42 echo -e "\n3. 算术赋值: "
43 echo "$a + $b = $sum"
44 echo "$a - $b = $sub"
45 echo "$a * $b = $mul"
46 echo "$a / $b = $div" # 输出: 3
47 echo "$a % $b = $mod" # 输出: 1
48 echo "-----"
49
50 # 4. 交互赋值 (read)
51 echo -e "\n4. 交互赋值: "
52 read -p "请输入你的姓名: " username
53 read -p "请输入你的年龄: " userage
54 echo "你输入的姓名: $username, 年龄: $userage"
55
56 16:08:29 root@redis01:~# bash define_variable.sh
57 1. 直接赋值:
58 var1 = hello
59 var3 = hello world
60 var4 = hello $var1
61 var5 = hello hello
62 -----
63
64 2. 命令替换赋值:
65 当前目录: /root
66 当前目录文件数: 8
67 -----
68
69 3. 算术赋值:
70 10 + 3 = 13
71 10 - 3 = 7
72 10 * 3 = 30
73 10 / 3 = 3

```



```
74 10 % 3 = 1
75 -----
76
77 4. 交互赋值：
78 请输入你的姓名：xu
79 请输入你的年龄：22
80 你输入的姓名：xu，年龄：22
```

2.1.4.3 declare 语法

`declare` 是 Shell 内置命令，用于**声明变量类型 / 修改变量属性**，解决默认字符串类型的限制（如整数、数组、只读、环境变量等）。

```
1 declare [选项] 变量名[=值]
```

选项	作用	示例
<code>-i</code>	声明变量为 整数类型 （自动算术运算）	<code>declare -i num=10</code>
<code>-r</code>	声明变量为 只读变量 （不可修改 / 删除）	<code>declare -r readonly_var=test</code>
<code>-a</code>	声明变量为 索引数组 （默认数组类型）	<code>declare -a arr=(1 2 3)</code>
<code>-A</code>	声明变量为 关联数组 （键值对）	<code>declare -A dict=([name]=张三 [age]=25)</code>
<code>-x</code>	声明变量为 环境变量 （等同于 <code>export</code> ）	<code>declare -x env_var=hello</code>
<code>-p</code>	查看变量的 值 + 属性 （无参数时查看所有）	<code>declare -p num</code>
<code>-u</code>	变量值转为 大写 （赋值时自动转换）	<code>declare -u str=hello</code>
<code>-l</code>	变量值转为 小写 （赋值时自动转换）	<code>declare -l str=HELLO</code>

- 只读变量（`-r`）一旦声明，无法修改值、无法用 `unset` 删除，仅能通过重启 Shell 清除；
- 整数变量（`-i`）赋值非数字时，值会被设为 0；
- 关联数组（`-A`）仅支持 Bash 4.0+，需确认 Bash 版本（`bash --version`）。

2.1.4.4 declare 实践

```
1 16:08:35 root@redis01:~# vim define_variable_2.sh
2 16:11:30 root@redis01:~# cat define_variable_2.sh
3 #!/bin/bash
4 #
5 =====
6 =====
7 # 脚本基础信息
8 # filename: define_variable_2.sh
9 # name: xuruizhao
10 # email: xuruizhao00@163.com
```

```
9  # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
=====
=====
12 #!/bin/bash
13 # 1. -i: 声明整数变量（自动算术运算）
14 echo "1. 声明整数变量（-i）："
15 declare -i num1=10
16 declare -i num2=5
17 num1=num1 + num2 # 整数类型可直接算术赋值（无需 $(( ))）
18 echo "num1 = $num1" # 输出: 15
19 num3=abc          # 赋值非数字，值为 0
20 declare -i num3
21 echo "num3 = $num3" # 输出: 0
22 echo "-----"
23
24 # 2. -r: 声明只读变量
25 echo -e "\n2. 声明只读变量（-r）："
26 declare -r readonly_var="只读变量"
27 echo "readonly_var = $readonly_var"
28 # readonly_var=修改 # 运行会报错: readonly variable
29 # unset readonly_var # 运行会报错: cannot unset: readonly variable
30 echo "-----"
31
32 # 3. -a: 声明索引数组
33 echo -e "\n3. 声明索引数组（-a）："
34 declare -a arr=(apple banana cherry)
35 echo "数组第1个元素: ${arr[0]}" # 索引从0开始
36 echo "数组所有元素: ${arr[@]}" # 输出: apple banana cherry
37 echo "数组长度: ${#arr[@]}" # 输出: 3
38 # 追加元素
39 arr[3]="date"
40 echo "追加后数组: ${arr[@]}" # 输出: apple banana cherry date
41 echo "-----"
42
43 # 4. -A: 声明关联数组（键值对）
44 echo -e "\n4. 声明关联数组（-A）："
45 declare -A user_info=( [name]="王五" [age]=28 [city]="上海" )
46 echo "姓名: ${user_info[name]}" # 输出: 王五
47 echo "年龄: ${user_info[age]}" # 输出: 28
48 echo "所有键: ${!user_info[@]}" # 输出: name age city
49 echo "所有值: ${user_info[@]}" # 输出: 王五 28 上海
50 echo "-----"
51
52 # 5. -x: 声明环境变量（等同于 export）
53 echo -e "\n5. 声明环境变量（-x）："
54 declare -x env_test="环境变量测试"
55 echo "env_test = $env_test"
56 # 验证是否为环境变量
```

```
57 export | grep "^env_test=" # 输出: env_test=环境变量测试
58 echo "-----"
59
60 # 6. -u/-l: 大小写转换
61 echo -e "\n6. 大小写转换 (-u/-l): "
62 declare -u upper_str="hello world"
63 echo "大写转换: $upper_str" # 输出: HELLO WORLD
64 declare -l lower_str="HELLO SHELL"
65 echo "小写转换: $lower_str" # 输出: hello shell
66 echo "-----"
67
68 # 7. -p: 查看变量属性
69 echo -e "\n7. 查看变量属性 (-p): "
70 declare -p num1 readonly_var arr user_info
71
72 16:11:32 root@redis01:~# bash define_variable_2.sh
73 1. 声明整数变量 (-i):
74 define_variable_2.sh: line 15: +=: command not found
75 num1 = 10
76 num3 = abc
77 -----
78
79 2. 声明只读变量 (-r):
80 readonly_var = 只读变量
81 -----
82
83 3. 声明索引数组 (-a):
84 数组第1个元素: apple
85 数组所有元素: apple banana cherry
86 数组长度: 3
87 追加后数组: apple banana cherry date
88 -----
89
90 4. 声明关联数组 (-A):
91 姓名: 王五
92 年龄: 28
93 所有键: city age name
94 所有值: 上海 28 王五
95 -----
96
97 5. 声明环境变量 (-x):
98 env_test = 环境变量测试
99 -----
100
101 6. 大小写转换 (-u/-l):
102 大写转换: HELLO WORLD
103 小写转换: hello shell
104 -----
105
106 7. 查看变量属性 (-p):
```

```

107 declare -i num1="10"
108 declare -r readonly_var="只读变量"
109 declare -a arr=([0]="apple" [1]="banana" [2]="cherry" [3]="date")
110 declare -A user_info=([city]="上海" [age]="28" [name]="王五" )

```

2.1.4.5 变量移除

当变量不再使用时，可通过 `unset` 命令移除变量（释放内存），核心是删除变量的定义和值。

```
1 unset [选项] 变量名/数组名
```

1. `unset` 可移除普通变量、数组、环境变量，但**无法移除只读变量**；
2. 移除数组时：`unset 数组名` 移除整个数组，`unset 数组名[索引]` 移除指定元素；
3. 移除环境变量后，该变量仅在当前 Shell 失效，已经存在的子 Shell 不受影响，新建的子 shell 无法获取。

```

1 bash new
2 bash export a=13 father shell crontab unset age
3 |
4 |
5 \
6 bash .....
7 \
8 bash son shell 当前位置
9
10 # crontab
11 crontab -e
12 * * * * * /bin/bash unset age
13 # 使用文件控制变量值
14 echo 12 > /tmp/age
15
16 while true;do
17     if [[ -f /tmp/age ]];then
18         export age=$(cat /tmp/age)
19     else
20         age=""
21     done
22
23 * * * * * /bin/bash rm -f /tmp/age

```

```

1 16:13:22 root@redis01:~# cat unset_variable.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: unset_variable.sh

```

```

6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 #!/bin/bash
12 # 1. 移除普通变量
13 echo "1. 移除普通变量: "
14 test_var="要移除的变量"
15 echo "移除前: test_var = $test_var"
16 unset test_var
17 echo "移除后: test_var = $test_var"  # 输出为空
18 echo "-----"
19
20 # 2. 移除数组（整体/单个元素）
21 echo -e "\n2. 移除数组: "
22 declare -a fruit=(apple banana cherry)
23 echo "移除前数组: ${fruit[@]}"
24 # 移除单个元素（索引1）
25 unset fruit[1]
26 echo "移除索引1后: ${fruit[@]}"  # 输出: apple cherry
27 # 移除整个数组
28 unset fruit
29 echo "移除整个数组后: ${fruit[@]}"  # 输出为空
30 echo "-----"
31
32 # 3. 移除环境变量
33 echo -e "\n3. 移除环境变量: "
34 declare -x env_var="测试环境变量"
35 echo "移除前: env_var = $env_var"
36 unset env_var
37 echo "移除后: env_var = $env_var"  # 输出为空
38 # 验证环境变量是否存在
39 export | grep "^env_var="  # 无输出
40 echo "-----"
41
42 # 4. 尝试移除只读变量（报错）
43 echo -e "\n4. 尝试移除只读变量: "
44 declare -r readonly_var="只读变量"
45 echo "readonly_var = $readonly_var"
46 unset readonly_var  # 运行会报错: cannot unset: readonly variable
47
48 16:13:23 root@redis01:~# bash unset_variable.sh
49 1. 移除普通变量:
50 移除前: test_var = 要移除的变量
51 移除后: test_var =
52 -----
53

```

```
54 2. 移除数组:
55 移除前数组: apple banana cherry
56 移除索引1后: apple cherry
57 移除整个数组后:
58 -----
59
60 3. 移除环境变量:
61 移除前: env_var = 测试环境变量
62 移除后: env_var =
63 -----
64
65 4. 尝试移除只读变量:
66 readonly_var = 只读变量
67 unset_variable.sh: line 45: unset: readonly_var: cannot unset: readonly
variable
```

2.2 全局变量

2.2.1 基本操作

2.2.1.1 基础知识

```
1 全局变量是什么
2 全局变量就是: 在当前系统的所有环境下都能生效的变量。
3
4 查看全局环境变量
5 env 只显示全局变量,一般结合 grep 和管道符来使用
6 printenv 效果与 env 等同
7 export 查看所有的环境变量,包括声明的过程等信息,一般不用
8 declare -x 效果与 export 类似
9
10
11 定义全局变量方法一:
12 变量=值
13 export 变量
14
15 定义全局变量方法二: (最常用)
16 export 变量=值
17
18 定义全局变量方法三:
19 declare -x
```

2.2.1.2 查看全局变量

```
1 # 查看所有的全局变量
2 [root@rocky9 ~]# env
3 XDG_SESSION_ID=4
4 HOSTNAME=localhost
5 SHELL=/bin/bash
```

```

6 TERM=xterm
7 HISTSIZE=1000
8 ...
9 # 查看制定的全局变量
10 [root@rocky9 ~]# env | grep SHELL
11 SHELL=/bin/bash
12
13 # 定制本地变量
14 [root@rocky9 ~]# envtype=local
15 [root@rocky9 ~]# echo $envtype
16 local
17 # 从全局变量中查看
18 [root@rocky9 ~]# env | grep envtype
19 [root@rocky9 ~]#
20 结果显示：
21 无法从全局变量中查看本地变量的名称

```

2.2.1.3 定制全局变量

```

1 方法1定制全局变量
2 [root@rocky9 ~]# echo $envtype
3 local
4 [root@rocky9 ~]# env | grep envtype
5 [root@rocky9 ~]# export envtype
6 [root@rocky9 ~]# env | grep envtype
7 envtype=local
8 方法2定制全局变量
9 [root@rocky9 ~]# export myuser=root
10 [root@rocky9 ~]# env | grep myuser
11 myuser=root
12 鸡肋方法定制全局变量
13 [root@rocky9 ~]# declare -x mydir=/root
14 [root@rocky9 ~]# env | grep mydir
15 mydir=/root
16 清理全局变量
17 [root@rocky9 ~]# unset envtype myuser mydir

```

2.3.2 文件体系

Shell 变量的“持久化”和“全局生效”依赖于系统的配置文件体系，本节聚焦与变量紧密相关的核心文件：**变量文件（自定义 / 系统）**、`bashrc` / `bash_profile`、`profile`

2.3.2.1 变量文件

2.3.2.1.1 核心概念

变量文件是专门用于集中定义、管理 Shell 变量的文本文件（通常为 `.sh` 或无后缀），核心作用是：

- 避免变量重复定义（如多个脚本共用的路径、常量）；
- 统一维护变量（修改文件即可同步所有引用处）；
- 实现变量的“复用”和“批量加载”。

2.3.2.1.2 核心要点

1. **变量文件的格式**：与普通 Shell 脚本一致，仅包含变量定义、`export` 导出等逻辑（无复杂命令，避免加载时执行不必要操作）；
2. **加载方式**：
 - `source 变量文件路径`（或 `. 变量文件路径`）：**在当前 Shell 进程加载**（变量生效），不新建子进程；
 - 直接执行 `./变量文件`：新建子进程执行，变量仅在子进程生效（无意义）；
3. **生效范围**：
 - 临时生效：仅当前 Shell 会话（加载后未写入配置文件）；
 - 永久生效：将 `source 变量文件` 写入 `bashrc/profile` 等全局配置文件。

2.3.2.2 bashrc 和 bash_profile

`bashrc` 和 `bash_profile` 是 Bash Shell 的核心初始化文件，用于配置 Shell 环境（变量、别名、函数等），核心区别在于**加载时机**：

特性	<code>.bashrc</code>	<code>.bash_profile</code> （ <code>.bash_login/.profile</code> 是备选）
加载场景	非登录 Shell（如终端直接打开、SSH 连接后新建终端）	登录 Shell（如 SSH 首次登录、控制台登录）
加载次数	每次新建 Shell 会话都加载	仅登录时加载一次
核心作用	定义本地变量、别名、函数（高频使用）	定义环境变量、登录时的一次性操作（如加载 <code>bashrc</code> ）
生效范围	当前用户（ <code>~/.bashrc</code> ） / 系统全局（ <code>/etc/bashrc</code> ）	当前用户（ <code>~/.bash_profile</code> ） / 系统全局（ <code>/etc/profile</code> ）

1. **登录 Shell vs 非登录 Shell**：
 - 登录 Shell：需要输入用户名密码的 Shell（如 `bash -l`、SSH 登录）；
 - 非登录 Shell：无需认证的 Shell（如桌面终端、`bash` 命令新建会话）；
2. **联动逻辑**：多数系统的 `.bash_profile` 会主动加载 `.bashrc`（保证登录 Shell 也能使用 `.bashrc` 的配置），典型内容：

```
1 if [ -f ~/.bashrc ]; then
2     . ~/.bashrc
3 fi
```

3. **生效方式**：

- 临时生效: `source ~/.bashrc` / `source ~/.bash_profile`;
- 永久生效: 修改文件后, 重启 Shell 或重新登录。

2.3.2.2.1 修改 ~/.bashrc (定义本地变量 / 别名)

```

1 # 1. 定义本地变量 (仅当前 Shell 生效, 高频使用)
2 EDITOR="vim"
3 WORK_DIR="$HOME/workspace"
4
5 # 2. 定义别名 (简化命令)
6 alias ll='ls -alh' # 替代ls -l, 显示更友好
7 alias cdwork='cd $WORK_DIR' # 快速进入工作目录
8 alias grep='grep --color=auto' # grep结果高亮
9
10 # 3. 定义函数 (批量操作)
11 mkwork() {
12     mkdir -p $WORK_DIR/$1 # 创建工作子目录
13     cd $WORK_DIR/$1      # 进入该目录
14 }

```

2.3.2.2.2 修改 ~/.bash_profile (定义环境变量)

```

1 # 1. 加载.bashrc (保证登录Shell也能使用.bashrc的配置)
2 if [ -f ~/.bashrc ]; then
3     . ~/.bashrc
4 fi
5
6 # 2. 定义环境变量 (登录时加载, 全局生效)
7 export JAVA_HOME="/usr/lib/jvm/java-11-openjdk"
8 export MAVEN_HOME="/opt/maven"
9 export PATH="$PATH:$JAVA_HOME/bin:$MAVEN_HOME/bin"
10
11 # 3. 登录时的一次性操作
12 echo "欢迎登录! 当前时间: $(date +%Y-%m-%d\ %H:%M:%S)"
13 echo "工作目录: $WORK_DIR"

```

2.3.2.2.3 验证配置文件生效

```

1 # 1. 加载修改后的配置 (临时生效)
2 source ~/.bashrc
3 source ~/.bash_profile
4
5 # 2. 验证变量
6 echo "EDITOR = $EDITOR"          # 输出: vim
7 echo "JAVA_HOME = $JAVA_HOME"    # 输出: /usr/lib/jvm/java-11-openjdk
8
9 # 3. 验证别名
10 ll # 等价于ls -alh, 显示文件详情

```

```
11 cdwork # 快速进入~/workspace
12 mkwork test # 创建~/workspace/test并进入该目录
13
14 # 4. 验证登录Shell加载（SSH重新登录或执行bash -l）
15 bash -l # 模拟登录Shell，会输出欢迎语
16
17 # 执行source后验证
18 EDITOR = vim
19 JAVA_HOME = /usr/lib/jvm/java-11-openjdk
20
21 # 执行bash -l模拟登录Shell
22 欢迎登录！当前时间：2025-12-06 15:30:00
23 工作目录：/home/user/workspace
```

2.3.2.3 profile

`profile` 是 Shell 登录时的**全局初始化文件**，分为两个层级：

- 系统级： `/etc/profile`（对所有用户生效）；
- 用户级： `~/.bash_profile`、`~/.bash_login`、`~/.profile`（仅对当前用户生效，优先级： `.bash_profile` > `.bash_login` > `.profile`）。

`profile` 的核心作用是：

1. 配置**全局环境变量**（如 `PATH`、`LANG`、`USER` 等）；
2. 执行登录时的一次性初始化操作（如加载系统级配置、设置 `umask`）；
3. 联动加载 `bashrc`（保证登录 Shell 继承非登录 Shell 的配置）。

加载顺序（登录 Shell 场景）：

```
1 | /etc/profile → /etc/profile.d/*.sh → ~/.bash_profile → ~/.bashrc → /etc/bashrc
```

1. **umask 配置**： `profile` 通常会设置 `umask`（文件默认权限掩码），如 `umask 022`（新建文件权限为 644，目录为 755）；
2. **生效方式**：
 - 系统级： `source /etc/profile`；
 - 用户级： `source ~/.bash_profile`；
 - 永久生效：重启 Shell / 重新登录。

2.3.2.3.1 修改 /etc/profile

注意：修改系统级文件需 root 权限，谨慎操作！

```
1 # 切换到root用户
2 sudo su -
3
```

```

4 # 编辑/etc/profile, 追加以下内容
5 cat >> /etc/profile << EOF
6 # 系统级全局环境变量（所有用户生效）
7 export SYSTEM_APP_HOME="/opt/system_app"
8 export SYSTEM_LOG="/var/log/system_app"
9
10 # 设置umask（所有用户新建文件的默认权限）
11 umask 022
12
13 # 加载profile.d目录下的自定义脚本
14 if [ -d /etc/profile.d ]; then
15     for i in /etc/profile.d/*.sh; do
16         if [ -r $i ]; then
17             . $i
18         fi
19     done
20     unset i
21 fi
22 EOF

```

2.3.2.3.2 创建 /etc/profile.d/custom.sh（系统级细分配置）

```

1 # 切换到root用户
2 sudo su -
3
4 # 新建/etc/profile.d/custom.sh, 内容如下
5 cat > /etc/profile.d/custom.sh << EOF
6 #!/bin/bash
7 # 自定义系统级配置（所有用户生效）
8 alias syslog='tail -f $SYSTEM_LOG/app.log' # 实时查看系统应用日志
9 export PS1="[u@h W]\\$ " # 自定义终端提示符（用户名@主机名 目录）
10 EOF
11
12 # 给脚本执行权限
13 chmod +x /etc/profile.d/custom.sh

```

2.3.2.3.3 验证 profile 配置生效

```

1 # 1. 加载系统级profile（临时生效）
2 source /etc/profile
3
4 # 2. 验证系统级变量
5 echo "SYSTEM_APP_HOME = $SYSTEM_APP_HOME" # 输出: /opt/system_app
6 echo "SYSTEM_LOG = $SYSTEM_LOG"           # 输出: /var/log/system_app
7
8 # 3. 验证umask
9 umask # 输出: 0022
10
11 # 4. 验证别名

```

```
12 syslog # 等价于tail -f /var/log/system_app/app.log
13
14 # 5. 验证终端提示符（重新登录后生效）
15 bash -l # 模拟登录Shell，提示符变为 [user@hostname ~]$
```

2.3.2.3.4 用户级 .profile 备用配置（无 .bash_profile 时）

```
1 # 编辑 ~/.profile
2 cat > ~/.profile << EOF
3 # 替代 .bash_profile 的用户级配置
4 if [ -f ~/.bashrc ]; then
5     . ~/.bashrc
6 fi
7
8 export USER_BIN="$HOME/bin"
9 export PATH="$PATH:$USER_BIN"
10 EOF
11
12 # 加载生效
13 source ~/.profile
14
15 # 验证
16 echo "USER_BIN = $USER_BIN" # 输出: /home/user/bin
```

1. 系统级配置（`/etc/profile`、`/etc/profile.d/`）修改后，所有用户登录都会生效，需确保兼容性；
2. 不要在 `/etc/profile` 中定义用户专属配置（应放在 `~/.bash_profile`）；
3. 若修改后配置未生效，检查：
 - 是否用 `source` 加载，或是否重新登录；
 - 文件权限是否为可读（`-r`）；
 - 语法是否错误（如缺少 `fi`、`export` 拼写错误）。

2.3.2.4 总结

文件类型	作用范围	加载场景	核心用途
自定义变量文件	按需加载	source 手动加载	集中管理脚本共用变量
<code>~/.bashrc</code>	当前用户	非登录 Shell	本地变量、别名、函数（高频使用）
<code>~/.bash_profile</code> / <code>~/.profile</code>	当前用户	登录 Shell	环境变量、登录操作、加载 <code>.bashrc</code>
<code>/etc/profile</code>	所有用户	登录 Shell	系统级全局环境变量、 <code>umask</code> 、加载 <code>profile.d</code>

文件类型	作用范围	加载场景	核心用途
/etc/profile.d/*.sh	所有用户	登录 Shell（由 profile 加载）	系统级细分配置（语言、工具、别名）

2.2.3 嵌套 shell

2.2.3.1 export 原理

2.2.3.1.1 核心概念

嵌套 Shell 本质是「父 Shell 进程」创建「子 Shell 进程」，而进程的内存空间是**相互隔离**的——默认情况下，父 Shell 定义的变量（本地变量）不会被子 Shell 感知，`export` 是打破这种隔离的核心工具。

2.2.3.1.1.1 父 / 子 Shell 进程隔离

- 父 Shell：当前正在运行的 Shell 会话（如终端窗口、脚本主进程）；
- 子 Shell：由父 Shell 启动的新 Shell 进程（如执行 `bash` 命令、`(命令)` 子进程、脚本直接执行）；
- 隔离本质：每个进程有独立的内存空间，父 Shell 的变量存储在自身内存中，子 Shell 无法直接访问。

2.2.3.1.1.2 export 的核心作用

`export` 是 Shell 内置命令，核心功能是：

- 将「本地变量」标记为「环境变量」，并写入父 Shell 的**环境表**（环境变量列表）；
- 当创建子 Shell 时，父 Shell 会将自己的环境表**复制一份**到子 Shell 的内存中（单向复制）；
- 子 Shell 仅能读取复制的环境变量，修改后仅作用于自身，不会同步回父 Shell。

2.2.3.1.1.3 关键原理总结

特性	本地变量（未 export）	环境变量（export 标记）
父 Shell 可见性	可见	可见
子 Shell 可见性	不可见（进程隔离）	可见（创建子 Shell 时复制环境表）
子 Shell 修改影响	无（子 Shell 无该变量）	仅影响子 Shell 自身，父 Shell 不受影响
作用域	仅当前 Shell 进程	当前 Shell + 所有子 Shell 进程

2.2.3.1.1.4 底层逻辑（简化版）

```

1 父 Shell 内存
2  └─ 本地变量区: var1=123 (仅父可见)
3  └─ 环境表: export var2=456 (标记为环境变量)
4      ↓ 子 Shell 创建时, 环境表被复制
5 子 Shell 内存
6  └─ 本地变量区: (空, 或子Shell自己定义)
7  └─ 环境表: var2=456 (复制父的环境表)
8      → 子Shell修改var2=789 → 仅修改自身环境表, 父的var2仍为456

```

2.2.3.1.2 shell 执行顺序

Shell 在执行命令时, 顺序是这样的👉

① 父 shell 先解析命令行

② 做变量展开 (\$var、\$\$、\$? 等)

③ 再执行命令

```
bash -c "echo $name"
```

第一步:

```

1 # 父 shell 中看到的是
2 "echo $name"

```

第二步:

```

1 # 父 shell 中进行展开
2 "echo $name" -----> $name -----> tom
3
4 # 于是父 shell 实际执行的是:
5 bash -c "echo tom"

```

▲ 这一步已经把 name 替换完了

第三步:

```

1 # 启动子 shell
2 # 子 shell 实际拿到的命令是
3 echo tom
4
5 # 👉 子 shell 根本没见过 ` $name `

```

2.2.3.2 嵌套实践

2.2.3.2.1 验证本地变量无法传递给子 shell

```

1 21:37:35 root@redis01:~# cat test01.sh
2 #!/bin/bash

```

```

3  #
  =====
  ====
4  # 脚本基础信息
5  # filename: test01.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 unset local_var # 先清空可能残留的变量，避免干扰
12
13 echo "==== 示例1：本地变量的隔离性 ====="
14
15 # 父Shell定义本地变量（仅本地，未export）
16 local_var="父Shell的本地变量"
17 declare -p local_var
18 echo "父Shell中 local_var = $local_var" # 验证父Shell能看到
19
20 # 方式1：执行bash命令启动子Shell，查看变量
21 echo -e "\n1. 子Shell中查看本地变量："
22 bash -c 'echo "子Shell内 local_var = $local_var"' # 输出为空（不可见）
23
24 # 方式2：脚本直接执行（本质是启动子Shell）
25 cat > sub_script1.sh << EOF
26 #!/bin/bash
27 echo "脚本sub_script1.sh（子Shell）内 local_var = \"$local_var"
28 EOF
29 chmod +x sub_script1.sh
30 echo -e "\n3. 脚本直接执行（子Shell）查看本地变量："
31 ./sub_script1.sh # 输出为空（不可见）
32 rm -f sub_script1.sh # 清理测试脚本
33 21:37:36 root@redis01:~# bash test01.sh
34 ===== 示例1：本地变量的隔离性 =====
35 declare -- local_var="父Shell的本地变量"
36 父Shell中 local_var = 父Shell的本地变量
37
38 1. 子Shell中查看本地变量：
39 子Shell内 local_var =
40
41 2. 脚本直接执行（子Shell）查看本地变量：
42 脚本sub_script1.sh（子Shell）内 local_var =

```

2.2.3.2.2 export 环境变量传递给子 Shell

```

1 21:39:19 root@redis01:~# cat test02.sh
2 #!/bin/bash

```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: test02.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 # 示例2: export环境变量可传递给子Shell
12 echo "==== 示例2: export环境变量的传递性 ====="
13
14 # 父Shell定义并export环境变量
15 export env_var="父Shell的环境变量"
16
17 # 方式1: bash命令启动子Shell
18 echo -e "\n1. 子Shell中查看环境变量: "
19 bash -c 'echo "子Shell内 env_var = $env_var"' # 可见
20
21
22 # 方式2: 脚本直接执行
23 cat > sub_script2.sh << EOF
24 #!/bin/bash
25 echo "脚本sub_script2.sh (子Shell) 内 env_var = \${env_var}"
26 EOF
27 chmod +x sub_script2.sh
28 echo -e "\n3. 脚本直接执行 (子Shell) 查看环境变量: "
29 ./sub_script2.sh # 可见
30 rm -f sub_script2.sh
31
32 21:39:21 root@redis01:~# bash test02.sh
33 ===== 示例2: export环境变量的传递性 =====
34
35 1. 子Shell中查看环境变量:
36 子Shell内 env_var = 父Shell的环境变量
37
38 2. 脚本直接执行 (子Shell) 查看环境变量:
39 脚本sub_script2.sh (子Shell) 内 env_var = 父Shell的环境变量

```

2.2.3.2.3 子 Shell 修改环境变量不影响父 Shell

```

1  21:40:41 root@redis01:~# cat test03.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息

```



```

5  # filename: test03.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 # 示例3: 环境变量的单向传递 (子改不影响父)
12 echo "==== 示例3: 环境变量的单向传递 ====="
13
14 # 父Shell定义并export环境变量
15 export env_var="初始值 (父Shell)"
16 echo "父Shell初始值: env_var = $env_var"
17
18 # 子Shell修改环境变量
19 echo -e "\n1. 子Shell内修改环境变量: "
20 bash -c '
21     echo "子Shell修改前: env_var = $env_var"
22     env_var="修改后 (子Shell)" # 仅修改子Shell自身的环境表
23     echo "子Shell修改后: env_var = $env_var"
24 '
25
26 # 父Shell再次查看变量
27 echo -e "\n2. 父Shell查看变量 (不受子Shell修改影响): "
28 echo "父Shell当前值: env_var = $env_var" # 仍为初始值
29
30 21:40:42 root@redis01:~# bash test03.sh
31 ===== 示例3: 环境变量的单向传递 =====
32 父Shell初始值: env_var = 初始值 (父Shell)
33
34 1. 子Shell内修改环境变量:
35 子Shell修改前: env_var = 初始值 (父Shell)
36 子Shell修改后: env_var = 修改后 (子Shell)
37
38 2. 父Shell查看变量 (不受子Shell修改影响):
39 父Shell当前值: env_var = 初始值 (父Shell)

```

2.2.3.2.4 多层嵌套 Shell 的环境变量传递

```

1  22:11:16 root@redis01:~# cat test04.sh
2  #!/bin/bash
3  #
    =====
    ====
4  # 脚本基础信息
5  # filename: test04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru

```

```

9  # GitHub: xuruizhao00-sys
10 #
=====
====
11 # 父 Shell 定义 export 变量
12 export root_var="根变量（父Shell）"
13
14 # 父→子→孙：子 Shell 定义新的 export 变量，再传给孙 Shell
15 echo -e "\n1. 多层嵌套执行："
16 bash -c '
17     # 子 Shell: 查看父传递的变量，定义新的 export 变量
18     echo "子Shell: root_var = $root_var"
19     export child_var="子变量（子Shell）"
20
21     # 孙 Shell: 查看父（子 Shell）传递的变量
22     bash -c "
23         echo \"孙Shell: root_var = \$root_var\"
24         echo \"孙Shell: child_var = \$child_var\"
25         export grandchild_var=\"孙变量（孙Shell）\"
26     "
27 '
28
29 # 父 Shell 查看孙 Shell 定义的变量（不可见，仅子→孙传递）
30 echo -e "\n2. 父Shell查看孙Shell定义的变量："
31 echo "父Shell: grandchild_var = ${grandchild_var}" # 输出为空
32 22:11:18 root@redis01:~# bash test04.sh
33
34 1. 多层嵌套执行：
35 子Shell: root_var = 根变量（父Shell）
36 孙Shell: root_var = 根变量（父Shell）
37 孙Shell: child_var = 子变量（子Shell）
38
39 2. 父Shell查看孙Shell定义的变量：
40 父Shell: grandchild_var =

```

2.2.3.2.5 source 与直接执行的嵌套变量差异

```

1  11:29:37 root@redis01:~# cat test06.sh
2  #!/bin/bash
3  #
=====
====
4  # 脚本基础信息
5  # filename: test06.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys

```

```

10  #
    =====
    ====
11  # 示例5: source (同进程) vs 直接执行 (子Shell) 的变量传递
12  echo "==== 示例5: source vs 直接执行 ====="
13
14  # 父Shell定义变量
15  test_var="父Shell变量"
16
17  # 编写测试脚本
18  cat > test_source.sh << EOF
19  #!/bin/bash
20  echo "脚本内: test_var = \$test_var"
21  test_var="脚本内修改后"
22  echo "脚本内修改后: test_var = \$test_var"
23  EOF
24  chmod +x test_source.sh
25
26  # 方式1: 直接执行 (子Shell, 变量隔离)
27  echo -e "\n1. 直接执行脚本 (子Shell): "
28  ./test_source.sh
29  echo "父Shell (直接执行后): test_var = \$test_var" # 仍为原值
30
31  # 方式2: source执行 (同进程, 变量共享)
32  echo -e "\n2. source执行脚本 (同进程): "
33  source test_source.sh
34  echo "父Shell (source后): test_var = \$test_var" # 被脚本修改
35
36  rm -f test_source.sh
37
38  11:29:39 root@redis01:~# bash test06.sh
39  ===== 示例5: source vs 直接执行 =====
40
41  1. 直接执行脚本 (子Shell):
42  脚本内: test_var =
43  脚本内修改后: test_var = 脚本内修改后
44  父Shell (直接执行后): test_var = 父Shell变量
45
46  2. source执行脚本 (同进程):
47  脚本内: test_var = 父Shell变量
48  脚本内修改后: test_var = 脚本内修改后
49  父Shell (source后): test_var = 脚本内修改后

```

2.2.3.2.6 总结

1. **隔离性**: 父 / 子 Shell 进程内存独立, 本地变量默认不传递;
2. **export 作用**: 将变量标记为环境变量, 创建子 Shell 时单向复制给子 Shell;
3. **单向传递**: 子 Shell 修改环境变量仅影响自身, 父 Shell 不受影响;
4. **多层传递**: 环境变量可从父→子→孙 Shell 传递, 但反向不行;

5. 执行方式:

- 直接执行脚本: 启动子 Shell, 变量隔离;
- `source / .` 执行脚本: 同进程运行, 变量共享;

2.2.3.3 子 shell 创建机制

2.2.3.3.1 子 shell 创建方式

✓ 方式 1: 使用括号 (...) 启动子 shell (最常见方式)

✓ 特点

- (...) 中的命令在 **子 shell** 中执行;
- 子 shell 对变量的修改不会影响父 shell;
- 这是 bash 最常用的子 shell 方式。

```
1 11:43:58 root@redis01:~# cat demo01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo "父 shell PID: $BASHPID"
16
17 x=1
18
19 (
20     echo "子 shell PID: $BASHPID"
21     x=2
22     echo "子 shell 内 x=$x"
23 )
24
25 echo "父 shell 内 x=$x"
26 11:43:59 root@redis01:~# bash demo01.sh
27 父 shell PID: 5358
28 子 shell PID: 5359
29 子 shell 内 x=2
30 父 shell 内 x=1
31 11:44:00 root@redis01:~#
```

✅ 方式 2: 命令替换 `$(...)` 或 `...` (会隐式创建子 shell)

✓ 特点

- 用 `$(...)` 包裹的代码会在子 shell 执行
- 子 shell 的输出会替换为结果赋给变量;
- 子 shell 不能影响父 shell 的变量。

```
1 11:46:00 root@redis01:~# cat demo02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo "父 shell PID: $BASHPID"
16 X=1
17
18 Y=$(
19     echo "子 shell PID: $BASHPID"
20     X=2
21     echo "返回值"
22 )
23
24 echo "父 shell 中 X=$X"
25 echo "Y=$Y"
26
27 11:46:01 root@redis01:~# bash demo02.sh
28 父 shell PID: 5368
29 父 shell 中 X=1
30 Y=子 shell PID: 5369
31 返回值
```

✅ 方式 3: 执行脚本 (`./script.sh`、`sh script.sh`、`bash script.sh`)

✓ 特点

- 脚本以新进程执行 → 一定是子 shell;
- 子脚本不能修改父 shell 的变量;
- 必须 `chmod +x` 可执行。

```

1  # 父脚本和子脚本
2  11:50:52 root@redis01:~# cat main.sh children.sh
3  #!/bin/bash
4  #
   =====
   ====
5  # 脚本基础信息
6  # filename: main.sh
7  # name: xuruizhao
8  # email: xuruizhao00@163.com
9  # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
   =====
   ====
12
13 echo "main PID: $$"
14 VAR=100
15
16 ./children.sh
17
18 echo "main 中 VAR=$VAR"
19 #!/bin/bash
20 #
   =====
   ====
21 # 脚本基础信息
22 # filename: children.sh
23 # name: xuruizhao
24 # email: xuruizhao00@163.com
25 # v: LnxGuru
26 # GitHub: xuruizhao00-sys
27 #
   =====
   ====
28 echo "child PID: $$"
29 VAR=200
30 echo "child 中 VAR=$VAR"
31
32 11:50:55 root@redis01:~# chmod +x main.sh children.sh
33 11:50:58 root@redis01:~# ./main.sh
34 main PID: 5406
35 child PID: 5407
36 child 中 VAR=200
37 main 中 VAR=100

```

👉 子脚本修改 VAR 不会回传给父脚本。

▲ 方式 4：使用管道 |（每一段命令都在子 shell 中执行）

✓ 特点

- 管道右侧的命令一定在子 shell 执行；
- 在子 shell 中设置的变量无法在父 shell 使用。

```
1 11:52:48 root@redis01:~# cat demo03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo "父 shell PID: $BASHPID"
16 X=1
17
18 echo "test" | while read line; do
19     echo "子 shell PID: $BASHPID"
20     X=2
21 done
22 echo "父 shell 中 X=$X"
23
24 11:52:51 root@redis01:~# bash demo03.sh
25 父 shell PID: 5426
26 子 shell PID: 5428
27 父 shell 中 X=1
```

👉 管道导致 while 在子 shell 执行。

🔄 方式 5：使用子 shell 运算符 &（后台执行也会产生子 shell）

✓ 特点

- 后台任务会启动子 shell；
- 不能修改父 shell。

```
1 11:54:20 root@redis01:~# cat demo04.sh
2 #!/bin/bash
```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 echo "父 shell PID: $BASHPID"
12 X=1
13
14 (
15     echo "后台子 shell PID: $BASHPID"
16     X=2
17 )&
18
19 wait    # 等后台任务结束
20 echo "父 shell 中 X=$X"
21
22 11:54:21 root@redis01:~# bash demo04.sh
23 父 shell PID: 5441
24 后台子 shell PID: 5442
25 父 shell 中 X=1

```

 方式 6：在脚本中执行 `bash -c "..."` 手动创建子 shell

✓ 特点

- 显式启动一个新的 bash 进程；
- 常用于执行字符串命令。

```

1  11:56:19 root@redis01:~# cat demo05.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo05.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====

```



```

11 echo "父 shell PID: $BASHPID"
12 bash -c 'echo "子 shell PID: $BASHPID"'
13
14 11:56:22 root@redis01:~# bash demo05.sh
15 父 shell PID: 5450
16 子 shell PID: 5451
17 11:56:24 root@redis01:~#

```

方式 7：使用 subshell 语法 (COMMAND LIST) 中的多命令结构

这是方式 1 的扩展：里面可以写多条命令，依然是子 shell。

```

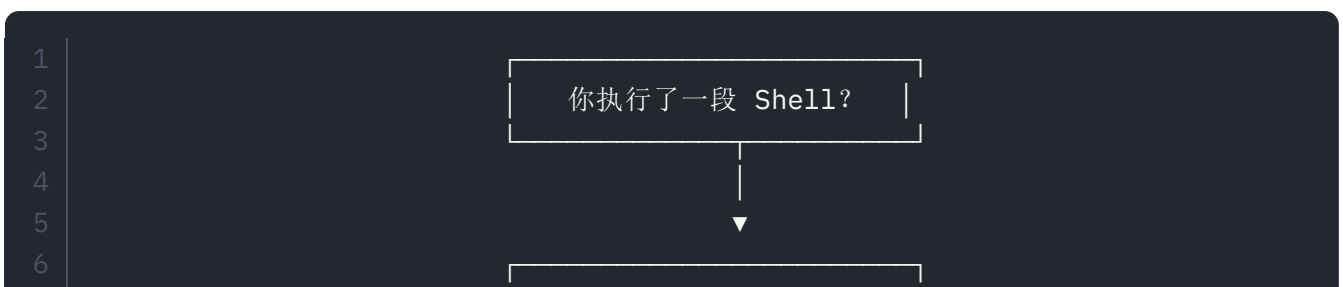
1 11:57:58 root@redis01:~# cat demo06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 (
16     echo "PID: $BASHPID"
17     VAR=999
18     sleep 1
19 )

```

2.2.3.3.2 × 不属于子 shell 的方式（容易混淆）

写法	是否是子 shell	说明
{...}	× 不是	复合命令，但在当前 shell 执行
source script.sh	× 不是	在父 shell 执行，可修改父 shell 变量
. script.sh	× 等同于 source	

2.2.3.3.3 ✖ 一图懂子 Shell（流程图）





2.2.3.3.4 ■ 子 Shell 速查表 (Cheat Sheet)

写法	子 Shell?	是否能修改父变量	用途说明
<code>(commands)</code>	● 是	● 不行	创建子 shell、隔离环境执行
<code>\$(commands)</code>	● 是	● 不行	命令替换 (子 shell 执行)
<code>commands</code>	● 是	● 不行	旧式命令替换 (不推荐)
<code>cmd cmd</code> (管道)	● 是 (管道段独立进程)	● 不行	文本处理、数据传递
<code>./script.sh</code>	● 是	● 不行	执行外部脚本
<code>bash script.sh</code>	● 是	● 不行	指定解释器运行脚本

写法	子 Shell?	是否能修改父变量	用途说明
<code>bash -c "commands"</code>	● 是	● 不行	动态执行字符串命令
<code>command &</code>	● 是	● 不行	后台执行任务
<code>{ commands; }</code>	● 否 (当前 shell)	● 可以	组合多条命令但不生成子 shell
<code>source script.sh / . script.sh</code>	● 否 (当前 shell)	● 可以	加载脚本并在当前 shell 执行

2.2.3.4 子 shell 创建注意事项

2.2.3.4.1 () 创建子 shell 和 bash -c 创建子 shell 区分

() 子 Shell (Subshell) 和 `bash -c` (独立子进程) 虽然都是“`bash -c`”，但它们的产生方式和内存继承机制完全不同。

核心原因：Fork 与 Exec 的区别

原因在于 Linux 系统调用 `fork()` 和 `exec()` 的行为差异：

1. () 子 Shell：只进行了 `fork` (克隆)
- 当 Shell 执行到 `()` 时，它会调用 `fork()` 系统调用产生一个子进程。
 - 关键点：**它没有接着调用 `exec()` 去加载一个新的程序。
 - 因此，子进程是父进程内存的**完整副本**。父进程此刻所有的变量（无论是否 `export`）、函数、别名、句柄，子进程都原封不动地继承了（Copy-on-Write）。
 - 这就是为什么 `local_var` 在 `()` 中可见。
2. `bash -c`：进行了 `fork + exec` (重置)
- 当执行 `bash -c ...` 时，Shell 先 `fork()` 一个子进程。
 - 关键点：**紧接着，子进程调用了 `exec()` 系统调用来启动一个新的 `bash` 程序。
 - `exec()` 会**清空**当前进程的内存空间，加载新的程序代码。
 - 只有被标记为环境变量（`export`）的数据，才会通过环境块（Environment Block）传递给新程序。普通的本地变量在内存清空时就丢失了。

详细对比机制

为了更清晰地理解，我们可以对比这两种方式的内部流程：

特性	方式 2: (...) 子 Shell	方式 1: <code>bash -c '...'</code>
系统调用	仅 <code>fork()</code>	<code>fork()</code> 随后立即 <code>exec()</code>
内存状态	继承副本： 直接复制父 Shell 当前的内存快照。	全新初始化： 内存被清空，重新初始化一个新的 Shell 环境。

特性	方式 2: (...) 子 Shell	方式 1: bash -c '...'
本地变量	可见（因为它就在内存副本里）。	不可见（内存已重置，未 export 的变量丢失）。
环境变量	可见。	可见（通过环境块传递）。
PID	新的 PID (子进程)。	新的 PID (子进程)。
对父影响	修改变量 不会 影响父 Shell（因为是副本）。	修改变量 不会 影响父 Shell（因为是完全独立的进程）。

一个生动的比喻

() 子 Shell 就像是“影分身”：你（父 Shell）使用影分身之术。分身拥有和你此刻一模一样的记忆和口袋里的东西（本地变量）。所以分身知道 local_var 是什么。

bash -c 就像是“雇佣新员工”：你（父 Shell）雇佣了一个全新的员工（新 bash）来干活。你没有把口袋里的私人物品（本地变量）给他，只给了他一份公用的操作手册（环境变量）。所以新员工不知道你的 local_var 是什么，除非你把它写在手册里（export）。

补充验证

虽然 () 能读到父 Shell 的变量，但它是**无法修改**父 Shell 的变量的。你可以尝试在这个子 Shell 里修改变量，退出后父 Shell 的值不会变：

```
1 local_var="Original"
2 (
3     echo "Inside: $local_var"
4     local_var="Modified"
5     echo "Inside Changed: $local_var"
6 )
7 echo "Outside: $local_var" # 这里依然是 "Original"
```

() 之所以能获取到 local_var，是因为它本质上是父 Shell 进程的一个**内存克隆**，它继承了父 Shell 所有的内部状态，而不仅仅是环境变量。

2.2.3.4.2 父子 shell PID 打印问题

```

1 # 测试脚本
2 # 这里的 () 开启了子 shell，但是打印的 PID 号却是完全相同的
3 echo "父 shell PID: $$"
4 X=1
5
6 (
7     echo "子 shell PID: $$"
8     X=2
9     echo "子 shell 内 X=$X"
10 )
11
12 echo "父 shell 中 X=$X"

```

✅ 首先: \$\$ 的含义

在 **bash** 脚本文件中执行时

场景	\$\$ 的值
脚本启动后	永远是脚本进程的 PID (固定)
subshell <code>()</code> 内	\$\$ 不变，仍是脚本 PID
👉 因为在脚本中，bash 不会改变 \$\$ 的值，即使进入 subshell，也保持为脚本自己的 PID。	

🔍 为什么 subshell (子 shell) 里 \$(PID) 不会改变?

因为 POSIX 规定:

在脚本中执行时，\$\$ 必须保持脚本的 PID，不随 subshell 改变。

bash 确实是这样实现的:

✓ subshell 会改变进程 ID (PID)

但

✗ 在脚本内 \$\$ 不会更新为新的 PID!

也就是说:

- subshell `()` 确实 fork 了一个新进程。
- 但 \$\$ 是 shell 启动时设定的变量，在脚本中是固定值。

🔧 用实验验证（真正的子进程 PID）

在 subshell 内打印真实 PID，需要用：

```
echo "真实 pid: $(/bin/ps -o pid= -p $$)"
```

或者更简单用：

```
echo $BASHPID
```

`$BASHPID` 是 bash 内部变量，会随 subshell 改变。

✓ 例子

```
1
2 echo "父 shell BASHPID: $BASHPID"
3 (
4     echo "子 shell BASHPID: $BASHPID"
5 )
6 父 shell BASHPID: 12345
7 子 shell BASHPID: 12346
```

🔥 如果在交互式 shell 执行呢？

在 interactive shell（命令行直接输入）中 `$$` 会随着 subshell 改变：

```
1 23:05:24 root@redis02:~# echo $$
2 20652
3 23:05:41 root@redis02:~# bash -c 'echo $$'
4 20792
5 23:05:49 root@redis02:~#
```

2.2.3.4.3 export 数组问题

Bash 到现在（5.x）为止都 不支持把数组真正 `export` 成环境变量

```
1 declare -a arr_parent=(a b c)
2 export arr_parent
3 bash -c 'echo "子Shell数组: ${arr_parent[@]}"'
```

实际上：

- `export arr_parent` 只是把 变量 `arr_parent` 的 字符串值 放进环境里；
- 环境变量本质上就是一串 `name=value` 字符串，没有数组这种类型；
- 子 bash 进程拿到的是一个普通的标量变量，而不是数组。

这里的 `${arr_parent[@]}` 对一个标量来说，效果和 `$arr_parent` 一样，只是打印一串字符串，并没有真正的“数组元素”概念。

2.3 内置变量

Shell (Bash) 中内置变量非常重要，它们由 Shell 自动提供，用于获取脚本信息、位置参数、进程信息、默认值处理等操作，提高脚本编写效率与可读性。

2.3.1 脚本相关变量 (Script-related Variables)

脚本运行时 Bash 自动提供一些与脚本本身、参数、状态有关的变量。

2.3.1.1 常见脚本相关内置变量表

变量名	含义	示例	备注
<code>\$0</code>	当前脚本名称	<code>./test.sh</code>	可能包含路径
<code>\$1 ... \$9</code>	脚本的第 1-9 个参数	<code>\$1</code> → 第一个参数	超过 9 需用 <code>\${10}</code>
<code>\$#</code>	参数个数	<code>3</code>	常用于判断参数是否够
<code>\$@</code>	所有参数 (独立处理)	<code>"a" "b" "c"</code>	逐个展开
<code>\$*</code>	所有参数 (整体处理)	<code>"a b c"</code>	与 <code>\$@</code> 有区别
<code>\$\$</code>	当前脚本进程 PID	<code>28410</code>	常用于生成唯一文件名
<code>\$_</code>	最近一次后台任务 PID	<code>1293</code>	用于跟踪后台任务
<code>\$?</code>	上一条命令退出状态	<code>0 / 非 0</code>	判断命令是否成功
<code>_</code>	上条命令最后一个参数	很少用	也可用于命令历史

2.3.1.2 示例：打印脚本相关变量

```
1 17:22:38 root@redis01:~/note# cat test01.sh
2 #!/bin/bash
3 #
4 =====
5 # 脚本基础信息
6 # filename: test01.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 #
14 #
15 #
16 #
17 #
18 #
19 #
20 #
21 #
22 #
23 #
24 #
25 #
26 #
27 #
28 #
29 #
30 #
31 #
32 #
33 #
34 #
35 #
36 #
37 #
38 #
39 #
40 #
41 #
42 #
43 #
44 #
45 #
46 #
47 #
48 #
49 #
50 #
51 #
52 #
53 #
54 #
55 #
56 #
57 #
58 #
59 #
60 #
61 #
62 #
63 #
64 #
65 #
66 #
67 #
68 #
69 #
70 #
71 #
72 #
73 #
74 #
75 #
76 #
77 #
78 #
79 #
80 #
81 #
82 #
83 #
84 #
85 #
86 #
87 #
88 #
89 #
90 #
91 #
92 #
93 #
94 #
95 #
96 #
97 #
98 #
99 #
100 #
101 #
102 #
103 #
104 #
105 #
106 #
107 #
108 #
109 #
110 #
111 #
112 #
113 #
114 #
115 #
116 #
117 #
118 #
119 #
120 #
121 #
122 #
123 #
124 #
125 #
126 #
127 #
128 #
129 #
130 #
131 #
132 #
133 #
134 #
135 #
136 #
137 #
138 #
139 #
140 #
141 #
142 #
143 #
144 #
145 #
146 #
147 #
148 #
149 #
150 #
151 #
152 #
153 #
154 #
155 #
156 #
157 #
158 #
159 #
160 #
161 #
162 #
163 #
164 #
165 #
166 #
167 #
168 #
169 #
170 #
171 #
172 #
173 #
174 #
175 #
176 #
177 #
178 #
179 #
180 #
181 #
182 #
183 #
184 #
185 #
186 #
187 #
188 #
189 #
190 #
191 #
192 #
193 #
194 #
195 #
196 #
197 #
198 #
199 #
200 #
201 #
202 #
203 #
204 #
205 #
206 #
207 #
208 #
209 #
210 #
211 #
212 #
213 #
214 #
215 #
216 #
217 #
218 #
219 #
220 #
221 #
222 #
223 #
224 #
225 #
226 #
227 #
228 #
229 #
230 #
231 #
232 #
233 #
234 #
235 #
236 #
237 #
238 #
239 #
240 #
241 #
242 #
243 #
244 #
245 #
246 #
247 #
248 #
249 #
250 #
251 #
252 #
253 #
254 #
255 #
256 #
257 #
258 #
259 #
260 #
261 #
262 #
263 #
264 #
265 #
266 #
267 #
268 #
269 #
270 #
271 #
272 #
273 #
274 #
275 #
276 #
277 #
278 #
279 #
280 #
281 #
282 #
283 #
284 #
285 #
286 #
287 #
288 #
289 #
290 #
291 #
292 #
293 #
294 #
295 #
296 #
297 #
298 #
299 #
300 #
301 #
302 #
303 #
304 #
305 #
306 #
307 #
308 #
309 #
310 #
311 #
312 #
313 #
314 #
315 #
316 #
317 #
318 #
319 #
320 #
321 #
322 #
323 #
324 #
325 #
326 #
327 #
328 #
329 #
330 #
331 #
332 #
333 #
334 #
335 #
336 #
337 #
338 #
339 #
340 #
341 #
342 #
343 #
344 #
345 #
346 #
347 #
348 #
349 #
350 #
351 #
352 #
353 #
354 #
355 #
356 #
357 #
358 #
359 #
360 #
361 #
362 #
363 #
364 #
365 #
366 #
367 #
368 #
369 #
370 #
371 #
372 #
373 #
374 #
375 #
376 #
377 #
378 #
379 #
380 #
381 #
382 #
383 #
384 #
385 #
386 #
387 #
388 #
389 #
390 #
391 #
392 #
393 #
394 #
395 #
396 #
397 #
398 #
399 #
400 #
401 #
402 #
403 #
404 #
405 #
406 #
407 #
408 #
409 #
410 #
411 #
412 #
413 #
414 #
415 #
416 #
417 #
418 #
419 #
420 #
421 #
422 #
423 #
424 #
425 #
426 #
427 #
428 #
429 #
430 #
431 #
432 #
433 #
434 #
435 #
436 #
437 #
438 #
439 #
440 #
441 #
442 #
443 #
444 #
445 #
446 #
447 #
448 #
449 #
450 #
451 #
452 #
453 #
454 #
455 #
456 #
457 #
458 #
459 #
460 #
461 #
462 #
463 #
464 #
465 #
466 #
467 #
468 #
469 #
470 #
471 #
472 #
473 #
474 #
475 #
476 #
477 #
478 #
479 #
480 #
481 #
482 #
483 #
484 #
485 #
486 #
487 #
488 #
489 #
490 #
491 #
492 #
493 #
494 #
495 #
496 #
497 #
498 #
499 #
500 #
501 #
502 #
503 #
504 #
505 #
506 #
507 #
508 #
509 #
510 #
511 #
512 #
513 #
514 #
515 #
516 #
517 #
518 #
519 #
520 #
521 #
522 #
523 #
524 #
525 #
526 #
527 #
528 #
529 #
530 #
531 #
532 #
533 #
534 #
535 #
536 #
537 #
538 #
539 #
540 #
541 #
542 #
543 #
544 #
545 #
546 #
547 #
548 #
549 #
550 #
551 #
552 #
553 #
554 #
555 #
556 #
557 #
558 #
559 #
560 #
561 #
562 #
563 #
564 #
565 #
566 #
567 #
568 #
569 #
570 #
571 #
572 #
573 #
574 #
575 #
576 #
577 #
578 #
579 #
580 #
581 #
582 #
583 #
584 #
585 #
586 #
587 #
588 #
589 #
590 #
591 #
592 #
593 #
594 #
595 #
596 #
597 #
598 #
599 #
600 #
601 #
602 #
603 #
604 #
605 #
606 #
607 #
608 #
609 #
610 #
611 #
612 #
613 #
614 #
615 #
616 #
617 #
618 #
619 #
620 #
621 #
622 #
623 #
624 #
625 #
626 #
627 #
628 #
629 #
630 #
631 #
632 #
633 #
634 #
635 #
636 #
637 #
638 #
639 #
640 #
641 #
642 #
643 #
644 #
645 #
646 #
647 #
648 #
649 #
650 #
651 #
652 #
653 #
654 #
655 #
656 #
657 #
658 #
659 #
660 #
661 #
662 #
663 #
664 #
665 #
666 #
667 #
668 #
669 #
670 #
671 #
672 #
673 #
674 #
675 #
676 #
677 #
678 #
679 #
680 #
681 #
682 #
683 #
684 #
685 #
686 #
687 #
688 #
689 #
690 #
691 #
692 #
693 #
694 #
695 #
696 #
697 #
698 #
699 #
700 #
701 #
702 #
703 #
704 #
705 #
706 #
707 #
708 #
709 #
710 #
711 #
712 #
713 #
714 #
715 #
716 #
717 #
718 #
719 #
720 #
721 #
722 #
723 #
724 #
725 #
726 #
727 #
728 #
729 #
730 #
731 #
732 #
733 #
734 #
735 #
736 #
737 #
738 #
739 #
740 #
741 #
742 #
743 #
744 #
745 #
746 #
747 #
748 #
749 #
750 #
751 #
752 #
753 #
754 #
755 #
756 #
757 #
758 #
759 #
760 #
761 #
762 #
763 #
764 #
765 #
766 #
767 #
768 #
769 #
770 #
771 #
772 #
773 #
774 #
775 #
776 #
777 #
778 #
779 #
780 #
781 #
782 #
783 #
784 #
785 #
786 #
787 #
788 #
789 #
790 #
791 #
792 #
793 #
794 #
795 #
796 #
797 #
798 #
799 #
800 #
801 #
802 #
803 #
804 #
805 #
806 #
807 #
808 #
809 #
810 #
811 #
812 #
813 #
814 #
815 #
816 #
817 #
818 #
819 #
820 #
821 #
822 #
823 #
824 #
825 #
826 #
827 #
828 #
829 #
830 #
831 #
832 #
833 #
834 #
835 #
836 #
837 #
838 #
839 #
840 #
841 #
842 #
843 #
844 #
845 #
846 #
847 #
848 #
849 #
850 #
851 #
852 #
853 #
854 #
855 #
856 #
857 #
858 #
859 #
860 #
861 #
862 #
863 #
864 #
865 #
866 #
867 #
868 #
869 #
870 #
871 #
872 #
873 #
874 #
875 #
876 #
877 #
878 #
879 #
880 #
881 #
882 #
883 #
884 #
885 #
886 #
887 #
888 #
889 #
890 #
891 #
892 #
893 #
894 #
895 #
896 #
897 #
898 #
899 #
900 #
901 #
902 #
903 #
904 #
905 #
906 #
907 #
908 #
909 #
910 #
911 #
912 #
913 #
914 #
915 #
916 #
917 #
918 #
919 #
920 #
921 #
922 #
923 #
924 #
925 #
926 #
927 #
928 #
929 #
930 #
931 #
932 #
933 #
934 #
935 #
936 #
937 #
938 #
939 #
940 #
941 #
942 #
943 #
944 #
945 #
946 #
947 #
948 #
949 #
950 #
951 #
952 #
953 #
954 #
955 #
956 #
957 #
958 #
959 #
960 #
961 #
962 #
963 #
964 #
965 #
966 #
967 #
968 #
969 #
970 #
971 #
972 #
973 #
974 #
975 #
976 #
977 #
978 #
979 #
980 #
981 #
982 #
983 #
984 #
985 #
986 #
987 #
988 #
989 #
990 #
991 #
992 #
993 #
994 #
995 #
996 #
997 #
998 #
999 #
1000 #
1001 #
1002 #
1003 #
1004 #
1005 #
1006 #
1007 #
1008 #
1009 #
1010 #
1011 #
1012 #
1013 #
1014 #
1015 #
1016 #
1017 #
1018 #
1019 #
1020 #
1021 #
1022 #
1023 #
1024 #
1025 #
1026 #
1027 #
1028 #
1029 #
1030 #
1031 #
1032 #
1033 #
1034 #
1035 #
1036 #
1037 #
1038 #
1039 #
1040 #
1041 #
1042 #
1043 #
1044 #
1045 #
1046 #
1047 #
1048 #
1049 #
1050 #
1051 #
1052 #
1053 #
1054 #
1055 #
1056 #
1057 #
1058 #
1059 #
1060 #
1061 #
1062 #
1063 #
1064 #
1065 #
1066 #
1067 #
1068 #
1069 #
1070 #
1071 #
1072 #
1073 #
1074 #
1075 #
1076 #
1077 #
1078 #
1079 #
1080 #
1081 #
1082 #
1083 #
1084 #
1085 #
1086 #
1087 #
1088 #
1089 #
1090 #
1091 #
1092 #
1093 #
1094 #
1095 #
1096 #
1097 #
1098 #
1099 #
1100 #
1101 #
1102 #
1103 #
1104 #
1105 #
1106 #
1107 #
1108 #
1109 #
1110 #
1111 #
1112 #
1113 #
1114 #
1115 #
1116 #
1117 #
1118 #
1119 #
1120 #
1121 #
1122 #
1123 #
1124 #
1125 #
1126 #
1127 #
1128 #
1129 #
1130 #
1131 #
1132 #
1133 #
1134 #
1135 #
1136 #
1137 #
1138 #
1139 #
1140 #
1141 #
1142 #
1143 #
1144 #
1145 #
1146 #
1147 #
1148 #
1149 #
1150 #
1151 #
1152 #
1153 #
1154 #
1155 #
1156 #
1157 #
1158 #
1159 #
1160 #
1161 #
1162 #
1163 #
1164 #
1165 #
1166 #
1167 #
1168 #
1169 #
1170 #
1171 #
1172 #
1173 #
1174 #
1175 #
1176 #
1177 #
1178 #
1179 #
1180 #
1181 #
1182 #
1183 #
1184 #
1185 #
1186 #
1187 #
1188 #
1189 #
1190 #
1191 #
1192 #
1193 #
1194 #
1195 #
1196 #
1197 #
1198 #
1199 #
1200 #
1201 #
1202 #
1203 #
1204 #
1205 #
1206 #
1207 #
1208 #
1209 #
1210 #
1211 #
1212 #
1213 #
1214 #
1215 #
1216 #
1217 #
1218 #
1219 #
1220 #
1221 #
1222 #
1223 #
1224 #
1225 #
1226 #
1227 #
1228 #
1229 #
1230 #
1231 #
1232 #
1233 #
1234 #
1235 #
1236 #
1237 #
1238 #
1239 #
1240 #
1241 #
1242 #
1243 #
1244 #
1245 #
1246 #
1247 #
1248 #
1249 #
1250 #
1251 #
1252 #
1253 #
1254 #
1255 #
1256 #
1257 #
1258 #
1259 #
1260 #
1261 #
1262 #
1263 #
1264 #
1265 #
1266 #
1267 #
1268 #
1269 #
1270 #
1271 #
1272 #
1273 #
1274 #
1275 #
1276 #
1277 #
1278 #
1279 #
1280 #
1281 #
1282 #
1283 #
1284 #
1285 #
1286 #
1287 #
1288 #
1289 #
1290 #
1291 #
1292 #
1293 #
1294 #
1295 #
1296 #
1297 #
1298 #
1299 #
1300 #
1301 #
1302 #
1303 #
1304 #
1305 #
1306 #
1307 #
1308 #
1309 #
1310 #
1311 #
1312 #
1313 #
1314 #
1315 #
1316 #
1317 #
1318 #
1319 #
1320 #
1321 #
1322 #
1323 #
1324 #
1325 #
1326 #
1327 #
1328 #
1329 #
1330 #
1331 #
1332 #
1333 #
1334 #
1335 #
1336 #
1337 #
1338 #
1339 #
1340 #
1341 #
1342 #
1343 #
1344 #
1345 #
1346 #
1347 #
1348 #
1349 #
1350 #
1351 #
1352 #
1353 #
1354 #
1355 #
1356 #
1357 #
1358 #
1359 #
1360 #
1361 #
1362 #
1363 #
1364 #
1365 #
1366 #
1367 #
1368 #
1369 #
1370 #
1371 #
1372 #
1373 #
1374 #
1375 #
1376 #
1377 #
1378 #
1379 #
1380 #
1381 #
1382 #
1383 #
1384 #
1385 #
1386 #
1387 #
1388 #
1389 #
1390 #
1391 #
1392 #
1393 #
1394 #
1395 #
1396 #
1397 #
1398 #
1399 #
1400 #
1401 #
1402 #
1403 #
1404 #
1405 #
1406 #
1407 #
1408 #
1409 #
1410 #
1411 #
1412 #
1413 #
1414 #
1415 #
1416 #
1417 #
1418 #
1419 #
1420 #
1421 #
1422 #
1423 #
1424 #
1425 #
1426 #
1427 #
1428 #
1429 #
1430 #
1431 #
1432 #
1433 #
1434 #
1435 #
1436 #
1437 #
1438 #
1439 #
1440 #
1441 #
1442 #
1443 #
1444 #
1445 #
1446 #
1447 #
1448 #
1449 #
1450 #
1451 #
1452 #
1453 #
1454 #
1455 #
1456 #
1457 #
1458 #
1459 #
1460 #
1461 #
1462 #
1463 #
1464 #
1465 #
1466 #
1467 #
1468 #
1469 #
1470 #
1471 #
1472 #
1473 #
1474 #
1475 #
1476 #
1477 #
1478 #
1479 #
1480 #
1481 #
1482 #
1483 #
1484 #
1485 #
1486 #
1487 #
1488 #
1489 #
1490 #
1491 #
1492 #
1493 #
1494 #
1495 #
1496 #
1497 #
1498 #
1499 #
1500 #
1501 #
1502 #
1503 #
1504 #
1505 #
1506 #
1507 #
1508 #
1509 #
1510 #
1511 #
1512 #
1513 #
1514 #
1515 #
1516 #
1517 #
1518 #
1519 #
1520 #
1521 #
1522 #
1523 #
1524 #
1525 #
1526 #
1527 #
1528 #
1529 #
1530 #
1531 #
1532 #
1533 #
1534 #
1535 #
1536 #
1537 #
1538 #
1539 #
1540 #
1541 #
1542 #
1543 #
1544 #
1545 #
1546 #
1547 #
1548 #
1549 #
1550 #
1551 #
1552 #
1553 #
1554 #
1555 #
1556 #
1557 #
1558 #
1559 #
1560 #
1561 #
1562 #
1563 #
1564 #
1565 #
1566 #
1567 #
1568 #
1569 #
1570 #
1571 #
1572 #
1573 #
1574 #
1575 #
1576 #
1577 #
1578 #
1579 #
1580 #
1581 #
1582 #
1583 #
1584 #
1585 #
1586 #
1587 #
1588 #
1589 #
1590 #
1591 #
1592 #
1593 #
1594 #
1595 #
1596 #
1597 #
1598 #
1599 #
1600 #
1601 #
1602 #
1603 #
1604 #
1605 #
1606 #
1607 #
1608 #
1609 #
1610 #
1611 #
1612 #
1613 #
1614 #
1615 #
1616 #
1617 #
1618 #
1619 #
1620 #
1621 #
1622 #
1623 #
1624 #
1625 #
1626 #
1627 #
1628 #
1629 #
1630 #
1631 #
1632 #
1633 #
1634 #
1635 #
1636 #
1637 #
1638 #
1639 #
1640 #
1641 #
1642 #
1643 #
1644 #
1645 #
1646 #
1647 #
1648 #
1649 #
1650 #
1651 #
1652 #
1653 #
1654 #
1655 #
1656 #
1657 #
1658 #
1659 #
1660 #
1661 #
1662 #
1663 #
1664 #
1665 #
1666 #
1667 #
1668 #
1669 #
1670 #
1671 #
1672 #
1673 #
1674 #
1675 #
1676 #
1677 #
1678 #
1679 #
1680 #
1681 #
1682 #
1683 #
1684 #
1685 #
1686 #
1687 #
1688 #
1689 #
1690 #
1691 #
1692 #
1693 #
1694 #
1695 #
1696 #
1697 #
1698 #
1699 #
1700 #
1701 #
1702 #
1703 #
1704 #
1705 #
1706 #
1707 #
1708 #
1709 #
1710 #
1711 #
1712 #
1713 #
1714 #
1715 #
1716 #
1717 #
1718 #
1719 #
1720 #
1721 #
1722 #
1723 #
1724 #
1725 #
1726 #
1727 #
1728 #
1729 #
1730 #
1731 #
1732 #
1733 #
1734 #
1735 #
1736 #
1737 #
1738 #
1739 #
1740 #
1741 #
1742 #
1743 #
1744 #
1745 #
1746 #
1747 #
1748 #
1749 #
1750 #
1751 #
1752 #
1753 #
1754 #
1755 #
1756 #
1757 #
1758 #
1759 #
1760 #
1761 #
1762 #
1763 #
1764 #
1765 #
1766 #
1767 #
1768 #
1769 #
1770 #
1771 #
1772 #
1773 #
1774 #
1775 #
1776 #
1777 #
1778 #
1779 #
1780 #
1781 #
1782 #
1783 #
1784 #
1785 #
1786 #
1787 #
1788 #
1789 #
1790 #
1791 #
1792 #
1793 #
1794 #
1795 #
1796 #
1797 #
1798 #
1799 #
1800 #
1801 #
1802 #
1803 #
1804 #
1805 #
1806 #
1807 #
1808 #
1809 #
1810 #
1811 #
1812 #
1813 #
1814 #
1815 #
1816 #
1817 #
1818 #
1819 #
1820 #
1821 #
1822 #
1823 #
1824 #
1825 #
1826 #
1827 #
1828 #
1829 #
1830 #
1831 #
1832 #
1833 #
1834 #
1835 #
1836 #
1837 #
1838 #
1839 #
1840 #
1841 #
1842 #
1843 #
1844 #
1845 #
1846 #
1847 #
1848 #
1849 #
1850 #
1851 #
1852 #
1853 #
1854 #
1855 #
1856 #
1857 #
1858 #
1859 #
1860 #
1861 #
1862 #
1863 #
1864 #
1865 #
1866 #
1867 #
1868 #
1869 #
1870 #
1871 #
1872 #
1873 #
1874 #
1875 #
1876 #
1877 #
1878 #
1879 #
1880 #
1881 #
1882 #
1883 #
1884 #
1885 #
1886 #
1887 #
1888 #
1889 #
1890 #
1891 #
1892 #
1893 #
1894 #
1895 #
1896 #
1897 #
1898 #
1899 #
1900 #
1901 #
1902 #
1903 #
1904 #
1905 #
1906 #
1907 #
1908 #
1909 #
1910 #
1911 #
1912 #
1913 #
1914 #
1915 #
1916 #
1917 #
1918 #
1919 #
1920 #
1921 #
1922 #
1923 #
1924 #
1925 #
1926 #
1927 #
1928 #
1929 #
1930 #
1931 #
1932 #
1933 #
1934 #
1935 #
1936 #
1937 #
1938 #
1939 #
1940 #
1941 #
1942 #
1943 #
1944 #
1945 #
1946 #
1947 #
1948 #
1949 #
1950 #
1951 #
1952 #
1953 #
1954 #
1955 #
1956 #
1957 #
1958 #
1959 #
1960 #
1961 #
1962 #
1963 #
1964 #
1965 #
1966 #
1967 #
1968 #
1969 #
1970 #
1971 #
1972 #
1973 #
1974 #
1975 #
1976 #
1977 #
1978 #
1979 #
1980 #
1981 #
1982 #
1983 #
1984 #
1985 #
1986 #
1987 #
1988 #
1989 #
1990 #
1991 #
1992 #
1993 #
1994 #
1995 #
1996 #
1997 #
1998 #
1999 #
2000 #
2001 #
2002 #
2003 #
2004 #
2005 #
2006 #
2007 #
2008 #
2009 #
2010 #
2011 #
2012 #
2013 #
2014 #
2015 #
2016 #
2017 #
2018 #
2019 #
2020 #
2021 #
2022 #
2023 #
2024 #
2025 #
2026 #
2027 #
2028 #
2029 #
2030 #
2031 #
2032 #
2033 #
2034 #
2035 #
2036 #
2037 #
2038 #
2039 #
2040 #
2041 #
2042 #
2043 #
2044 #
2045 #
2046 #
2047 #
2048 #
2049 #
2050 #
2051 #
2052 #
2053 #
2054 #
2055 #
2056 #
2057 #
2058 #
2059 #
2060 #
2061 #
2062 #
2063 #
2064 #
2065 #
2066 #
2067 #
2068 #
2069 #
2070 #
2071 #
2072 #
2073 #
2074 #
2075 #
2076 #
2077 #
2078 #
2079 #
2080 #
2081 #
2082 #
2083 #
2084 #
2085 #
2086 #
2087 #
2088 #
2089 #
2090 #
2091 #
2092 #
2093 #
2094 #
2095 #
2096 #
2097 #
2098 #
2099 #
2100 #
210
```

```

15 echo "所有参数(*): $*"
16 echo "当前 PID: $$"
17
18 ls /not/exist
19 echo "上一条命令退出状态: $?"
20
21 17:22:41 root@redis01:~/note# bash test01.sh
22 脚本名称: test01.sh
23 第一个参数:
24 参数个数: 0
25 所有参数(@):
26 所有参数(*):
27 当前 PID: 6152
28 ls: cannot access '/not/exist': No such file or directory
29 上一条命令退出状态: 2
30 17:22:43 root@redis01:~/note#

```

2.3.1.3 🇺🇸 \$@ 与 \$* 的区别

`$@` 和 `$*` 都表示 Shell 脚本的 **全部位置参数**，但它们在 **被双引号包裹时的行为完全不同**。他们的最大差异体现在：

`"$@"` 会将每个参数作为单独的独立字符串展开
`"$*"` 会把所有参数视为一个整体字符串展开

2.3.1.3.1 示意图

```

1  -- `"$@"` 展开示意图
2  +-----+ +-----+ +-----+
3  |  "aa"   | |  "bb cc" | |  "dd"   |
4  +-----+ +-----+ +-----+
5          ↑           ↑           ↑
6      参数1（独立）  参数2（独立）  参数3（独立）
7
8  -- "$*" 展开示意图
9  +-----+
10 |  "aa bb cc dd"  |
11 +-----+
12          ↑
13      所有参数被合并成一个字符串

```

特性	"@"	"*"
参数展开方式	逐个独立	合并成一个字符串
参数数量	保持原数量	变成 1 个

空格敏感性	保留每个参数的格式	空格会被合并
是否受 IFS 影响	× 不影响	✓ 由 IFS 连接
典型用途	传递参数给其他命令	拼接所有参数成一句话
是否推荐	✓ 强烈推荐	⚠ 不建议在脚本循环中

2.3.1.3.2 代码示例

```
1 17:28:14 root@redis01:~/note# cat test02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo '--- "$@" 展开 ---'
16 for arg in "$@"; do
17     echo "$arg"
18 done
19
20 echo '--- "$*" 展开 ---'
21 for arg in "$*"; do
22     echo "$arg"
23 done
24
25 17:28:15 root@redis01:~/note# bash test02.sh aa "bb cc" dd
26 --- "$@" 展开 ---
27 [aa]
28 [bb cc]
29 [dd]
30
31 --- "$*" 展开 ---
32 [aa bb cc dd]
```

2.3.1.3.3 生产最佳实践

🔥 几乎所有情况下都推荐使用 `"$@"`

原因：

- 保留参数边界
- 不会破坏包含空格的参数
- 与函数参数传递保持一致
- 脚本可移植性强

而：

▲ "\$*" 在循环中几乎永远不该使用

因为它会把所有参数变成一个。

2.3.2 字符串相关

Bash 提供强大的字符串操作能力。

2.3.2.1 子串长度、截取、删除、替换等操作表

表达式	功能	示例	结果
<code>\${#var}</code>	字符串长度	<code>var="hello"</code>	5
<code>\${var:pos}</code>	从 pos 开始截取	<code>\${var:2}</code>	llo
<code>\${var:pos:len}</code>	截取指定长度	<code>\${var:1:3}</code>	ell
<code>\${var#pattern}</code>	从前匹配最短删除	<code>\${file#*/}</code>	删除第一个 / 前内容
<code>\${var##pattern}</code>	从前匹配最长删除	<code>\${file##*/}</code>	获取文件名
<code>\${var%pattern}</code>	从后匹配最短删除	<code>\${file%.*}</code>	去掉后缀
<code>\${var%%pattern}</code>	从后匹配最长删除	<code>\${path%%/*}</code>	获取第一个路径元素
<code>\${var/pattern/repl}</code>	替换首个匹配	<code>\${a/a/A}</code>	A bc
<code>\${var//pattern/repl}</code>	替换全部匹配	<code>\${a//a/A}</code>	A bc A bc

2.3.2.2 示例：常用字符串处理

```
1 17:31:30 root@redis01:~/note# cat test03.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: test03.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 # =====
12 #
13 name="linux_shell_study.txt"
14 echo ${#name}          # 字符串长度
15 echo ${name:0:5}       # 前 5 个字符
```

```

15 echo ${name%.txt}      # 删除后缀
16 echo ${name#*_}        # 删除第一个下划线前内容 (shell_study.txt)
17 echo ${name//_/_}      # 替换所有下划线为空格
18
19 17:31:32 root@redis01:~/note# bash test03.sh
20 21
21 linux
22 linux_shell_study
23 shell_study.txt
24 linux shell study.txt

```

2.3.2.3 🖼️ 截取示意图 (图解)

```

1 index 0 1 2 3 4 5 6 7 8 9 10
2 text l i n u x _ s h e l l
3
4 ${text:2:4}
5
6 0
7 l i n u x _ s h e l l
8 |-----|
9     ↑  ↑
10    2  4 个字符
11 nux_
12

```

2.3.3 默认值相关

用于给变量提供默认值、赋值、报错等功能。

2.3.3.1 🚩 默认值、赋值、报错、替换等规则表

表达式	含义	示例	行为说明
<code>\${var:-default}</code>	var 未设置或为空，则使用 default	<code>\${name:-guest}</code>	不修改变量
<code>\${var:=default}</code>	var 未设置或为空，则赋值为 default	<code>\${name:=guest}</code>	修改变量
<code>\${var:+alt}</code>	var 已设置，则使用 alt，否则空	<code>\${name:+exists}</code>	常用于判断
<code>\${var:?message}</code>	var 为空或未设置则报错退出	<code>\${config:?未设置配置文件}</code>	出错后脚本退出

2.3.3.2 📌 示例：默认值、赋值、错误处理

```

1 17:37:29 root@redis01:~/note# cat test04.sh
2 #!/bin/bash

```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: test04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 echo "用户名: ${USER_NAME:-未指定}"
12 # 未赋值则不会改变变量本身
13
14 echo "设置用户: ${USER_NAME:=guest}"
15 # 如果之前为空, 则 USER_NAME 现在变成 guest
16
17 echo "用户状态: ${USER_NAME:+已登记}" # USER_NAME 非空 → 输出“已登记”
18
19 echo "配置文件位置: ${CONFIG_FILE:?必须提供 CONFIG_FILE}"
20 # 如果变量为空 → 脚本中断 + 打印错误
21
22 17:37:30 root@redis01:~/note# bash test04.sh
23 用户名: 未指定
24 设置用户: guest
25 用户状态: 已登记
26 test04.sh: line 18: CONFIG_FILE: 必须提供 CONFIG_FILE

```

2.3.3.3 🇨🇳 默认值机制图解

```

1  ${var:-default}
2  [ var ] → 空或未定义 → 使用 default 但不赋值
3
4
5  ${var:=default}
6  [ var ] → 空或未定义 → 赋值为 default → var=default

```

三、脚本交互

3.1 基础知识

3.1.1 shell 登录解读

3.1.1.1 shell 配置文件

Shell (以 Bash 为例) 常见的配置文件如下:

配置文件	作用	生效场景
------	----	------

配置文件	作用	生效场景
<code>/etc/profile</code>	系统级初始化配置	登录 shell
<code>/etc/bashrc</code> 或 <code>/etc/bash.bashrc</code>	系统级 bash 配置	非登录 shell
<code>~/.bash_profile</code>	用户级登录配置（优先）	登录 shell
<code>~/.bash_login</code>	用户级登录配置（次优先）	登录 shell
<code>~/.profile</code>	用户级登录配置（最末）	登录 shell
<code>~/.bashrc</code>	交互式、非登录 shell 配置	终端打开 Bash 时
常见内容		

- 设置环境变量：`PATH`、`LANG` 等
- 设置别名：`alias ll='ls -l'`
- 设置提示符：`PS1`
- 加载函数、脚本等

🔴 **小提示：**多数 Linux 默认在 `~/.bash_profile` 中再加载 `~/.bashrc`，确保登录/非登录行为一致。

3.1.1.2 shell 登录方式

3.1.1.2.1 🔒 登录 shell (Login Shell)

触发方式：

- SSH 登录
- 控制台登录 (tty)
- `bash --login`
- `su - username`

会读取：

```
1 | /etc/profile → ~/.bash_profile → ~/.bashrc
```

3.1.1.2.2 🌿 非登录 shell (Not-login Shell)

触发方式：

- 图形界面打开终端（默认非登录）
- `su username`
- 在 shell 中执行 `bash`
- 运行脚本时的 Shell

```
1 | /etc/bashrc → ~/.bashrc
2 |
```

3.1.1.3 shell 配置文件生效顺序

```
1 | 登录shell的文件生效流程
2 |
3 | /etc/profile.d/*.sh
4 |
5 | -> /etc/profile
6 |
7 | -> /etc/bashrc
8 |
9 | -> ~/.bashrc
10 |
11 | -> ~/.bash_profile
12 |
13 | 非登录shell的文件生效流程
14 |
15 | /etc/profile.d/*.sh
16 |
17 | -> /etc/bashrc
18 |
19 | -> ~/.bashrc
20 |
21 | 注意：
22 |
23 | 若多配置文件中设置相同的变量，则后面配置文件中变量的值会覆盖前面配置文件中同一变量的值。
```

3.1.1.4 su 命令生效行为

```
1 | su的相关参数
2 |
3 | -: 当前用户不仅切换为指定用户的身份，同时所用的工作环境也切换为此用户的环境。
4 |
5 | -l: 同 - 的使用类似，完整切换工作环境，后面需要添加欲切换的使用者账号。
6 |
7 | -p: 表示切换为指定用户的身份，但不改变当前的工作环境（不使用切换用户的配置文件）。
8 |
9 | -m: 和 -p 一样；
10 |
11 | -c 命令：仅切换用户执行一次命令，执行后自动切换回来，该选项后通常会带有要执行的命令
```

命令	类型	配置文件加载
<code>su user</code>	非登录 shell	加载 <code>~/.bashrc</code>

命令	类型	配置文件加载
<code>su - user</code>	登录 shell	加载 <code>/etc/profile</code> + 用户全部登录文件

示例说明：

- `su user`：保留原用户环境变量
- `su - user`：模拟完整登录环境

3.1.1.5 配置文件如何生效

- **重新登录**
- 使用 `source` 或 `.`
`source ~/.bashrc`
- 使用 `exec` 重启当前 shell
`exec bash`

3.1.1.6 配套代码示例

3.1.1.6.1 非登录场景下环境差异实践

3.1.1.6.1.1 Rocky Linux

```
1 [root rockylinux-1 ~] WORK 0 # echo "echo '1 - /etc/profile'" >> /etc/profile
2 echo "echo '2 - /etc/profile.d/2.sh'" >> /etc/profile.d/2.sh
3 echo "echo '3 - /etc/bashrc'" >> /etc/bashrc
4 echo "echo '4 - ~/.bash_profile'" >> ~/.bash_profile
5 echo "echo '5 - ~/.bashrc'" >> ~/.bashrc
6 [root rockylinux-1 ~] WORK 0 # su - test01
7 2 - /etc/profile.d/2.sh
8 3 - /etc/bashrc
9 1 - /etc/profile
10 3 - /etc/bashrc
11 [test01 rockylinux-1 ~] WORK 0 # su root
12 Password:
13 2 - /etc/profile.d/2.sh
14 3 - /etc/bashrc
15 5 - ~/.bashrc
```

3.1.1.6.1.2 Ubuntu Linux

```

1 [root rockylinux-1 ~] WORK 0 # echo "echo '1 - /etc/profile'" >>
  /etc/profile
2 echo "echo '2 - /etc/profile.d/2.sh'" >> /etc/profile.d/2.sh
3 echo "echo '3 - /etc/bashrc'" >> /etc/bashrc
4 echo "echo '4 - ~/.bash_profile'" >> ~/.bash_profile
5 echo "echo '5 - ~/.bashrc'" >> ~/.bashrc
6 13:18:19 root@redis01:~# su - test01
7 2 - /etc/profile.d/2.sh
8 1 - /etc/profile
9 test01@redis01:~$ su root
10 Password:
11 2 - /etc/profile.d/2.sh
12 1 - /etc/profile
13 5 - ~/.bashrc

```

Login shell 加载顺序 (Debian/Ubuntu 特性)

```

1 /etc/profile
2   ↳ /etc/profile.d/*.sh
3 ~/.bash_profile      (如果存在则只读它)
4 ~/.profile           (如果没有 bash_profile 才读)

```

不会自动加载:

- `/etc/bashrc` (在 RHEL/CentOS 才有)
- `~/.bashrc`

3.2 脚本内外交互

3.2.1 read 基础

3.2.1.1 基础知识

`read` 是 Shell 中用于从标准输入读取用户输入的命令，通常用于脚本与用户交互

`read` [选项] 变量名

常用选项说明:

选项	意义
<code>-p</code>	指定提示信息
<code>-t</code>	限制等待输入的秒数 (超时)
<code>-n</code>	限制读取的字符数
<code>-s</code>	静默输入 (常用于密码)
<code>-r</code>	读取原始输入, 不解析反斜杠

默认情况下, `read` 从键盘读取一整行并存入变量。

3.2.1.2 命令简单实践

3.2.1.2.1 读取单个变量

```
1 15:08:59 root@redis01:~/shell/lesson02# cat demo01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 read name
16
17 echo "Hello $name"
18
19 15:09:01 root@redis01:~/shell/lesson02# bash demo01.sh
20 aaaa
21 Hello aaaa
22 15:09:04 root@redis01:~/shell/lesson02#
```

3.2.1.2.2 读取多个变量

```
1 15:10:02 root@redis01:~/shell/lesson02# cat demo02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 read a b c
16 echo "A=$a B=$b C=$c"
17
18 15:10:03 root@redis01:~/shell/lesson02# bash demo02.sh
19 1
```

```
16 A=1 B= C=
17 15:10:07 root@redis01:~/shell/lesson02# bash demo02.sh
18 1 2 3
19 A=1 B=2 C=3
```

3.2.1.3 带提示符

```
1 15:13:25 root@redis01:~/shell/lesson02# cat demo03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 read -rp "你的年龄是: " age
16 echo "你输入的年龄是 $age"
17
18 15:13:41 root@redis01:~/shell/lesson02# bash demo03.sh
19 你的年龄是: 12
20 你输入的年龄是 12
21 15:13:46 root@redis01:~/shell/lesson02# bash demo03.sh
22 你的年龄是: 1/2
23 你输入的年龄是 1/2
24 15:13:54 root@redis01:~/shell/lesson02# bash demo03.sh
25 你的年龄是: 1\2
26 你输入的年龄是 1\2
27 15:13:58 root@redis01:~/shell/lesson02#
```

3.2.1.2.4 静默读取密码

```
1 15:15:55 root@redis01:~/shell/lesson02# cat demo04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
```

```

10 #
    =====
    ====
11 read -rsp "请输入密码: " pass
12 echo      # -s 会吞掉换行，手动补一个
13 echo "你的密码长度是 ${#pass} 位"
14
15 15:15:56 root@redis01:~/shell/lesson02# bash demo04.sh
16 请输入密码:
17 你的密码长度是 3 位
18 15:16:00 root@redis01:~/shell/lesson02#

```

3.2.1.4 信息提示

```

1 15:16:40 root@redis01:~/shell/lesson02# cat demo05.sh
2 #!/bin/bash
3 #
    =====
    ====
4 # 脚本基础信息
5 # filename: demo05.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 # 颜色函数
12 red(){ echo -e "\033[31m$*\033[0m"; }
13 green(){ echo -e "\033[32m$*\033[0m"; }
14
15 read -rp "$(green '>>> 请输入文件名: ')" file
16 [[ -f $file ]] && green "文件存在" || red "文件不存在"
17
18
19 15:16:45 root@redis01:~/shell/lesson02# bash demo05.sh
20 >>> 请输入文件名: aaa.sh
21 文件不存在
22 15:16:51 root@redis01:~/shell/lesson02#

```

3.2.1.5 其他实践

3.2.1.5.1 指定分隔符

```

1 15:17:52 root@redis01:~/shell/lesson02# cat demo06.sh
2 #!/bin/bash

```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo06.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 IFS=, read -rp "请输入用逗号分隔的 学科-分数: " subject score
12 echo "学科=$subject, 分数=$score"
13
14 15:17:53 root@redis01:~/shell/lesson02# bash demo06.sh
15 请输入用逗号分隔的 学科-分数: Linux,88
16 学科=Linux, 分数=88
17 15:18:04 root@redis01:~/shell/lesson02#

```

3.2.1.5.2 限时输入

```

1 15:18:45 root@redis01:~/shell/lesson02# cat demo07.sh
2 #!/bin/bash
3 #
   =====
   ====
4 # 脚本基础信息
5 # filename: demo07.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 if read -t 3 -rp "3 秒内输入 YES: " ans && [[ $ans == YES ]]; then
12     echo "快速通过! "
13 else
14     echo "超时或输入错误! "
15 fi
16
17 15:18:47 root@redis01:~/shell/lesson02# bash demo07.sh
18 3 秒内输入 YES: yes
19 超时或输入错误!
20 15:18:55 root@redis01:~/shell/lesson02# bash demo07.sh
21 3 秒内输入 YES: YES
22 快速通过!
23 15:19:00 root@redis01:~/shell/lesson02# bash demo07.sh
24 3 秒内输入 YES: 超时或输入错误!

```

3.2.2 案例

3.2.2.1 登录模拟实验

3.2.2.1.1 简易登录模式

```

1 USER_OK=admin
2 PASS_OK=123456
3
4 red(){ echo -e "\033[31m$*\033[0m"; }
5 green(){ echo -e "\033[32m$*\033[0m"; }
6
7 read -rp "账号: " user
8 read -rsp "密码: " pass
9 echo
10
11 [[ $user == "$USER_OK" && $pass == "$PASS_OK" ]] &&
12     green "登录成功! " ||
13     red "账号或密码错误! "
```

3.2.2.1.2 3 次失败锁定

```

1 18:31:46 root@redis01:~/shell/lesson02# cat demo08.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo08.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 user="admin"
16 pass="123456"
17
18 for i in {1..3}
19 do
20     read -p "用户名: " u
21     read -s -p "密码: " p
22     echo
23
24     if [[ "$u" == "$user" && "$p" == "$pass" ]]; then
25         echo "登录成功! "
```

```

23     else
24         echo "登录失败（第 $i 次）"
25     fi
26 done
27
28 echo "连续三次失败，锁定登录！"
29 exit 1

```

3.2.2.2 堡垒机实践

```

1  18:34:18 root@redis01:~/shell/lesson02# cat jumpserver_module.sh
2  #!/bin/bash
3  #
4  =====
5  # 脚本基础信息
6  # filename: jumpserver_module.sh
7  # name: xuruizhao
8  # email: xuruizhao00@163.com
9  # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 #!/bin/bash
14 #
15 # =====
16 # 🏰 简易堡垒机 Jump Server Script
17 # =====
18 # ---- 用户认证（可接 DB / LDAP） ----
19 admin_user="admin"
20 admin_pass="123456"
21 log_file="/tmp/jumpserver.log"
22
23 echo "===== 🏰 服务器堡垒机 Login ====="
24 for i in {1..3}
25 do
26     read -p "👤 用户名: " u
27     read -s -p "🔒 密码: " p
28     echo
29
30     if [[ "$u" == "$admin_user" && "$p" == "$admin_pass" ]]; then
31         echo "✅ 登录成功，欢迎你 $u"
32         echo "$(date +%F %T) 登录成功: 用户 $u" >> "$log_file"
33         break
34     else
35         echo "X 登录失败（第 $i 次）"

```

```
36     echo "$(date +"%F %T") 登录失败：用户 $u" >> "$log_file"
37     fi
38 done
39
40 if [[ "$u" != "$admin_user" || "$p" != "$admin_pass" ]]; then
41     echo "🚫 三次失败，拒绝访问！"
42     exit 1
43 fi
44
45 # ---- 主机清单（可扩展） ----
46 declare -A host_list=(
47     [1]="10.1.1.10 WebServer"
48     [2]="10.1.1.20 Database"
49     [3]="10.1.1.30 Redis"
50     [4]="172.16.1.99 TestServer"
51 )
52
53 # ---- 主菜单 ----
54 while true
55 do
56     echo
57     echo "===== 🖥️ 请选择要访问的服务器 ====="
58     for key in "${!host_list[@]}"; do
59         ip=$(echo "${host_list[$key]}" | awk '{print $1}')
60         name=$(echo "${host_list[$key]}" | awk '{print $2}')
61         echo "$key) 🖥️ $name ($ip)"
62     done
63     echo "0) ✕ 退出"
64     echo "===== "
65
66     read -p "请输入编号： " choice
67
68     # ---- 阻止非数字输入 ----
69     if ! [[ "$choice" =~ ^[0-9]+$ ]]; then
70         echo "⚠️ 请输入数字编号！"
71         continue
72     fi
73
74     if [[ "$choice" -eq 0 ]]; then
75         echo "👋 已退出堡垒机，再见！"
76         exit 0
77     fi
78
79     if [[ -z "${host_list[$choice]}" ]]; then
80         echo "⚠️ 无效的编号，请重新输入！"
81         continue
82     fi
83
84     # ---- 解析 ip 和主机名 ----
85     ip=$(echo "${host_list[$choice]}" | awk '{print $1}')
```

```
86     name=$(echo "${host_list[$choice]}" | awk '{print $2}')
87
88     echo "🔗 正在连接到 $name ($ip)..."
89
90     # ---- 输入 SSH 用户 ----
91     read -p "SSH 用户名: " ssh_user
92     read -s -p "SSH 密码: " ssh_pass
93     echo
94
95     echo "$(date +%F %T) 访问主机: $name ($ip) 用户:$ssh_user" >>
"$log_file"
96
97     # ---- 真实登录方式（可选 3 种） ----
98     echo
99     echo "■ 请选择登录方式: "
100    echo "1) sshpass 自动登录"
101    echo "2) expect 自动登录"
102    echo "3) 手动 ssh 登录（推荐高安全）"
103    read -p "请选择: " login_type
104
105    case $login_type in
106        1)
107            echo "⚡ 使用 sshpass 登录（需要系统安装 sshpass）"
108            sshpass -p "$ssh_pass" ssh -o StrictHostKeyChecking=no
"$ssh_user@$ip"
109            ;;
110        2)
111            echo "⚡ 使用 expect 自动登录"
112            expect << EOF
113    spawn ssh "$ssh_user@$ip"
114    expect "password:"
115    send "$ssh_pass\r"
116    interact
117    EOF
118            ;;
119        3)
120            echo "👋 请手动输入密码"
121            ssh "$ssh_user@$ip"
122            ;;
123        *)
124            echo "X 无效登录方式！"
125            ;;
126    esac
127 done
```


四、表达式

4.1 运算符

4.1.1 运算符基础

4.1.1.1 基础知识

类型	运算符	示例	说明
算术运算	+	<code>a=\$((b + c))</code>	加
	-	<code>a=\$((b - c))</code>	减
	*	<code>a=\$((b * c))</code>	乘
	/	<code>a=\$((b / c))</code>	除 (整数)
	%	<code>a=\$((b % c))</code>	取模
	**	<code>a=\$((b ** c))</code>	幂
赋值运算	=	<code>a=5</code>	赋值
	+=	<code>a+=3</code>	累加
	-=	<code>a-=2</code>	累减
	=	<code>a=4</code>	累乘
	/=	<code>a/=2</code>	累除
	%=	<code>a%=3</code>	累取模
关系运算 (数字)	-eq	<code>[\$a -eq \$b]</code>	等于
	-ne	<code>[\$a -ne \$b]</code>	不等于
	-gt	<code>[\$a -gt \$b]</code>	大于
	-lt	<code>[\$a -lt \$b]</code>	小于
	-ge	<code>[\$a -ge \$b]</code>	大于等于
	-le	<code>[\$a -le \$b]</code>	小于等于
布尔运算	!	<code>[! \$a -eq \$b]</code>	取反
	-a	<code>[\$a -eq 1 -a \$b -eq 2]</code>	与
	-o	<code>[\$a -eq 1 -o \$b -eq 2]</code>	或
逻辑运算 ([[]] 支持)	&&	<code>[[\$a -eq 1 && \$b -eq 2]]</code>	与
		<code>[[\$a -eq 1 \$b -eq 2]]</code>	或

类型	运算符	示例	说明
	<code>!</code>	<code>[[! \$a -eq \$b]]</code>	非
字符串运算	<code>=</code>	<code>["\$a" = "\$b"]</code>	等于
	<code>!=</code>	<code>["\$a" != "\$b"]</code>	不等于
	<code><</code>	<code>["\$a" \< "\$b"]</code>	小于（字典序）
	<code>></code>	<code>["\$a" \> "\$b"]</code>	大于（字典序）
	<code>-z</code>	<code>[-z "\$a"]</code>	空字符串
	<code>-n</code>	<code>[-n "\$a"]</code>	非空字符串
文件测试	<code>-e</code>	<code>[-e /path]</code>	文件存在
	<code>-f</code>	<code>[-f /path]</code>	普通文件
	<code>-d</code>	<code>[-d /path]</code>	目录
	<code>-r</code>	<code>[-r /path]</code>	可读
	<code>-w</code>	<code>[-w /path]</code>	可写
	<code>-x</code>	<code>[-x /path]</code>	可执行
	<code>-s</code>	<code>[-s /path]</code>	非空
组合示例	<code>(())</code>	<code>((a+=2))</code>	算术操作的另一种写法
	<code>let</code>	<code>let a=a+1</code>	另一种算术赋值
▲ 注意：			

1. 关系运算符 `-eq` 系列与字符串运算符 `==` 混用是新手最常见错误。
2. 除法 `/` 向下取整（整数除法），`5/2 = 2`。
3. 取模 `%` 要求两侧为正整数，负值行为依赖版本

4.1.1.2 数据操作

- 1 运算符 - 数据值之间的操作
- 2 赋值运算
- 3 - 结果值输出
- 4 - 示例： `=`、`*=`、`/=`、`%=`、`+=`、`-=`、`<<=`、`>>=`、`&=`、`^=`、`|=`等
- 5 二元运算
- 6 - 数据值操作
- 7 - 示例： `+`、`-`、`*`、`/`、`%`等

高阶运算

- 更高级的数学运算

- 示例: `**`、`^`、`++`、`--`、

其他运算

- 其他运算操作

- 示例: `<<`、`>>`等

注意:

这些所谓的运算符一般很难单独来使用, 都需要结合计算表达式来实现相应的效果

表达式 - 数据值在特定场景的运算符操作

计算表达式

- 将多个值的运算操作执行起来

- 示例: `bc`、`let`、`expr`、`$(())`等

测试表达式

- 判断结果值是否满足需求

- 示例: `test`、`[]`等

逻辑表达式

- 多条件的多场景组合

- 示例: `&&`、`||`、`and`、`or`、`not`、`&`、`|`等

比较表达式

- 判断数据之间的适配关系

- 示例: `-f`、`|d`、`|s`、`-r`、`|x`、`|w`、`-e`、`-n`、`==`、`!=`、`>`、`<`、`<=`、`>=`等

三元表达式

- 多逻辑的简单计算表达式

- 示例: `expr?expr:expr`

集合表达式

- 表达式之间的关联关系

- 示例: `expr1` , `expr2`、`[[ ]]`、`[ -a|o ]`、`[ ! ]`等

4.1.1.3 赋值运算

运算符	等效展开	说明
<code>+=</code>	<code>var=\${var}...</code>	字符串拼接 / 算术加
<code>--</code> <code>*=</code> <code>/=</code> <code>%=</code>	同 C 语言	仅整数上下文
<code>++</code> <code>--</code>	前后自增 / 自减	仅整数上下文

```
1  # 为变量赋值
2  20:07:21 root@redis01:~# echo $age $name
3
4  20:07:24 root@redis01:~# age=12
5  20:07:27 root@redis01:~# name="zhangsan"
6  20:07:32 root@redis01:~# echo $age $name
7  12 zhangsan
8  20:07:33 root@redis01:~#
9
10
11 # 赋值时定制属性
12 20:07:33 root@redis01:~# declare string="abc"
13 20:08:23 root@redis01:~# declare -p string
14 declare -- string="abc"
15 20:08:54 root@redis01:~# declare -i a=12
16 20:09:02 root@redis01:~# declare -i b=aaa
17 20:09:08 root@redis01:~# echo $a $b
18 12 0
19 20:09:14 root@redis01:~# declare -p a b
20 declare -i a="12"
21 declare -i b="0"
22 20:09:29 root@redis01:~#
23
24 # 获取特定数据
25 20:09:29 root@redis01:~# user=`whoami`
26 20:10:00 root@redis01:~# echo $user
27 root
28 20:10:04 root@redis01:~# user=$(whoami)
29 20:10:13 root@redis01:~# echo $user
30 root
31 20:10:14 root@redis01:~#
```

4.1.2 简单计算

Bash 原生 **不支持浮点**，需借助 `bc` 或 `awk`。

下面 4 种写法均针对**整数**。

写法	是否 POSIX	子 Shell	备注
----	----------	---------	----

写法	是否 POSIX	子 Shell	备注
<code>\$[]</code>	否	否	已废弃，别用
<code>let</code>	否	否	需空格转义
<code>(())</code>	否	否	推荐，逻辑值可直连 <code>if</code>
<code>\$(())</code>	是	是	通用，可嵌套

4.1.2.1 `$[]`

```

1  # 简单数值计算
2  20:10:14 root@redis01:~# echo ${3+4}
3  7
4  20:11:31 root@redis01:~# echo ${3*4}
5  12
6  20:11:34 root@redis01:~# echo ${4/2}
7  2
8  20:11:38 root@redis01:~# echo ${4%2}
9  0
10 20:11:40 root@redis01:~#
11
12 # 集合变量使用，不会修改变量值
13 20:11:40 root@redis01:~# a=10
14 20:12:19 root@redis01:~# echo ${a+3}
15 13
16 20:12:25 root@redis01:~# echo $a
17 10
18 20:12:29 root@redis01:~#

```

4.1.2.2 `let`

- 内置命令，支持整数运算
- 可以用双引号或不加引号

```

1  22:37:00 root@redis01:~# cat test01.sh
2  #!/bin/bash
3  #
4  # =====
5  # 脚本基础信息
6  # filename: test01.sh
7  # name: xuruizhao
8  # email: xuruizhao00@163.com
9  # v: LnxGuru
10 # GitHub: xuruizhao00-sys

```

```

10  #
    =====
    ====
11  a=5
12  b=7
13  let c=a*b
14  echo "c=$c"
15
16  let d=a+b
17  echo "d=$d"
18
19  22:37:02 root@redis01:~# bash test01.sh
20  c=35
21  d=12
22  22:37:05 root@redis01:~#

```

4.1.2.3 (())

- 推荐的算术运算方式
- 不返回输出，但可以用于条件判断

```

1  22:38:50 root@redis01:~# cat test02.sh
2  #!/bin/bash
3  #
    =====
    ====
4  # 脚本基础信息
5  # filename: test02.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 a=10
12 b=20
13 ((a += b))
14 echo "a += b => $a"
15
16 if (( a > 25 )); then
17     echo "a 大于 25"
18 fi
19
20 22:38:57 root@redis01:~# bash test02.sh
21 a += b => 30
22 a 大于 25

```

4.1.2.4 \$(())

Bash 的算术扩展 `$(())` 遵循 C 语言表达式规则，因此可以包含：

类型	允许?	示例
赋值 <code>=</code>	✓ 允许	<code>a = 3</code>
复合赋值 <code>+= -= *= /=</code>	✓ 允许	<code>a += 2</code>
自增自减	✓ 允许	<code>a++</code> , <code>--b</code>
算术组合表达式	✓ 允许	<code>a = a + 2</code>

- 最常用，**返回计算结果**
- 可直接赋值或输出

```
1 #!/bin/bash
2 a=8
3 b=3
4 c=$((a**b))
5 echo "a 的 b 次方 = $c"
6
7 d=$((a+b*2))
8 echo "a+b*2=$d"
9
```

4.1.2.5 expr

`expr` 是一个外部命令，用来执行 **整数运算、字符串操作和逻辑判断**。

特点：

- 参数必须用**空格隔开**
- 运算符有优先级
- 遇到 shell 特殊字符 (`*`、`(`、`)` 等) 必须转义
- 失败返回 0，成功返回算式结果 (放在 stdout)

```
1 expr ARG1 OP ARG2
2
3 # 加减乘除
4 23:14:32 root@redis01:~# expr 1 + 2
5 3
6 23:16:28 root@redis01:~# expr 1 \* 2
7 2
8 23:16:33 root@redis01:~# expr 1 - 2
9 -1
10 23:16:36 root@redis01:~# expr 4 / 2
```

```

11 2
12 23:16:43 root@redis01:~# expr 5 / 2
13 2
14 23:16:47 root@redis01:~# expr 5 % 2
15 1
16 23:16:50 root@redis01:~#
17
18 # 字符串操作
19 23:16:50 root@redis01:~# expr "string"
20 string
21 # 输出字符串长度
22 23:17:16 root@redis01:~# expr length "string"
23 6
24 # 输出字符串索引
25 23:17:24 root@redis01:~# expr index "string" r
26 3
27 # 正则匹配
28 23:17:45 root@redis01:~# expr match "hello123" "[a-z]"
29 1
30 23:18:07 root@redis01:~#
31
32
33 # 关系比较
34 23:19:19 root@redis01:~# expr "abc" = "def"
35 0
36 23:19:22 root@redis01:~# expr "abc" = "abc"
37 1
38 # < > 必须转义
39 23:19:26 root@redis01:~# expr 4 \> 3
40 1
41 23:19:37 root@redis01:~# expr 4 \< 3
42 0
43 23:19:39 root@redis01:~#

```

4.1.3 计算时注意事项

4.1.3.1 变量和运算符之间是否需要加空格

4.1.3.1.1 算术扩展\$(())

变量与运算符之间可以加也可以不加空格

```

1 22:39:00 root@redis01:~# echo $((1+2))
2 3
3 22:52:53 root@redis01:~# echo $(( 1+2 ))
4 3
5 22:52:55 root@redis01:~# echo $(( 1 + 2 ))
6 3
7 22:52:58 root@redis01:~#
8 22:52:58 root@redis01:~# echo $(( 1 + 2 ))
9 3

```



```
10 22:53:29 root@redis01:~# echo $(( 1 +2))
11 3
12 22:53:31 root@redis01:~#
13
14 # $ 不能和 (()) 分开
15 22:57:04 root@redis01:~# echo $ ((a++))
16 -bash: syntax error near unexpected token `('
```

4.1.3.1.2 let

不需要空格

```
1 23:02:44 root@redis01:~# let a=1+2
2 23:02:49 root@redis01:~# echo $a
3 3
4
5 # 带空格时需加引号
6 23:03:01 root@redis01:~# let "a = 1 + 4"
7 23:03:04 root@redis01:~# echo $a
8 5
9 23:03:06 root@redis01:~#
```

4.1.3.1.3 expr

必须在变量和运算符之间加空格

```
1 23:04:16 root@redis01:~# a=3
2 23:04:19 root@redis01:~# expr $a+1
3 3+1
4 23:04:24 root@redis01:~# expr $a + 1
5 4
6 23:04:26 root@redis01:~# expr $a * 2
7 expr: syntax error: unexpected argument 'note'
8 23:04:33 root@redis01:~# expr $a \* 2
9 6
10 23:04:36 root@redis01:~#
```

注意: * 需要用 * 或引号避免被 shell 展开

4.1.3.1.4 条件表达式 [...] / test

[和] 都是命令，因此所有元素必须分隔开：

```

1 23:04:36 root@redis01:~# echo $a
2 3
3
4 23:06:29 root@redis01:~# if [ $a -eq 1 ];then
5 > echo true
6 > else
7 > echo flase
8 > fi
9 flase
10 23:06:56 root@redis01:~# if [ $a-eq1 ];then echo true; else echo flase;
11 fi
12 true
13 23:07:03 root@redis01:~#

```

4.1.3.1.5 双中括号 [[...]] (bash 扩展)

虽然 bash 会自动处理很多情况，但仍建议运算符分隔：

```

1 23:09:25 root@redis01:~# if [[ $a -eq 1 ]];then echo true; else echo
2 false; fi
3 false
4 23:09:51 root@redis01:~#

```

4.1.3.2 变量和括号之间什么时候需要加空格

4.1.3.2.1 \$(())

算术扩展语法是 **紧贴** 的，不能出现：

```

1 $(( a+1 ))    ✓ 正确
2 $((a+1))      ✓ 正确
3 $(( a + 1 ))  ✓ 正确
4 $ ((a+1))     ✗ 错误（`$` 和 `(` 不能被分开）
5 $ (( a+1 ))   ✗ 错误

```

要点：

- `$` 和 `((` 必须连在一起：`$((`
- 内部内容是否加空格完全无所谓

4.1.3.2.2 \$()

命令替换语法需要连续书写：

```

1 $(cmd)        ✓ 正确
2 $( cmd )      ✓ 正确
3 $ (cmd)       ✗ 错误（被分成 `$` 和 `(`）

```

4.1.3.2.3 ()

```
1 | (ls)           ✓ 正确
2 | ( ls )        ✓ 正确
3 | a=(1 2 3)     ✓ 正确（数组定义，bash）
4 | a= (1 2 3)    ✗ 错误（`a=` 和 `(` 不可分离）
```

要点：括号作为语法结构时，可以前后有空格，但作为赋值的一部分时不能分开。

4.1.3.2.4 []

因为 `[]` 本质是一个命令，`[` 和 `]` 都必须作为独立参数。

```
1 | [ $a -eq 1 ]   ✓ 正确
2 | [$a -eq 1]     ✗ 错误（没有空格）
```

规则：

`[ ... ]` 里外均要留空格，尤其是：

```
1 | [ "$var" = "x" ]
2 | ^ ^           ^ ^
```

4.1.3.2.5 [[]]

```
1 | [[ $a -eq 1 ]] ✓ 正确
2 | [[ $a -eq 1]]  ✗ 错误
3 |
```

4.1.4 bc 命令

`bc` (Basic Calculator) 是 Linux 中的：

- 任意精度计算器（支持大数与高精度小数）
- 支持浮点运算
- 支持变量、函数、条件、循环等 mini 语言

适合：

- 需要小数的场景（bash 自带不支持浮点）
- 大数（超出 bash integer 范围）
- 复杂计算表达式

4.1.4.1 交互式输入

```
1 13:08:48 root@redis01:~# bc
2 bc 1.07.1
3 Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006, 2008, 2012-2017 Free
4 Software Foundation, Inc.
5 This is free software with ABSOLUTELY NO WARRANTY.
6 For details type `warranty`.
7 1+2
8 3
9 3*4
10 12
11 exit
12 0
13 quit
14 13:09:35 root@redis01:~#
15
```

4.1.4.2 非交互输入

```
1 # 非交互模式
2 ### 管道
3 13:09:35 root@redis01:~# echo "3*4" | bc
4 12
5 13:10:24 root@redis01:~# a=2
6 13:10:27 root@redis01:~# b=3
7 13:10:28 root@redis01:~# echo "${a}*${b}" | bc
8
9 ### EOF 多行
10 13:10:40 root@redis01:~# bc << EOF
11 > 3+4
12 > EOF
13 7
14 13:11:22 root@redis01:~#
```

4.1.4.3 精度控制

`scale` 是 `bc` 中最重要的参数：

- 控制 **小数点后位数**
- 默认是 0（也就是 `bc` 默认不算小数！）

```
1 13:13:13 root@redis01:~# echo "scale=3;100/44" | bc
2 2.272
3 13:13:28 root@redis01:~#
```

4.1.5 运算符总结

写法	名字 / 类型	作用	是否推荐
<code>\$(())</code>	算术扩展 (arithmetic expansion)	把整数表达式的结果“展开”到命令里	✅ 强烈推荐
<code>\$()</code>	命令替换 (command substitution)	执行子命令，返回其标准输出	✅ 强烈推荐
<code>(())</code>	算术求值命令 (arithmetic evaluation)	执行整数表达式， 只影响退出状态和变量，不输出	✅ 在条件/循环里很常用
<code>[]</code>	test 命令 (老式条件判断)	字符串/整数/文件测试	🟡 可用但不如 <code>[[]]</code> 安全
<code>[[]]</code>	扩展条件判断 (bash/ksh)	更安全、更强大的条件判断	✅ 推荐 (在 bash/zsh 中)
<code>\$[]</code>	旧式算术扩展	和 <code>\$(())</code> 类似	✗ 不推荐，已弃用

```
1 00:04:18 root@redis01:~# cat test04.sh
2 #!/bin/bash
3 #
4 =====
5 # 脚本基础信息
6 # filename: test04.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 #!/bin/bash
14 # UTF-8 safe script demo
15 # Demonstrates: $(( )), (( )), $( ), [ ], [[ ]], and deprecated $[
16
17 echo "=====
18 echo "  Bash Syntax Demonstration"
19 echo "=====
20
21 a=3
22 echo
23 echo "Initial variable: a=$a"
24 echo "-----"
```

```

25 echo "1. \${( )}    -- Arithmetic expansion (outputs result)"
26 echo "-----"
27 echo "echo \${( a + 2 )} → \$(echo \${( a + 2 )})"
28 echo "Variable not modified: a=$a"
29
30 b=\${( a * 3 + 1 )}
31 echo "b=\${( a * 3 + 1 )} → b=$b"
32
33 echo
34 echo "-----"
35 echo "2. (( ))    -- Arithmetic evaluation (no output, modifies variable)"
36 echo "-----"
37 (( a = a + 5 ))
38 echo "After (( a = a + 5 )) → a=$a"
39 echo "Exit status: $?    (0 = true, non-zero = false)"
40
41 (( a - 8 ))    # Expression equals 0
42 echo "Exit status after (( a - 8 )): $? (non-zero = false)"
43
44 echo
45 echo "-----"
46 echo "3. \${}    -- Command substitution"
47 echo "-----"
48 now=\$(date +%H:%M:%S)
49 echo "Time now: $now"
50
51 lines=\$(wc -l < "$0")
52 echo "This script has $lines lines."
53
54 echo
55 echo "-----"
56 echo "4. [ ]    -- Test command (old style)"
57 echo "-----"
58 x=""
59 if [ -z "$x" ]; then
60     echo "x is empty"
61 else
62     echo "x is not empty"
63 fi
64
65 if [ $a -gt 3 ]; then
66     echo "a > 3"
67 fi
68
69 echo
70 echo "-----"
71 echo "5. [[ ]]    -- Modern test command (advanced and safer)"
72 echo "-----"
73 name="abc123"
74

```

```

75 if [[ $name == a* ]]; then
76     echo "name starts with 'a'"
77 fi
78
79 if [[ $name =~ ^[a-z]+[0-9]+$ ]]; then
80     echo "name matches regex: letters + digits"
81 fi
82
83 echo
84 echo "-----"
85 echo "6. \${ } -- Deprecated arithmetic expansion (avoid using)"
86 echo "-----"
87 echo "\${ 1 + 2 } → ${ 1 + 2 }"
88 echo "Use \${( 1 + 2 )} instead."
89
90 echo
91 echo "=====
92 echo "          End of Demo"
93 echo "=====
94
95
96 00:04:25 root@redis01:~# bash test04.sh
97 =====
98     Bash Syntax Demonstration
99 =====
100
101 Initial variable: a=3
102
103 -----
104 1. ${ ( )} -- Arithmetic expansion (outputs result)
105 -----
106 echo ${ ( a + 2 )} → 5
107 Variable not modified: a=3
108 b=${ ( a * 3 + 1 )} → b=10
109
110 -----
111 2. (( )) -- Arithmetic evaluation (no output, modifies variable)
112 -----
113 After (( a = a + 5 )) → a=8
114 Exit status: 0 (0 = true, non-zero = false)
115 Exit status after (( a - 8 )): 1 (non-zero = false)
116
117 -----
118 3. $( ) -- Command substitution
119 -----
120 Time now: 00:04:29
121 This script has 94 lines.
122
123 -----
124 4. [ ] -- Test command (old style)

```

```
125 -----
126 x is empty
127 a > 3
128
129 -----
130 5. [[ ]] -- Modern test command (advanced and safer)
131 -----
132 name starts with 'a'
133 name matches regex: letters + digits
134
135 -----
136 6. $[ ] -- Deprecated arithmetic expansion (avoid using)
137 -----
138 $[ 1 + 2 ] → 3
139 Use $(( 1 + 2 )) instead.
140
141 =====
142             End of Demo
143 =====
```

4.2 表达式

4.2.1 基础知识

4.2.1.1 基础知识

```
1 格式
2 "真实值 操作符 真实值" 比较运算符 预期值
3 示例
4 3 + 4 > 6
5 要点：
6 表达式应该具有判断的功能
```

4.2.1.2 测试表达式

能够进行测试的表达式常用的有以下几种

表达式	类型	用途	推荐度
test EXP	外部命令	传统测试语法	一般
[EXP]	test 的别名	条件判断	● 中等
[[EXP]]	bash / zsh 内置	增强条件判断	● 强烈推荐
((EXP))	算术求值	整数条件	● 推荐（整数）

4.2.1.3 test 示例

```
1 13:13:28 root@redis01:~# test 1==1
2 13:25:16 root@redis01:~# echo $?
3 0
4 13:25:19 root@redis01:~# test 2 == 2
5 13:25:27 root@redis01:~# echo $?
6 0
7 13:25:28 root@redis01:~# test 2 == 3
8 13:25:31 root@redis01:~# echo $?
9 1
10 13:25:32 root@redis01:~#
11
12
13 # test -v 能测试变量是否为空，为空则为假，非空则为真
14 13:25:32 root@redis01:~# echo $empty
15
16 13:25:51 root@redis01:~# test -v empty
17 13:25:56 root@redis01:~# echo $?
18 1
19 13:25:59 root@redis01:~# empty=aaa
20 13:26:11 root@redis01:~# echo $empty
21 aaa
22 13:26:13 root@redis01:~# test -v empty
23 13:26:15 root@redis01:~# echo $?
24 0
25 13:26:18 root@redis01:~#
```

4.2.1.4 [] 示例

```
1 13:27:26 root@redis01:~# [ 1==1 ];echo $?
2 0
3 13:27:31 root@redis01:~# [ 1 == 1 ];echo $?
4 0
5 13:27:34 root@redis01:~# [ ! 1 == 1 ];echo $?
6 1
7 13:27:38 root@redis01:~#
8
9
```

4.2.2 逻辑表达式

4.2.2.1 基础知识

Shell 中的逻辑表达式用于 **将多个条件组合在一起**，以便在 `if`、`while`、`&&`、`||` 等场景中进行更复杂的判断。

逻辑表达式主要分为三类运算符：

逻辑运算符	含义	符号用法示例
AND (与)	同时为真才为真	<code>cond1 && cond2</code>
OR (或)	任一为真即为真	<code>cond1 cond2</code>
NOT (非)	取反	<code>! cond</code>

4.2.2.2 基础语法

4.2.2.2.1 [] 的逻辑语法 (传统 test)

操作	写法
AND	<code>-a</code>
OR	<code>-o</code>
NOT	<code>!</code>

```
1 16:28:47 root@redis01:~# cat test05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 read -p "请输入一个数字: " num
16
17 if [ $num -ge 18 -a $num -lt 60 ];then
18     echo "成年了"
19 else
20     echo "未成年"
21 fi
22
23 if [ "$user" == "root" -o "$user" == "admin" ];then
24     echo "管理员"
25 fi
26
27 if [ ! -e /etc/test.txt ];then
28     echo "/etc/test.txt not exists"
29 else
30     echo "/etc/test.txt exists"
31 fi
32
```

```

29 16:28:49 root@redis01:~# bash test05.sh
30 请输入一个数字: 23
31 成年了
32 /etc/test.txt not exists
33 16:28:53 root@redis01:~#

```

4.2.2.2 [[]]

特点:

- 不需要对变量加引号 (不会因空值报错)
- 支持正则表达式
- 支持通配符匹配

操作	写法
AND	&&
OR	\
NOT	!

```

1 16:32:53 root@redis01:~# cat test06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16 if [[ $age -ge 18 && $age -lt 60 ]];then
17     echo "成年了"
18 else
19     echo "未成年"
20 fi
21
22 16:32:56 root@redis01:~# bash test06.sh
23 未成年

```

4.2.2.2.3 (()) 的逻辑语法（整数表达式）

特点：

- 只处理整数
- 内部变量不需要 `$`

操作	写法
AND	<code>&&</code>
OR	<code> </code>
NOT	<code>!</code>

```
1 16:35:22 root@redis01:~# cat test07.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test07.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16
17 x=90
18 if (( x > 0 && x < 100 )); then
19     echo "范围内"
20 fi
21
22 16:35:23 root@redis01:~# bash test07.sh
23 范围内
24 16:35:25 root@redis01:~#
```

4.2.2.2.4 使用 `&&`、`||` 直接写命令逻辑

除了测试表达式，shell 可以直接用命令作为逻辑条件：

```
1 mkdir testdir && echo "目录创建成功"
2 rm a.txt || echo "删除失败"
```

逻辑规则：

- `cmd1 && cmd2` : `cmd1` 成功才执行 `cmd2`
- `cmd1 || cmd2` : `cmd1` 失败才执行 `cmd2`

4.2.2.3 逻辑表达式组合使用

逻辑表达式可以组合形成复杂条件判断。
以下表格展示各测试语法的逻辑组合写法：

表：不同语法的逻辑表达式写法对比

测试方式	AND	OR	NOT	推荐度	备注
<code>[]</code>	<code>-a</code>	<code>-o</code>	<code>!</code>	☆☆	语法严格，容易出错
<code>[[]]</code>	<code>&&</code>	<code>\ </code>	<code>!</code>	☆☆☆☆☆	安全、推荐
<code>(())</code> (整数)	<code>&&</code>	<code>\ </code>	<code>!</code>	☆☆☆☆☆	仅整数判断
<code>test</code> (命令)	<code>-a</code>	<code>-o</code>	<code>!</code>	☆☆	与 <code>[]</code> 一样

4.2.2.4 短路运算

4.2.2.4.1 什么是短路运算

短路运算 (Short-Circuit Evaluation) 指：

当表达式的最终结果已确定后，后续表达式就不会执行。
常用于：

- 多条件判断
- 避免不必要的计算
- 命令执行控制
- 错误处理链

4.2.2.4.2 短路规则

表达式	含义说明	执行规则 (短路逻辑)	示例		
<code>A && B</code>	逻辑与 (AND)	A 为真 (退出状态 0) 时才执行 B ；A 为假 (非 0) 则 B 不执行	<code>[-f file] && echo "存在"</code>		
<code>A B</code>	逻辑或 (OR)	A 为假 (退出状态非 0) 时才执行 B ；A 为真 (0) 则 B 不执行	<code>command \ echo "失败"</code>		
<code>! A</code>	逻辑非 (NOT)	反转 A 的真假 (A 成功变失败，失败变成功)	<code>! [-f file]</code>		

表达式	含义说明	执行规则（短路逻辑）	示例		
-a (AND)	[] 中逻辑与 (老式、不推荐)	两个条件都为真时整体为真；无真正短路（test 会解析整个表达式）	<code>[-f file -a -s file]</code>		
-o (OR)	[] 中逻辑或 (老式、不推荐)	任一条件为真整个表达式为真；无真正短路	<code>["\$u" = root -o "\$u" = admin]</code>		
&& /	命令级短路控制流（最推荐）	跟 C/JavaScript 逻辑一致，用于根据命令成功/失败执行后续命令	<code>make && make install</code>		

```

1  # AND (&&) 短路逻辑
2
3  A 成功 (0) → 执行 B
4  A 失败 (!=0) → 不执行 B
5
6  逻辑图：
7  A ----成功----> B 执行
8  A ----失败----> B 跳过
9
10
11 # OR (||) 短路逻辑
12
13 A 失败 (!=0) → 执行 B
14 A 成功 (0) → 不执行 B
15
16 逻辑图：
17 A ----失败----> B 执行
18 A ----成功----> B 跳过
19

```

4.2.2.4.3 命令级短路

Shell 中所有命令都有“成功=0，失败≠0”的返回码。
短路逻辑依赖这个规则。

4.2.2.4.3.1 && —— 前一个命令成功才执行后一个命令

```

1 | mkdir /tmp/test && echo "目录创建成功"

```

解释：

- 若 mkdir 成功 → 执行 echo
- 若 mkdir 失败 → echo 不执行

4.2.2.4.3.2 `||` 前一个命令失败才执行后一个命令

```
1 command1 && command2 || command3
2 17:00:09 root@redis01:~# [ -f access.log ] && tail -n 10 access.log || echo
   "文件不存在"
3 文件不存在
4 17:00:21 root@redis01:~#
```

效果:

- 若 `command1` 成功 → 执行 `command2`
 - 若 `command1` 失败 → 执行 `command3`
- 用于: 主逻辑失败→走补救逻辑

4.2.2.4.3.3 `[ ]` 与短路 (`-a -o`)

传统 `test` 的逻辑:

操作符	作用
<code>-a</code>	AND
<code>-o</code>	OR

```
1 if [ -f "$file" -a -s "$file" ]; then
2     echo "文件存在且非空"
3 fi
```

▲ 注意:

- `[ ]` 中逻辑优先级低
- 容易产生歧义
- 不推荐写复杂逻辑
- 建议使用 `[[ ]]`

4.2.2.4.3.4 `[[ ]]` 的短路 (推荐)

`[[ ]]` 是 Bash 的增强测试语法, 其 `&&`、`||` 和命令级一致:

```
1 # 年龄范围判断
2 if [[ $age -ge 18 && $age -le 60 ]]; then
3     echo "合法年龄"
4 fi
5
6 # 用户名判断
7 if [[ $user == "root" || $user == "admin" ]]; then
8     echo "管理员"
```

```

9 fi
10
11 # 防止变量为空导致错误
12 if [[ -z $input || $input == quit ]]; then
13     echo "退出程序"
14 fi
15

```

4.2.2.4.3.5 (()) 数字短路判断 (C风格)

```

1 if (( x > 0 && x < 10 )); then
2     echo "范围内"
3 fi
4

```

与C类似:

- `x > 0` 为假, 后面的表达式不会执行
- 只适用于整数比较

4.2.2.4.4 综合示例

4.2.2.4.4.1 用户不存在则退出

```

1 id "$user" &>/dev/null || { echo "用户不存在"; exit 1; }

```

说明:

- `id` 执行失败 → 左边返回非 0 → 执行 `{ ... }`
- 成功则不执行右侧代码块

4.2.2.4.4.2 配置文件存在才解析

```

1 [ -f config.yaml ] && parse_config config.yaml

```

4.2.2.4.4.3 判断字符串是否有效 (空、空格、null 全排除)

```

1 if [[ -n $str && $str != "null" ]]; then
2     echo "有效字符串"
3 fi

```

4.2.2.4.5 短路运算最佳实践

场景	推荐语法
字符串比较	<code>[[]]</code>

场景	推荐语法
数字比较	<code>(())</code>
文件测试 + 多逻辑	<code>[[]]</code>
简单命令控制流	<code>command1 && command2, `command1</code>
POSIX 兼容脚本	<code>[]</code> (少用复杂逻辑)

4.2.2.5 综合实践

4.2.2.5.1 判断用户合法性

```

1 read -p "输入用户名: " user
2
3 if [[ $user == "admin" || $user == "root" ]]; then
4     echo "超级管理员"
5 else
6     echo "普通用户"
7 fi

```

4.2.2.5.2 判断数字是否在区间内

```

1 read -p "输入一个数: " n
2
3 if (( n >= 1 && n <= 100 )); then
4     echo "数字在 1~100 范围"
5 else
6     echo "数字不在范围"
7 fi

```

4.2.2.5.3 判断文件合法

```

1 file="config.yaml"
2
3 if [[ -f $file && -s $file ]]; then
4     echo "配置文件存在且非空"
5 else
6     echo "配置文件不存在或为空"
7 fi

```

4.2.2.5.4 判断字符串是否匹配复杂规则

需求：字符串必须以字母开头，并且只包含字母数字下划线

```

1 read -p "输入字符串: " str
2
3 if [[ $str =~ ^[a-zA-Z][a-zA-Z0-9_]*$ ]]; then
4     echo "格式合法"
5 else
6     echo "格式不合法"
7 fi

```

4.2.2.5.5 多条件综合判断（文件 + 数字 + 字符串）

```

1 user="admin"
2 size=$(stat -c %s access.log)
3
4 if [[ -f access.log && $size -gt 1000 && ( $user == "admin" || $user ==
  "root" ) ]]; then
5     echo "满足所有条件：可查看访问日志"
6 else
7     echo "权限不足"
8 fi
9

```

4.2.3 字符串表达式

字符串表达式用于判断字符串是否为空、是否相等、是否匹配特定内容等，是 Shell 条件判断中使用最频繁的操作之一。

4.2.3.1 基础知识

4.2.3.1.1 常用字符串测试表达式

表达式	含义
<code>-z STRING</code>	判断字符串是否为空（长度为 0）
<code>-n STRING</code>	判断字符串是否非空
<code>STRING = STRING</code>	判断两个字符串是否相等
<code>STRING != STRING</code>	判断两个字符串是否不相等
<code>STRING < STRING</code>	字典序比较（仅限 <code>[[]]</code> ）
<code>STRING > STRING</code>	字典序比较（仅限 <code>[[]]</code> ）

4.2.3.1.2 示例

```

1 # 传统写法
2 22:43:30 root@redis01:~# a=1
3 22:43:33 root@redis01:~# b=2
4 22:43:35 root@redis01:~# [ "$a" = "$b" ]

```

```

5 22:43:39 root@redis01:~# echo $?
6 1
7 22:43:41 root@redis01:~# [ "$a" != "$b" ]
8 22:43:47 root@redis01:~# echo $?
9 0
10 22:43:48 root@redis01:~#
11
12
13 # 推荐写法
14 22:43:48 root@redis01:~# a=1
15 22:44:08 root@redis01:~# b=2
16 22:44:11 root@redis01:~# [[ $a = $b ]]
17 22:44:20 root@redis01:~# echo $?
18 1
19 22:44:22 root@redis01:~# [[ $a != $b ]]
20 22:44:25 root@redis01:~# echo $?
21 0
22 22:44:27 root@redis01:~#

```

✖ 为什么推荐 [[]]?

- 支持更多操作符（如 <、>）
- 空值不会导致语法错误
- 不需要转义 < > 号

4.2.3.2 内容比较

字符串内容比较常用于脚本输入判断、菜单选择等。

4.2.3.2.1 ✨ 相等比较 == 或 =

```

1 22:48:39 root@redis01:~/shell# cat demo01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 str1="string1"
16 str2="string1"
17
18 if [[ ${str1} == ${str2} ]];then

```

```

15     echo "str1 等于 str2"
16 fi
17
18
19 if [[ ${str1} = ${str2} ]];then
20     echo "str1 等于 str2"
21 fi
22
23
24 22:48:40 root@redis01:~/shell# bash demo01.sh
25 str1 等于 str2
26 str1 等于 str2
27 22:48:41 root@redis01:~/shell#

```

4.2.3.2.2 ✨ 不相等比较 `!=`

```

1 22:49:53 root@redis01:~/shell# cat demo01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 str1="string1"
16 str2="string2"
17
18
19 if [[ ${str1} != ${str2} ]];then
20     echo "str1 不等于 str2"
21 fi
22
23
24 22:49:55 root@redis01:~/shell# bash demo01.sh
25 str1 不等于 str2
26
27 22:49:57 root@redis01:~/shell#

```

4.2.3.2.3 ✨ 字典序比较 `<`、`>`

(仅 `[[ ]]` 支持, 且无需转义)

```

1 22:51:39 root@redis01:~/shell# cat demo01.sh
2 #!/bin/bash

```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo01.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11
12 a=10
13 b=20
14 if [[ ${a} < ${b} ]];then
15     echo "a 小于 b"
16 fi
17 22:51:40 root@redis01:~/shell# bash demo01.sh
18 a 小于 b
19 22:51:42 root@redis01:~/shell#

```

在 `[ ]` 中必须写成: `[ "$a" \< "$b" ]`

4.2.3.3 空值判断

空值判断是写 Shell 最容易出错的地方，因此必须掌握。

4.2.3.3.1 🍷 判断为空: `-z`

```

1  13:45:49 root@redis01:~/shell# cat demo02.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo02.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 var=""
12
13 if [[ -z ${var} ]];then
14     echo "var 变量为空"
15 fi

```

```

16
17 13:45:50 root@redis01:~/shell# bash demo02.sh
18 var 变量为空
19 13:45:51 root@redis01:~/shell#

```

4.2.3.3.2 🟡 判断非空: `-n`

```

1 13:46:47 root@redis01:~/shell# cat demo03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 var="aaa"
16
17 if [[ -n ${var} ]];then
18     echo "var 变量非空"
19 fi
20
21 13:46:51 root@redis01:~/shell# bash demo03.sh
22 var 变量非空
23 13:46:55 root@redis01:~/shell#

```

4.2.3.3.3 ⚠️ 容易犯的错误

```

1 # 如果变量为空，我们执行下面的命令
2 if [ $var == "aaa" ];then
3     ....
4
5 # 会变成，这就是空变量导致错误
6 if [ == "aaa" ];then
7
8 # 正确写法，加双引号、双中括号
9 if [ "$var" = "abc" ]; then
10 if [[ $var == abc ]]; then

```

4.2.3.4 综合案例

以下综合案例结合了：

- 空值判断
- 内容比较
- 多条件判断
- 用户输入

4.2.3.4.1 🚩 案例：用户输入菜单判断脚本

```
1 21:49:11 root@redis01:~/shell# cat demo04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 echo "请输入指令：（start/stop/restart）"
16 read action
17
18 if [[ -z ${action} ]];then
19     echo "错误：没有输入指令"
20     exit 1
21 fi
22
23 if [[ ${action} = "start" ]];then
24     echo "start..."
25 elif [[ ${action} = "stop" ]];then
26     echo "stop..."
27 elif [[ ${action} = "restart" ]];then
28     echo "restart..."
29 else
30     echo "无效指令：${action}"
31 fi
32
33 21:49:13 root@redis01:~/shell# bash demo04.sh
34 请输入指令：（start/stop/restart）
35
36 错误：没有输入指令
37
38 21:49:16 root@redis01:~/shell# bash demo04.sh
39 请输入指令：（start/stop/restart）
40
41 aaa
```

```
35 | 无效指令: aaa
36 | 21:49:19 root@redis01:~/shell# bash demo04.sh
37 | 请输入指令: (start/stop/restart)
38 | start
39 | start...
40 | 21:49:22 root@redis01:~/shell#
```

4.2.3.5 逻辑总结表格

判断内容	使用表达式	场景示例
空值判断	-z	检查用户是否输入内容
非空判断	-n	判断变量是否有值
相等	== / =	菜单选择、配置检查
不相等	!=	输入过滤
字典序	< / >	字母排序、版本比较（简单场景）

4.2.4 文件表达式

文件表达式用于判断文件是否存在、类型是否正确、权限是否满足等，是脚本编写中非常常见的判断类型。

4.2.4.1 基础知识

Shell 提供了一组测试文件状态的表达式，可用于 test、[] 和 []] 中。

4.2.4.1.1 常用文件测试操作符

4.2.4.1.1.1 文件存在性判断

操作符	含义	说明	示例
-e FILE	文件存在	不关心类型	[[-e file]]
-a FILE	文件存在	等价于 -e（不推荐）	[[-a file]]
🔴 说明:			

- -a 已不推荐使用，建议统一使用

4.2.4.1.1.2 文件类型判断

操作符	含义	说明	示例
-f FILE	普通文件	如文本、脚本、二进制	[[-f file]]
-d FILE	目录	Directory	[[-d /etc]]
-L FILE	符号链接	Soft link	[[-L link]]

操作符	含义	说明	示例
<code>-h FILE</code>	符号链接	等价于 <code>-L</code>	<code>[[-h link]]</code>
<code>-b FILE</code>	块设备文件	磁盘设备	<code>[[-b /dev/sda]]</code>
<code>-c FILE</code>	字符设备文件	tty、console	<code>[[-c /dev/tty]]</code>
<code>-p FILE</code>	管道文件	FIFO	<code>[[-p pipe]]</code>
<code>-S FILE</code>	套接字文件	socket	<code>[[-S sock]]</code>

4.2.4.1.1.3 文件大小判断

操作符	含义	说明	示例
<code>-s FILE</code>	文件非空	size > 0	<code>[[-s file]]</code>

4.2.4.1.1.4 文件权限判断（针对当前用户）

操作符	含义	说明	示例
<code>-r FILE</code>	可读	read permission	<code>[[-r file]]</code>
<code>-w FILE</code>	可写	write permission	<code>[[-w file]]</code>
<code>-x FILE</code>	可执行	execute permission	<code>[[-x file]]</code>
 注意：			
权限判断是基于 当前执行脚本的用户身份 。			

4.2.4.1.1.5 文件属主和属组判断

操作符	含义	说明	示例
<code>-0 FILE</code>	当前用户是属主	Owner	<code>[[-0 file]]</code>
<code>-G FILE</code>	当前用户属于属组	Group	<code>[[-G file]]</code>

4.2.4.1.1.6 时间戳比较（文件新旧）

操作符	含义	说明	示例
<code>FILE1 -nt FILE2</code>	FILE1 更新	newer than	<code>[[a -nt b]]</code>
<code>FILE1 -ot FILE2</code>	FILE1 更旧	older than	<code>[[a -ot b]]</code>

4.2.4.1.1.7 inode/硬链接判断

操作符	含义	说明	示例
<code>FILE1 -ef FILE2</code>	同一文件	同 inode	<code>[[a -ef b]]</code>
常用于判断：			

- 硬链接
- 不同路径是否指向同一文件

4.2.4.1.1.8 特殊权限判断

操作符	含义	说明
<code>-u FILE</code>	SUID 位	Set User ID
<code>-g FILE</code>	SGID 位	Set Group ID
<code>-k FILE</code>	Sticky 位	粘滞位

```
1 | [[ -u /usr/bin/passwd ]] && echo "设置了 SUID"
```

4.2.4.1.2 文件表达式使用格式

传统写法

```
[ -e filename ]
```

推荐写法

```
[[ -e filename ]]
```

原因：`[[ ]]` 支持更安全的字符串处理，避免文件名中包含空格或通配符导致错误。

4.2.4.2 属性判断

属性判断用于确定文件的存在状态、类型等。

4.2.4.2.1 文件存在性判断 `-e`

```
1 10:57:30 root@redis01:~/shell# cat demo05.sh
2 #!/bin/bash
3 #
4 =====
5 # 脚本基础信息
6 # filename: demo05.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13
14 file_path=/etc/passwd
15 if [[ -e ${file_path} ]];then
16     echo "${file_path} exists"
17 else
18     echo "${file_path} not exists"
19 fi
```

```
17
18 10:57:31 root@redis01:~/shell# bash demo05.sh
19 /etc/passwd exists
20 10:57:33 root@redis01:~/shell#
```

4.2.4.2.2 判断是否为普通文件 -f

```
1 11:02:38 root@redis01:~/shell# cat file_test_general-file.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: file_test_general-file.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 file_path=/etc/passwd
16 if [[ -f ${file_path} ]];then
17     echo "${file_path} is a general file"
18 else
19     echo "${file_path} is not a general file"
20 fi
21
22 11:02:39 root@redis01:~/shell# bash file_test_general-file.sh
23 /etc/passwd is a general file
24 11:02:40 root@redis01:~/shell#
```

4.2.4.2.3 判断是否为目录 -d

```
1 11:06:43 root@redis01:~/shell# cat file_test_is_dir.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: file_test_is_dir.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 file_path=/etc/passwd
```

```

12 dir_path=/etc
13 if [[ -d ${file_path} ]];then
14     echo "${file_path} is a dir"
15 else
16     echo "${file_path} is not a dir"
17 fi
18
19
20 if [[ -d ${dir_path} ]];then
21     echo "${dir_path} is a dir"
22 else
23     echo "${dir_path} is not a dir"
24 fi
25 11:06:45 root@redis01:~/shell# bash file_test_is_dir.sh
26 /etc/passwd is not a dir
27 /etc is a dir
28 11:06:46 root@redis01:~/shell#

```

4.2.4.2.4 判断是否为符号链接 -L

```

1 11:45:18 root@redis01:~/shell# cat file_test_is_softlink.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: file_test_is_softlink.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 file_path=/usr/bin/vi
16 if [[ -L ${file_path} ]];then
17     echo "${file_path} is soft link file"
18 else
19     echo "${file_path} is not soft link file"
20 fi
21 11:45:19 root@redis01:~/shell# bash file_test_is_softlink.sh
22 /usr/bin/vi is soft link file
23 11:45:21 root@redis01:~/shell#

```

4.2.4.2.5 判断文件是否非空 -s

```

1 11:50:54 root@redis01:~/shell# cat file_test_is_not-empty.sh
2 #!/bin/bash

```

```

3  #
=====
====
4  # Script basic information
5  # filename: file_test_is_null.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
=====
====
11 file_path=/etc/passwd
12 empty_path=/root/empty.txt
13 touch ${empty_path}
14 if [[ -s ${file_path} ]];then
15     echo "${file_path} is not empty"
16 else
17     echo "${file_path} is empty"
18 fi
19
20
21 if [[ -s ${empty_path} ]];then
22     echo "${empty_path} is not empty"
23 else
24     echo "${empty_path} is empty"
25 fi
26 11:50:55 root@redis01:~/shell# bash file_test_is_not-empty.sh
27 /etc/passwd is not empty
28 /root/empty.txt is empty
29 11:50:58 root@redis01:~/shell#

```

4.2.4.2.6 总结

表达式	判断内容
<code>-e FILE</code>	是否存在
<code>-f FILE</code>	是否为普通文件
<code>-d FILE</code>	是否为目录
<code>-L FILE</code>	是否符号链接
<code>-s FILE</code>	文件是否非空

4.2.4.3 权限判断

用于判断文件是否具备读、写、执行权限。

4.2.4.3.1 判断可读 -r

```
1 11:53:53 root@redis01:~/shell# cat file_test_is_readable.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: file_test_is_readable.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 if [[ -r "/etc/passwd" ]]; then
16     echo "文件可读"
17 fi
18
19 11:53:55 root@redis01:~/shell# bash file_test_is_readable.sh
20 文件可读
21 11:53:56 root@redis01:~/shell#
```

4.2.4.3.2 判断可写 -w

```
1 11:55:02 root@redis01:~/shell# cat file_test_is_writeable.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: file_test_is_writeable.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 if [[ -w "./demo1.sh" ]]; then
16     echo "文件可写"
17 fi
```

4.2.4.3.3 判断可执行 -x

```
1 12:00:44 root@redis01:~/shell# cat file_test_is_executable.sh
2 #!/bin/bash
```

```

3  #
  =====
  ====
4  # Script basic information
5  # filename: file_test_is_executable.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 if [[ -x "/usr/bin/bash" ]]; then
12     echo "文件可执行"
13 fi
14
15 12:00:46 root@redis01:~/shell# bash file_test_is_executable.sh
16 文件可执行
17 12:00:48 root@redis01:~/shell#

```

4.2.4.3.3 总结

表达式	含义
<code>-r FILE</code>	文件可读
<code>-w FILE</code>	文件可写
<code>-x FILE</code>	文件可执行

4.2.4.4 其他判断

除了属性和权限，Shell 还提供一些其他测试功能。

4.2.4.4.1 判断文件是否更新（比另一个文件新） `FILE1 -nt FILE2`

```

1 12:02:59 root@redis01:~/shell# cat file_test_newer.sh
2 #!/bin/bash
3 #
  =====
  ====
4 # Script basic information
5 # filename: file_test_newer.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 if [[ "./a.txt" -nt "./b.txt" ]]; then

```

```

12     echo "a.txt 较新"
13 fi
14
15 12:03:05 root@redis01:~/shell# touch b.txt
16 12:03:12 root@redis01:~/shell# touch a.txt
17 12:03:14 root@redis01:~/shell# bash file_test_newer.sh
18 a.txt 较新
19 12:03:22 root@redis01:~/shell#

```

4.2.4.4.2 判断两个文件是否属于同一设备 inode `FILE1 -ef FILE2`

```

1 12:04:04 root@redis01:~/shell# cat file_test_inode.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: file_test_inode.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 if [[ "/bin/sh" -ef "/usr/bin/sh" ]]; then
16     echo "两个路径指向同一个文件"
17 fi
18
19 12:04:05 root@redis01:~/shell# bash file_test_inode.sh
20 两个路径指向同一个文件
21 12:04:06 root@redis01:~/shell#

```

4.2.4.4.3 总结

表达式	说明
<code>FILE1 -nt FILE2</code>	FILE1 比 FILE2 新
<code>FILE1 -ot FILE2</code>	FILE1 比 FILE2 旧
<code>FILE1 -ef FILE2</code>	FILE1 与 FILE2 是同一个文件（硬链接或同 inode）

4.2.4.5 综合案例

```

1 13:59:35 root@redis01:~/shell# cat file_test_comprehensive_case.sh
2 #!/bin/bash
3 #
4 =====
5 ====

```



```

4  # Script basic information
5  # filename: file_test_comprehensive_case.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 #!/bin/bash
12
13 file="./run.sh"
14
15 if [[ ! -e "$file" ]]; then
16     echo "文件不存在"
17     exit 1
18 fi
19
20 if [[ -f "$file" ]]; then
21     echo "这是一个普通文件"
22 fi
23
24 if [[ -x "$file" ]]; then
25     echo "文件可执行，现在运行它..."
26     ./"$file"
27 else
28     echo "文件不可执行，正在赋予执行权限..."
29     chmod +x "$file"
30 fi
31
32
33 13:59:37 root@redis01:~/shell# bash file_test_comprehensive_case.sh
34 文件不存在
35 13:59:49 root@redis01:~/shell# touch run.sh
36 13:59:54 root@redis01:~/shell# bash file_test_comprehensive_case.sh
37 这是一个普通文件
38 文件不可执行，正在赋予执行权限...
39 14:00:01 root@redis01:~/shell#

```

4.2.5 数字表达式

数字表达式用于对 **整数** 进行大小比较、等值判断、算术运算结果判断等，常用于循环控制、条件分支、脚本执行结果判断。

 **注意：**Shell 的数字表达式 **默认只支持整数**，不支持浮点数（需借助 `bc`、`awk` 等）。

4.2.5.1 基础知识

4.2.5.1.1 数字比较操作符（仅适用于整数）

操作符	含义	对应数学含义
-eq	等于	==
-ne	不等于	!=
-gt	大于	>
-lt	小于	<
-ge	大于等于	>=
-le	小于等于	<=

4.2.5.1.2 数字表达式写法

① 使用 `[ ]`（传统写法）

```
[ 10 -gt 5 ]
```

② 使用 `[[ ]]`（推荐）

```
[[ 10 -gt 5 ]]
```

③ 使用算术表达式 `(( ))`（最推荐）

```
(( 10 > 5 ))
```

🔴 `(( ))` 特点：

- 不需要 `$`
- 支持数学运算
- 语法更接近编程语言

写法	是否推荐	特点
<code>[]</code>	一般	需要操作符，易出错
<code>[[]]</code>	推荐	更安全
<code>(())</code>	★ 强烈推荐	直观、支持运算

4.2.5.2 具体实践

4.2.5.2.1 等值与大小判断

```
1 14:08:01 root@redis01:~/shell/num# cat demo01.sh
2 #!/bin/bash
```

```

3  #
   =====
   ====
4  # Script basic information
5  # filename: demo01.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 num=10
12 if [[ ${num} -eq 10 ]];then
13     echo "num 等于 10"
14 fi
15
16 14:08:02 root@redis01:~/shell/num# bash demo01.sh
17 num 等于 10
18 14:08:04 root@redis01:~/shell/num#

```

4.2.5.2.1 使用 (()) 进行判断

```

1  14:08:45 root@redis01:~/shell/num# cat demo02.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # Script basic information
5  # filename: demo02.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 num=20
12
13 if (( num > 10 )); then
14     echo "num 大于 10"
15 fi
16
17 14:08:48 root@redis01:~/shell/num# bash demo02.sh
18 num 大于 10
19 14:08:50 root@redis01:~/shell/num#

```

4.2.5.2.2 数字范围判断

```
1 14:09:39 root@redis01:~/shell/num# cat demo03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: demo03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 score=85
16
17 if (( score >= 60 && score < 90 )); then
18     echo "成绩合格"
19 fi
20
21 14:09:41 root@redis01:~/shell/num# bash demo03.sh
22 成绩合格
```

4.2.5.2.3 结合用户输入的数字判断

```
1 14:11:22 root@redis01:~/shell/num# cat demo04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: demo04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 read -p "请输入一个数字: " n
16
17 if (( n % 2 == 0 )); then
18     echo "偶数"
19 else
20     echo "奇数"
21 fi
22
```

```

19
20 14:11:23 root@redis01:~/shell/num# bash demo04.sh
21 请输入一个数字: 3
22 奇数
23 14:11:28 root@redis01:~/shell/num# bash demo04.sh
24 请输入一个数字: 4
25 偶数
26 14:11:30 root@redis01:~/shell/num#

```

4.2.5.3 脚本执行判断

脚本中常通过 **退出状态码 (exit code)** 判断命令是否执行成功。

4.2.5.3.1 什么是 \$?

- `$?` 表示 **上一条命令的退出状态**
- `0` : 成功
- **非 0** : 失败

4.2.5.3.2 使用 \$? 判断执行结果

```

1 14:12:49 root@redis01:~/shell/num# cat demo05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: demo05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 ls /tmp >/dev/null 2>&1
16
17 if [[ $? -eq 0 ]]; then
18     echo "命令执行成功"
19 else
20     echo "命令执行失败"
21 fi
22
23 14:12:50 root@redis01:~/shell/num# bash demo05.sh
24 命令执行成功
25 14:12:51 root@redis01:~/shell/num#

```

4.2.5.3.3 推荐写法：直接在 if 中判断

```
1 14:18:00 root@redis01:~/shell/num# cat demo06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: demo06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 if ls /tmp >/dev/null 2>&1; then
16     echo "命令执行成功"
17 else
18     echo "命令执行失败"
19 fi
20
21 14:18:00 root@redis01:~/shell/num# bash demo06.sh
22 命令执行成功
23 14:18:02 root@redis01:~/shell/num#
```

4.2.5.3.4 结合 exit 状态的完整示例

```
1 14:19:15 root@redis01:~/shell/num# cat demo07.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: demo07.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 ping -c 1 192.168.121.2 >/dev/null 2>&1
16
17 if [[ $? -ne 0 ]]; then
18     echo "网络不可达"
19     exit 1
20 fi
```

```
17
18 echo "网络正常"
19
20 14:19:18 root@redis01:~/shell/num# bash demo07.sh
21 网络不可达
22 14:19:29 root@redis01:~/shell/num#
```

4.2.5.3.5 脚本执行判断总结表

判断方式	示例	说明
<code>\$?</code>	<code>[[\$? -eq 0]]</code>	判断上一条命令
if 直接判断	<code>if command; then</code>	★ 推荐
exit 码	<code>exit 1</code>	自定义错误状态

4.2.5.4 总结

场景	推荐写法
数字比较	<code>((a > b))</code>
等值判断	<code>((a == b))</code>
多条件	<code>((a > 0 && b < 10))</code>
判断命令成功	<code>if command; then</code>
判断退出状态	<code>[[\$? -eq 0]]</code>

4.3 表达式进阶

4.3.1 [[]] 测试进阶

`[[ ]]` 是 Bash 提供的 **增强型条件测试结构**，比传统 `[ ]` 更安全、更强大，支持模式匹配、逻辑组合、正则表达式等高级特性。

4.3.1.1 基础知识

4.3.1.1.1 `[[ ]]` 与 `[ ]` 的核心区别

对比项	<code>[]</code>	<code>[[]]</code>
是否 Bash 内建	是（兼容 POSIX）	是（Bash 扩展）
空变量是否报错	可能	不会
是否支持通配符	否（需转义）	✅ 支持
是否支持正则	×	✅ <code>=~</code>
是否需要转义 <code>< ></code>	是	×

对比项	[]	[[]]
逻辑运算	-a -o	`&&
🔴 推荐原则:		
👉 在 Bash 脚本中, 优先使用 [[]]		

4.3.1.1.2 基本语法结构

```
1 [[ 条件表达式 ]]  
2 [[ -f file && -r file ]]
```

4.3.1.1.3 逻辑运算符 (仅 [[]] 支持)

运算符	含义
&&	逻辑与
\ \	逻辑或
!	逻辑非

```
[[ -f file && -x file ]]
```

4.3.1.2 内容匹配实践

[[]] 支持 **通配符匹配** 和 **正则匹配**, 非常适合字符串判断。

4.3.1.2.1 通配符匹配 (模式匹配)

支持的通配符:

符号	含义
*	任意长度字符串
?	单个字符
[abc]	字符集合

4.3.1.2.1.1 前缀匹配

```
1 17:12:35 root@redis01:~/shell/expression# cat demo01.sh  
2 #!/bin/bash  
3 #  
  =====  
  ====  
4 # Script basic information  
5 # filename: demo01.sh  
6 # name: xuruizhao
```



```

7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 str="user_admin"
12
13 if [[ $str == user* ]]; then
14     echo "以 user 开头"
15 fi
16
17 17:12:37 root@redis01:~/shell/expression# /bin/bash demo01.sh
18 以 user 开头
19 17:12:44 root@redis01:~/shell/expression#

```

4.3.1.2.1.2 后缀匹配

```

1  17:13:29 root@redis01:~/shell/expression# cat demo02.sh
2  #!/bin/bash
3  #
    =====
    ====
4  # Script basic information
5  # filename: demo02.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 file="log_2024.txt"
12
13 if [[ $file == *.txt ]]; then
14     echo "文本文件"
15 fi
16 17:13:31 root@redis01:~/shell/expression# bash demo02.sh
17 文本文件
18 17:13:33 root@redis01:~/shell/expression#

```

4.3.1.2.1.3 单字符匹配

```

1  17:14:10 root@redis01:~/shell/expression# cat demo03.sh
2  #!/bin/bash
3  #
    =====
    ====

```

```

4  # Script basic information
5  # filename: demo03.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
=====
====
11 code="a1"
12
13 if [[ $code == a? ]]; then
14     echo "a 开头，长度为 2"
15 fi
16
17 17:14:13 root@redis01:~/shell/expression# bash demo03.sh
18 a 开头，长度为 2

```

4.3.1.2.2 正则表达式匹配 =~

`[[ STRING =~ REGEX ]]`

⚠ 注意：

- 正则 **不要加引号**
- 仅支持 ERE（扩展正则）

4.3.1.2.2.1 判断是否为数字

```

1  17:48:06 root@redis01:~/shell/expression# cat demo04.sh
2  #!/bin/bash
3  #
=====
====
4  # Script basic information
5  # filename: demo04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
=====
====
11 input="12345"
12
13 if [[ $input =~ ^[0-9]+$ ]]; then
14     echo "纯数字"
15 fi
16
17 17:48:08 root@redis01:~/shell/expression# bash demo04.sh

```

```
18 纯数字
19 17:48:10 root@redis01:~/shell/expression#
```

4.3.1.2.2 邮件格式判断

```
1 17:48:56 root@redis01:~/shell/expression# cat demo05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: demo05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 email="test@example.com"
16
17 if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
18     echo "合法邮箱"
19 fi
20
21 17:48:57 root@redis01:~/shell/expression# bash demo05.sh
22 合法邮箱
23 17:48:59 root@redis01:~/shell/expression#
```

4.3.1.2.3 总结

匹配方式	操作符	适用场景
字符串相等	<code>==</code>	精确匹配
通配符	<code>== pattern</code>	文件名、前后缀
正则	<code>=~</code>	输入验证

4.3.1.3 文件匹配实践

`[[ ]]` 在文件判断中更安全，支持逻辑组合。

4.3.1.3.1 基础文件匹配

```
1 file="/etc/passwd"
2
3 if [[ -f $file ]]; then
4     echo "普通文件"
5 fi
```

4.3.1.3.2 组合条件判断

```
1 if [[ -f "$file" && -r "$file" ]]; then
2     echo "文件存在且可读"
3 fi
```

4.3.1.3.3 通配符结合文件判断

```
1 for f in /var/log/*.log; do
2     if [[ -s $f ]]; then
3         echo "$f 非空"
4     fi
5 done
```

4.3.1.3.4 综合案例：脚本文件检查

```
1 #!/bin/bash
2
3 script="$1"
4
5 if [[ -z $script ]]; then
6     echo "请指定脚本文件"
7     exit 1
8 fi
9
10 if [[ ! -f $script ]]; then
11     echo "文件不存在"
12     exit 2
13 fi
14
15 if [[ ! -x $script ]]; then
16     echo "文件不可执行"
17     exit 3
18 fi
19
20 echo "脚本检查通过，可执行"
21
```

4.3.1.3.5 文件匹配实践总结表

判断需求	推荐写法
文件存在	<code>[[-e file]]</code>
普通文件	<code>[[-f file]]</code>
可执行	<code>[[-x file]]</code>
多条件	<code>[[-f file && -x file]]</code>
否定判断	<code>[[! -f file]]</code>

4.3.1.4 最佳实践

- ✓ 在 Bash 中 **优先使用** `[[ ]]`
- ✓ 使用 `&& || !` 代替 `-a -o`
- ✓ 内容匹配首选通配符，复杂校验用正则
- ✓ 文件判断与逻辑组合放在一个 `[[ ]]` 中

4.3.2 集合基础（逻辑运算）

在 Shell 条件判断中，“集合”通常指 **多个条件的组合判断**。
通过 **或 (OR)**、**与 (AND)**、**非 (NOT)**，可以构建复杂的判断逻辑。

4.3.2.1 基础知识

4.3.2.1.1 三种基本逻辑关系

逻辑	含义	符号
或	满足任意一个条件	<code>,</code>
与	必须同时满足所有条件	<code>&&</code>
非	取反条件	<code>!</code>

4.3.2.1.2 常见写法对比

场景	推荐写法	不推荐写法
多条件判断	<code>[[A && B]]</code>	<code>[[A -a B]]</code>
或判断	<code>[[A \ B]]</code>	<code>[A -o B]</code>
非判断	<code>[[! A]]</code>	<code>! [[A]]</code>
💡 原因：		

- `-a`、`-o` 易产生歧义
- `[[ ]]` 更安全，支持复杂表达式

4.3.2.1.3 真值表

A	B	A B	A && B
真	真	真	真
真	假	真	假
假	真	真	假
假	假	假	假

4.3.2.2 或与非

或 (OR) `||`

```
[[ 条件1 || 条件2 ]]
```

与 (AND) `&&`

```
[[ 条件1 && 条件2 ]]
```

非 (NOT) `!`

```
[[ ! 条件 ]]
```

运算优先级

1. `!` (非)
2. `&&` (与)
3. `||` (或)

必要时使用 `()` 增强可读性:

```
[[ ( A || B ) && C ]]
```

4.3.2.3 或实践

 场景：判断用户输入是否合法

```
1 21:20:24 root@redis01:~/shell# cat test01.sh
2 #!/bin/bash
3 #
4 =====
5 # Script basic information
6 # filename: test01.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 =====
```

```

11
12 read -p "请输入 yes 或 no: " input
13
14 if [[ "$input" == "yes" || "$input" == "no" ]]; then
15     echo "输入合法"
16 else
17     echo "非法输入"
18 fi
19
20
21 21:20:25 root@redis01:~/shell# bash test01.sh
22 请输入 yes 或 no: yes
23 输入合法
24 21:20:28 root@redis01:~/shell# bash test01.sh
25 请输入 yes 或 no: no
26 输入合法
27 21:20:31 root@redis01:~/shell# bash test01.sh
28 请输入 yes 或 no: asdasdsadsad
29 非法输入
30 21:20:34 root@redis01:~/shell#
31

```

 场景：文件不存在或不可读

```

1 21:22:03 root@redis01:~/shell# cat test02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: test02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 file="/etc/passwd"
16
17 if [[ ! -x "${file}" || ! -e "$file" || ! -r "$file" ]]; then
18     echo "文件不存在或不可读或不可执行"
19 fi
20
21 21:22:04 root@redis01:~/shell# bash test02.sh
22 文件不存在或不可读或不可执行
23 21:22:06 root@redis01:~/shell#

```

OR 实践总结

条件	逻辑
A 或 B	<code>A b</code>
非 A 或 非 B	<code>! A B</code>

4.3.2.4 与实践

🎯 场景：文件存在且可执行

```
1 21:23:09 root@redis01:~/shell# cat test03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: test03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 file="./run.sh"
16
17 if [[ -f "$file" && -x "$file" ]]; then
18     echo "可执行脚本"
19 fi
20
21 21:23:12 root@redis01:~/shell# touch run.sh
22 21:23:18 root@redis01:~/shell# chmod +x run.sh
23 21:23:22 root@redis01:~/shell# bash test03.sh
24 可执行脚本
25 21:23:26 root@redis01:~/shell#
```

🎯 场景：成绩范围判断

```
1 21:23:53 root@redis01:~/shell# cat test04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: test04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
```



```

9  # GitHub: xuruizhao00-sys
10 #
=====
====
11 score=85
12
13 if (( score >= 60 && score <= 100 )); then
14     echo "成绩合格"
15 fi
16
17 21:23:55 root@redis01:~/shell# bash test04.sh
18 成绩合格
19 21:24:00 root@redis01:~/shell#

```

📁 AND 实践总结

场景	示例
多条件同时满足	<code>[[A && B]]</code>
数字区间	<code>((x >= a && x <= b))</code>

4.3.2.5 非实践

🎯 场景：变量为空判断

```

1  21:24:43 root@redis01:~/shell# cat test05.sh
2  #!/bin/bash
3  #
=====
====
4  # Script basic information
5  # filename: test05.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
=====
====
11 var=""
12
13 if [[ ! -n "$var" ]]; then
14     echo "变量为空"
15 fi
16
17 21:24:46 root@redis01:~/shell# bash test05.sh
18 变量为空
19 21:24:53 root@redis01:~/shell#

```

🔗 场景：文件不是目录

```
1 21:25:19 root@redis01:~/shell# cat test06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: test06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 path="/etc/passwd"
16
17 if [[ ! -d "$path" ]]; then
18     echo "不是目录"
19 fi
20
21 21:25:21 root@redis01:~/shell# bash test06.sh
22 不是目录
23 21:25:23 root@redis01:~/shell#
```

🔗 场景：非法用户输入

```
1 21:25:23 root@redis01:~/shell# vim test07.sh
2 21:25:52 root@redis01:~/shell# cat test07.sh
3 #!/bin/bash
4 #
5 =====
6 ====
7 # Script basic information
8 # filename: test07.sh
9 # name: xuruizhao
10 # email: xuruizhao00@163.com
11 # v: LnxGuru
12 # GitHub: xuruizhao00-sys
13 #
14 =====
15 ====
16 read -p "请输入数字: " num
17
18 if [[ ! $num =~ ^[0-9]+$ ]]; then
19     echo "非法输入（非数字）"
20 fi
21
```

```

18 21:25:55 root@redis01:~/shell# bash test07.sh
19 请输入数字：99
20 21:25:59 root@redis01:~/shell# bash test07.sh
21 请输入数字：asdsadsa
22 非法输入（非数字）
23 21:26:01 root@redis01:~/shell#

```

NOT 实践总结

逻辑	示例
非存在	<code>[[! -e file]]</code>
非目录	<code>[[! -d path]]</code>
非匹配	<code>[[! str =~ regex]]</code>

4.3.2.6 综合案例

```

1 21:27:39 root@redis01:~/shell# cat test08.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # Script basic information
7 # filename: test08.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 file="$1"
16
17 if [[ -z "$file" ]]; then
18     echo "未指定文件"
19     exit 1
20 fi
21
22 if [[ ! -f "$file" || ! -r "$file" ]]; then
23     echo "文件不存在或不可读"
24     exit 2
25 fi
26
27 if [[ -f "$file" && -x "$file" ]]; then
28     echo "文件可直接执行"
29 else
30     echo "文件不可执行"
31 fi
32

```

```

29 21:27:47 root@redis01:~/shell# bash test08.sh
30 未指定文件
31 21:27:50 root@redis01:~/shell# bash test08.sh run.sh
32 文件可直接执行
33 21:27:53 root@redis01:~/shell# bash test08.sh run.shsss
34 文件不存在或不可读
35 21:27:59 root@redis01:~/shell#

```

4.3.2.6 最佳实践总结表

需求	推荐写法
或	<code>[[A \ \ B]]</code>
与	<code>[[A && B]]</code>
非	<code>[[! A]]</code>
组合	<code>[[(A \ \ B) && C]]</code>
Bash 判断	<code>[[]]</code>

五、数组实践

5.1 数组基础

5.1.1 基础知识

数组（Array）是有序的元素序列，它是数据结构在 shell 当中非常常见的一种数据存储方式，它将有限个类型相同的数据放到一个集合中，这个集合就是数组。

为了操作方便，我们为数组定制一个名称变量，数组内的每一个数据都是数组元素，这些数组元素在集合中有顺序的观念，顺序的位置值我们称为下标。

数组（Array）是 **有序的元素序列**，是 Shell 中非常常见的一种数据存储方式。

它将 **多个类型相同的数据** 放入一个集合中，通过 **下标（index）** 来访问每一个元素。

在 Shell 中：

- 数组名是一个变量
- 数组元素通过 **下标** 标识
- 下标默认从 **0** 开始
- Bash 数组是 **动态数组**（长度可变）

5.1.2 数组分类

在 Bash 中，数组主要分为 **普通索引数组** 和 **关联数组** 两类。

5.1.2.1 索引数组（Index Array）

✦ **特点：**

- 使用 **数字下标**

- Bash 默认数组类型
- 最常用

```
1 11:42:37 root@redis02:~# cat test.sh
2 #
   =====
   ====
3 # 个人信息
4 # name: xuruizhao
5 # email: xuruizhao00@163.com
6 # v: LnxGuru
7 # GitHub: xuruizhao00-sys
8 #
   =====
   ====
9 #!/bin/bash
10 fruits=(apple banana orange)
11 echo "${fruits[0]}"
12 11:42:39 root@redis02:~# bash test.sh
13 apple
14 11:42:45 root@redis02:~#
```

5.1.2.2 关联数组 (Associative Array)

✦ 特点:

- 使用 **字符串**作为下标
- 类似“键值对”
- 需要 Bash 4.0 及以上

```
1 11:44:57 root@redis02:~# cat test02.sh
2 #!/bin/bash
3 #
   =====
   ====
4 # 脚本基础信息
5 # filename: test02.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 # 定义关联数组（必须先声明）
12 declare -A user
13 user[name]="tom"
14 user[age]=12
```

```

15 user[address]="beijing"
16
17 echo "${user[name]}"
18
19 11:44:58 root@redis02:~# bash test02.sh
20 tom
21 11:45:00 root@redis02:~#

```

5.1.2.3 关联数组遍历

```

1 11:49:58 root@redis02:~# cat test02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 定义关联数组（必须先声明）
16 declare -A user
17 user[name]="tom"
18 user[age]=12
19 user[address]="beijing"
20
21 for key in "${!user[@]}";do
22     echo "$key ==> ${user[$key]}"
23 done
24 11:50:00 root@redis02:~# bash test02.sh
25 address ==> beijing
26 age ==> 12
27 name ==> tom
28 11:50:02 root@redis02:~#

```

5.1.2.4 总结

对比项	索引数组	关联数组
下标类型	数字	字符串
是否默认	是	否
声明方式	可省略	必须 <code>declare -A</code>
使用场景	列表、队列	配置、映射关系

5.2 数组定义

在 Bash 中，数组属于 **变量的一种扩展形式**。

数组的定义方式灵活，不同定义方式适合不同使用场景。

5.2.1 直接赋值定义

```
1 11:54:54 root@redis02:~# cat test03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 arr=(a b c d)
16 echo "${arr[0]}"
17
18 11:54:56 root@redis02:~# bash test03.sh
19 a
20 11:54:57 root@redis02:~#
```

✦ 特点：

- 最简洁、最常用
- 自动从下标 0 开始

5.2.2 指定下标赋值

✦ 特点：

- 下标可不连续
- 适合动态赋值

```
1 11:56:15 root@redis02:~# cat test03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test03.sh
```

```

6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 arr[0]=apple
12 arr[1]=banana
13 arr[2]=orange
14 arr[10]=grape
15
16 echo "${arr[10]}"
17 11:56:16 root@redis02:~# bash test03.sh
18 grape
19 11:56:18 root@redis02:~#

```

5.2.3 使用命令替换生成数组

```

1  11:58:00 root@redis02:~# cat test03.sh
2  #!/bin/bash
3  #
    =====
    ====
4  # 脚本基础信息
5  # filename: test03.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 files=($(ls))
12 echo "${files[0]}"
13 11:58:01 root@redis02:~# ls
14 aa.sh  init.sh  test02.sh  test03.sh  test.sh
15 11:58:03 root@redis02:~# bash test03.sh
16 aa.sh
17 11:58:07 root@redis02:~#

```

✦ 说明:

- 每个“空格分隔项”作为数组元素
- 对空格敏感

⚠ 风险:

- 文件名包含空格会被拆分

✓ 更安全的写法（推荐）：

```
1 11:58:56 root@redis02:~# cat test03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 mapfile -t files < <(ls)
16 echo "${files[1]}"
17 11:58:57 root@redis02:~# bash test03.sh
18 init.sh
19 11:59:00 root@redis02:~#
```

5.2.4 使用 read 定义数组

```
1 13:12:59 root@redis02:~# cat test04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 read -p "请输入多个值: " -a arr
16 echo "${arr[0]}"
17 13:13:00 root@redis02:~# bash test04.sh
18 请输入多个值: a b c d
19 a
20 13:13:05 root@redis02:~#
```

5.2.5 通过变量扩展定义数组

将已有变量转为数组

```
1 13:14:25 root@redis02:~# cat test04.sh
2 #!/bin/bash
3 #
4 =====
5 # 脚本基础信息
6 # filename: test04.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 str="a b c"
14 arr=($str)
15 echo "${arr[0]}"
16 13:14:26 root@redis02:~# bash test04.sh
17 a
18 13:14:29 root@redis02:~#
```

5.2.6 使用 declare 明确定义数组

✦ 作用:

- 明确变量类型
- 提高脚本可读性
- -a 定义索引数组
- -A 定义关联数组

```
1 declare -a arr
2 arr=(x y z)
```

5.2.7 定义关联数组 (扩展)

```
1 declare -A user
2 user[name]="tom"
3 user[age]=18
```

✦ 特点:

- 下标是字符串

- Bash 4.0+ 支持

5.2.8 总结

定义方式	示例	适用场景
直接赋值	<code>arr=(a b c)</code>	固定数据
下标赋值	<code>arr[0]=a</code>	动态赋值
命令替换	<code>arr=\$(cmd)</code>	命令结果
read 输入	<code>read -a arr</code>	用户输入
declare	<code>declare -a arr</code>	明确类型
关联数组	<code>declare -A arr</code>	键值对

5.2.9 最佳实践

- ✓ 推荐使用 `arr=(...)`
- ✓ 用户输入使用 `read -a`
- ✓ 文件列表使用 `mapfile -t`
- ✓ 关联数组必须 `declare -A`
- ✓ 含空格内容一定加双引号

5.3 索引数组遍历取值

在 Bash 中，数组本身不难，难点在“访问”和“遍历”。

90% 的数组 Bug 都出现在：

- 👉 没加引号
- 👉 用错 `@` 和 `*`
- 👉 遍历方式不当

5.3.1 数组的访问方式

5.3.1.1 访问单个元素

```
1 arr=(apple banana orange)
2
3 echo "${arr[0]}"
4 echo "${arr[1]}"
```

📌 规则：

- 下标从 `0` 开始
- 访问时必须使用 `${}`

5.3.1.2 访问所有元素

写法	含义
<code>\${arr[@]}</code>	所有元素（推荐）
<code>\${arr[*]}</code>	所有元素
<code>_bash</code>	
<code>echo "\${arr[@]}"</code>	
<code>~</code>	
▲ 区别非常重要（后面重点讲）	

5.3.1.3 获取数组长度

```
1 | echo "${#arr[@]}"
```

表达式	含义
<code>\${#arr[@]}</code>	数组元素个数
<code>\${#arr[0]}</code>	单个元素字符串长度

5.3.1.4 获取所有下标

```
1 | echo "${!arr[@]}"
2
3 # 示例输出
4 0 1 2
```

✦ 常用于：按下标遍历数组

5.3.2 数组遍历方式

5.3.2.1 直接遍历元素（★ 强烈推荐）

```
1 | 13:26:51 root@redis02:~# cat test04.sh
2 | #!/bin/bash
3 | #
4 | # =====
5 | # 脚本基础信息
6 | # filename: test04.sh
7 | # name: xuruizhao
8 | # email: xuruizhao00@163.com
9 | # v: LnxGuru
10 | # GitHub: xuruizhao00-sys
```

```

10  #
    =====
    ====
11  str="a b c"
12  arr=($str)
13  for item in "${arr[@]%%}"; do
14      echo "$item"
15  done
16  13:26:53 root@redis02:~# bash test04.sh
17  a
18  b
19  c
20  13:26:55 root@redis02:~#

```

🔴 特点:

- 最安全
- 支持元素中包含空格
- 实战首选

5.3.2.2 按下标遍历 (进阶必会)

```

1  13:28:32 root@redis02:~# cat test04.sh
2  #!/bin/bash
3  #
    =====
    ====
4  # 脚本基础信息
5  # filename: test04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 str="a b c"
12 arr=($str)
13 for key in "${!arr[@]%%}"; do
14     echo "$key ---> ${arr[$key]}"
15 done
16 13:28:34 root@redis02:~# bash test04.sh
17 0 ---> a
18 1 ---> b
19 2 ---> c
20 13:28:35 root@redis02:~#

```

🔴 适合场景:

- 需要下标
- 需要修改元素

5.3.2.3 C 风格遍历（不推荐新手）

```

1  13:30:35 root@redis02:~# cat test04.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: test04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 str="a b c"
12 arr=($str)
13 for (( i=0;i<${#arr[@]};i++ ));do
14     echo "$i ---> ${arr[i]}"
15 done
16 13:30:38 root@redis02:~# bash test04.sh
17 0 ---> a
18 1 ---> b
19 2 ---> c
20 13:30:40 root@redis02:~#

```

5.3.3 \${@} vs \${*}（重点 + 易错）

5.3.3.1 不加引号时

👉 效果相同（都会被拆分）

```

1 echo ${arr[@]}
2 echo ${arr[*]}

```

5.3.3.2 在加双引号时（重点）

```

1 "${arr[@]}" # 每个元素独立
2 "${arr[*]}" # 所有元素合并成一个字符串

```

写法	遍历效果
"\${arr[@]}"	★ 每个元素一个

写法	遍历效果
<code>"\${arr[*]}"</code>	× 所有元素当成一个

5.3.3.3 示例

```

1 13:40:08 root@redis02:~# cat test05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 arr=("hello world" "shell script")
16 for i in "${arr[@]}"; do
17     echo "$i"
18 done
19 for i in "${arr[*]}"; do
20     echo "$i"
21 done
22 13:40:11 root@redis02:~# bash test05.sh
23 hello world
24 shell script
25 hello world shell script
26 13:40:14 root@redis02:~#

```

5.3.4 数组访问与遍历的常见错误（重点）

5.3.4.1 忘记双引号

```

1 for i in ${arr[@]}; do
2     echo "$i"
3 done

```

! 问题:

- 遇到空格直接拆分
- 非法数据

✓ 正确:

```
1 | for i in "${arr[@]}; do
```

5.3.4.2 直接 echo 数组名

```
1 | echo $arr
```

! 结果:

- 只输出第一个元素

✓ 正确:

```
1 | echo "${arr[@]}"
```

5.3.4.3 误用 \${arr[*]}

```
1 | for i in "${arr[*]}; do
```

! 结果:

- 整个数组只循环一次

✓ 正确:

```
1 | for i in "${arr[@]}; do
```

5.3.5 数组访问与遍历速查表

需求	推荐写法
单个元素	<code>\${arr[0]}</code>
所有元素	<code>"\${arr[@]}"</code>
数组长度	<code>\${#arr[@]}</code>
所有下标	<code>\${!arr[@]}</code>
安全遍历	<code>for i in "\${arr[@]}"</code>
带下标遍历	<code>for i in "\${!arr[@]}"</code>

5.4 关联数组访问与遍历

5.4.1 定义与取值

```
1 # 关联数组定义
2 declare -A user
3
4 # 关联数组赋值
5 user[name]="tom"
6 user[age]=18
7 user[city]="beijing"
8
9 # 访问某一个值
10 echo "${user[age]}"
```

5.4.2 遍历关联数组

5.4.2.1 遍历 key

```
1 17:33:43 root@redis02:~# cat demo02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 declare -A user
16 user[name]="zhangsan"
17 user[age]=12
18 user[city]="beijing"
19
20 for key in "${!user[@]}";
21 do
22     echo "$key"
23 done
24 17:33:44 root@redis02:~# bash demo02.sh
25 city
26 age
27 name
28 17:33:46 root@redis02:~#
```

5.4.2.2 遍历 value

```
1 17:34:25 root@redis02:~# cat demo02.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: demo02.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 # =====
13 #
14 declare -A user
15 user[name]="zhangsan"
16 user[age]=12
17 user[city]="beijing"
18
19 for value in "${user[@]}";
20 do
21     echo "$value"
22 done
23
24 17:34:26 root@redis02:~# bash demo02.sh
25 beijing
26 12
27 zhangsan
28 17:34:28 root@redis02:~#
```

5.4.2.3 遍历 key + value

```
1 17:35:10 root@redis02:~# cat demo02.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: demo02.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 # =====
13 #
14 declare -A user
15 user[name]="zhangsan"
```

```

13 user[age]=12
14 user[city]="beijing"
15 for key in "${!user[@]}"; do
16     echo "$key => ${user[$key]}"
17 done
18 17:35:11 root@redis02:~# bash demo02.sh
19 city => beijing
20 age => 12
21 name => zhangsan
22 17:35:13 root@redis02:~#

```

5.4.3 总结

操作	写法
所有 key	<code>\${!arr[@]}</code>
所有 value	<code>\${arr[@]}</code>
单个值	<code>\${arr[key]}</code>
删除 key	<code>unset arr[key]</code>

5.5 数组元素操作

5.5.1 数组的修改操作

5.5.1.1 修改指定下标的元素

```

1 17:35:13 root@redis02:~# arr=(a b c d)
2 17:43:05 root@redis02:~# echo "${arr[@]}"
3 a b c d
4 17:43:16 root@redis02:~# arr[0]=aaaa
5 17:43:22 root@redis02:~# echo "${arr[@]}"
6 aaaa b c d
7 17:43:23 root@redis02:~#
8 17:43:23 root@redis02:~# arr[4]=e
9 17:43:47 root@redis02:~# echo "${arr[@]}"
10 aaaa b c d e
11 17:43:49 root@redis02:~# arr[10]=aaa
12 17:43:54 root@redis02:~# echo "${arr[@]}"
13 aaaa b c d e aaa
14 17:43:56 root@redis02:~#

```

🚩 特点:

- 下标存在 → 覆盖
- 下标不存在 → 自动新增

5.5.1.2 追加元素

```
1 17:43:56 root@redis02:~# echo "${arr[@]}"
2 aaaa b c d e aaa
3 17:50:46 root@redis02:~# arr+=(g h)
4 17:50:54 root@redis02:~# echo "${arr[@]}"
5 aaaa b c d e aaa g h
6 17:50:55 root@redis02:~#
```

5.5.1.3 批量修改

```
1 17:52:22 root@redis02:~# cat demo02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 arr=(a b c d)
16 for i in "${!arr[@]}"; do
17     arr[$i]="${arr[$i]}_new"
18 done
19 echo "${arr[@]}"
20 17:52:23 root@redis02:~# bash demo02.sh
21 a_new b_new c_new d_new
22 17:52:25 root@redis02:~#
```

5.5.2 数据的删除操作

5.5.2.1 删除指定元素

```
1 17:53:19 root@redis02:~# echo "${arr[@]}"
2 aaaa b c d e aaa g h
3 17:53:23 root@redis02:~# unset arr[1]
4 17:53:26 root@redis02:~# echo "${arr[@]}"
5 aaaa c d e aaa g h
6 17:53:27 root@redis02:~#
```

⚠ 注意:

- 不会自动重排下标

```

1 17:53:27 root@redis02:~# arr=(x y z)
2 17:53:59 root@redis02:~# unset arr[1]
3 17:54:13 root@redis02:~# echo "${!arr[@]}"
4 0 2
5 17:54:23 root@redis02:~# echo "${arr[1]}"
6
7 17:54:35 root@redis02:~# echo "${arr[0]}"
8 x
9 17:54:37 root@redis02:~# echo "${arr[2]}"
10 z
11 17:54:39 root@redis02:~#

```

5.5.2.2 删除整个数组

```

1 unset arr

```

5.5.2.3 删除后重新整理下标（技巧）

```

1 17:55:12 root@redis02:~# arr=(x y z)
2 17:55:50 root@redis02:~# unset arr[1]
3 17:55:56 root@redis02:~# echo "${!arr[@]}"
4 0 2
5 17:56:10 root@redis02:~# arr=("${arr[@]}")
6 17:56:25 root@redis02:~# echo "${!arr[@]}"
7 0 1
8 17:56:28 root@redis02:~#

```

5.5.3 数组切片

5.5.3.1 基本切片语法

```

1 ${arr[@]:起始位置:长度}

```

5.5.3.2 示例

```

1 17:56:28 root@redis02:~# arr=(a b c d e f)
2 17:57:31 root@redis02:~# echo "${arr[@]:1:3}"
3 b c d
4 17:57:43 root@redis02:~# echo "${arr[@]:2}"
5 c d e f
6 17:58:11 root@redis02:~#

```

场景	示例
跳过前 N 个	<code>\${arr[@]:N}</code>

场景	示例
取子集	<code>\${arr[@]:start:len}</code>

5.6 脚本案例

5.6.1 批量主机处理

```
1 18:03:43 root@redis02:~# cat host_list.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: host_list.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 hosts=(web1 web2 db1 cache1)
16
17 for host in "${hosts[@]}"; do
18     echo "正在处理 $host"
19 done
```

5.6.2 批量检查文件是否存在

```
1 18:04:15 root@redis02:~# cat file_list.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: file_list.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 files=(/etc/passwd /etc/shadow /etc/hosts)
16
17 for f in "${files[@]}"; do
18     if [[ -e "$f" ]]; then
19         echo "$f 存在"
20     fi
21 done
```

```

16     else
17         echo "$f 不存在"
18     fi
19 done
20
21 18:04:16 root@redis02:~# bash file_list.sh
22 /etc/passwd 存在
23 /etc/shadow 存在
24 /etc/hosts 存在
25 18:04:20 root@redis02:~#

```

5.6.3 使用关联数组管理配置

```

1 18:05:04 root@redis02:~# cat config.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: config.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 declare -A service_port=(
16     [nginx]=80
17     [mysql]=3306
18     [redis]=6379
19 )
20
21 for service in "${!service_port[@]}"; do
22     echo "$service 使用端口 ${service_port[$service]}"
23 done
24
25 18:05:05 root@redis02:~# bash config.sh
26 nginx 使用端口 80
27 redis 使用端口 6379
28 mysql 使用端口 3306
29 18:05:06 root@redis02:~#

```

5.6.4 参数转数组（脚本开发常用）

```

1 18:06:52 root@redis02:~# cat demo03.sh
2 #!/bin/bash

```

```

3  #
  =====
  ====
4  # 脚本基础信息
5  # filename: demo03.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11
12 args=("$@")
13
14 for i in "${!args[@]}"; do
15     echo "参数$i = ${args[$i]}"
16 done
17 18:06:55 root@redis02:~# bash demo03.sh 1 2 3 4 5
18 参数0 = 1
19 参数1 = 2
20 参数2 = 3
21 参数3 = 4
22 参数4 = 5
23 18:07:03 root@redis02:~#

```

5.6.5 服务管理

```

1 18:12:47 root@redis02:~# cat service_manager.sh
2 #!/bin/bash
3 #
  =====
  ====
4 # 脚本基础信息
5 # filename: service_manager.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11
12 # 定义关联数组，存储服务动作与命令的映射
13 declare -A service_actions=(
14     [start]="start"
15     [stop]="stop"
16     [restart]="restart"
17     [reload]="reload"

```



```

18     [status]="status"
19 )
20
21 # 定义索引数组，存储所有的服务动作
22 actions=("start" "stop" "restart" "reload" "status")
23
24 # 提示用户选择服务动作
25 echo "请选择服务管理动作： "
26 select action in "${actions[@]}"; do
27     # 如果选择了一个有效的动作
28     if [[ -n "$action" ]]; then
29         # 提示用户输入服务名
30         read -p "请输入服务名称： " service_name
31         # 输出对应的 systemctl 命令
32         echo "systemctl ${service_actions[$action]} $service_name"
33         break
34     else
35         echo "无效选择，请重新选择！ "
36     fi
37 done
38
39
40 18:12:49 root@redis02:~# bash service_manager.sh
41 请选择服务管理动作：
42 1) start
43 2) stop
44 3) restart
45 4) reload
46 5) status
47 #? 1
48 请输入服务名称： dsdasd
49 systemctl start dsdasd

```

5.6.6 信息收集

```

1 18:18:35 root@redis02:~# cat info.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: info.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====

```

```

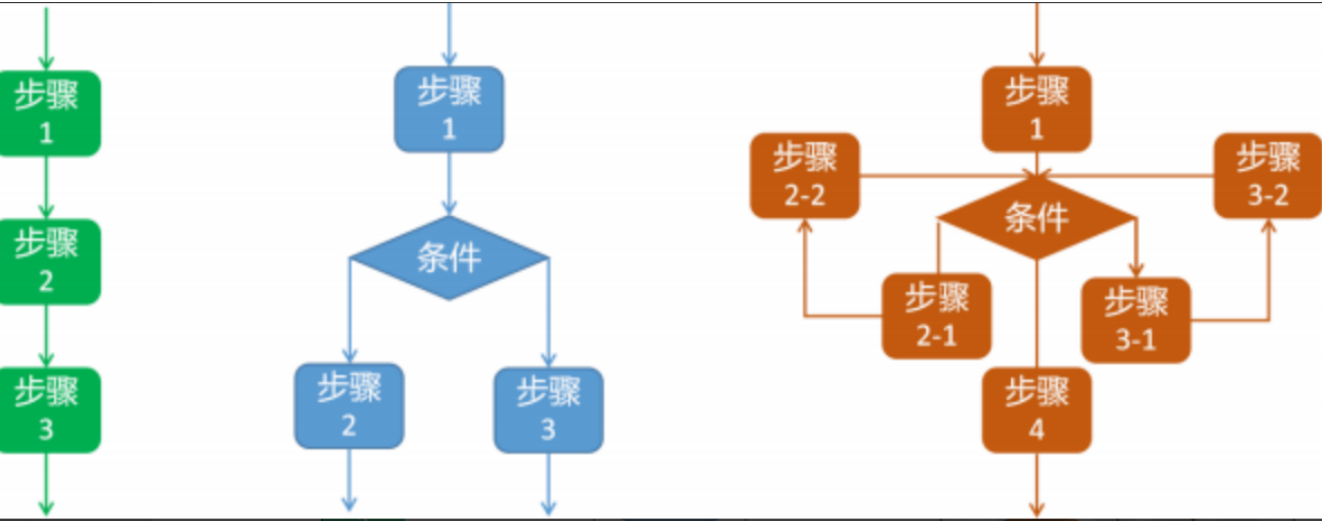
11
12 # =====
13 # CPU Load 监控脚本
14 # 功能：获取 CPU Load 1/5/15分钟的负载，并根据阈值进行告警。
15 # 使用关联数组和索引数组来实现负载管理。
16 # =====
17
18 # 1. 定义阈值（可以根据实际情况调整）
19 HIGH_LOAD_THRESHOLD=1.00 # 当负载大于此值时发送告警
20 ALERT_EMAIL="admin@example.com" # 假设需要发送邮件给管理员
21
22 # 2. 定义索引数组：表示不同时间的负载
23 periods=("1min" "5min" "15min")
24
25 # 3. 定义关联数组：映射周期到数组中的下标
26 declare -A index_map=(
27     [1min]=0
28     [5min]=1
29     [15min]=2
30 )
31
32 # 4. 获取系统的 CPU load (1/5/15分钟)
33 loads=( $(uptime | awk -F'load average: ' '{print $2}' | tr ',' ' ') )
34
35 # 5. 输出当前 CPU Load 和检查是否超过阈值
36 echo "当前系统负载: "
37 for p in "${periods[@]"; do
38     idx=${index_map[$p]}
39     load="${loads[$idx]}"
40
41     # 打印 CPU 负载
42     printf "CPU %-5s 平均负载为: %.2f\n" "$p" "$load"
43
44     # 检查是否超过负载阈值
45     if (( $(echo "$load > $HIGH_LOAD_THRESHOLD" | bc -l) )); then
46         echo "警告: $p 负载超出阈值! 当前负载为 $load"
47         # 如果负载超过阈值，发送告警邮件（此处使用 mail 命令，确保系统已配置邮件服
务）
48         echo "CPU $p 负载已达到 $load, 请注意检查系统状态。" | mail -s "CPU
Load Alert: $p" "$ALERT_EMAIL"
49     fi
50 done
51
52 # 6. 返回 0 表示脚本成功执行
53 exit 0
54
55 18:18:36 root@redis02:~# bash info.sh
56 当前系统负载:
57 CPU 1min 平均负载为: 0.03
58 CPU 5min 平均负载为: 0.02

```

5.7 数组操作速查总表

需求	推荐写法
定义数组	<code>arr=(a b c)</code>
访问元素	<code>\${arr[0]}</code>
所有元素	<code>"\${arr[@]}"</code>
所有下标	<code>"\${!arr[@]}"</code>
数组长度	<code>\${#arr[@]}</code>
追加	<code>arr+=(x y)</code>
删除元素	<code>unset arr[i]</code>
切片	<code>\${arr[@]:1:2}</code>
关联数组	<code>declare -A arr</code>

六、流程控制



- 1 编程语言的目的是通过风格化的编程思路将代码写出来后，实现项目功能的。为了实现功能，我们
- 2 通过在代码层面通过一些代码逻辑来实现：
- 3 顺序执行 - 程序按从上到下顺序执行
- 4 选择执行 - 程序执行过程中，根据条件选择不同的顺序执行
- 5 循环执行 - 程序执行过程中，根据条件重复执行代码
- 6 条件逻辑 - 多分支执行命令块
- 7 - `if` 控制语句
- 8 - `case` 控制语句
- 9 - `select` 控制语句
- 10 循环逻辑 - 多循环执行命令块

```
11 - for 控制语句
12 - while 控制语句
13 - until 控制语句
14
15 逻辑控制 - 命令块执行过程中，精细化控制
16 - continue 控制
17 - break 控制
18 - exit 控制
19 - shift 控制
```

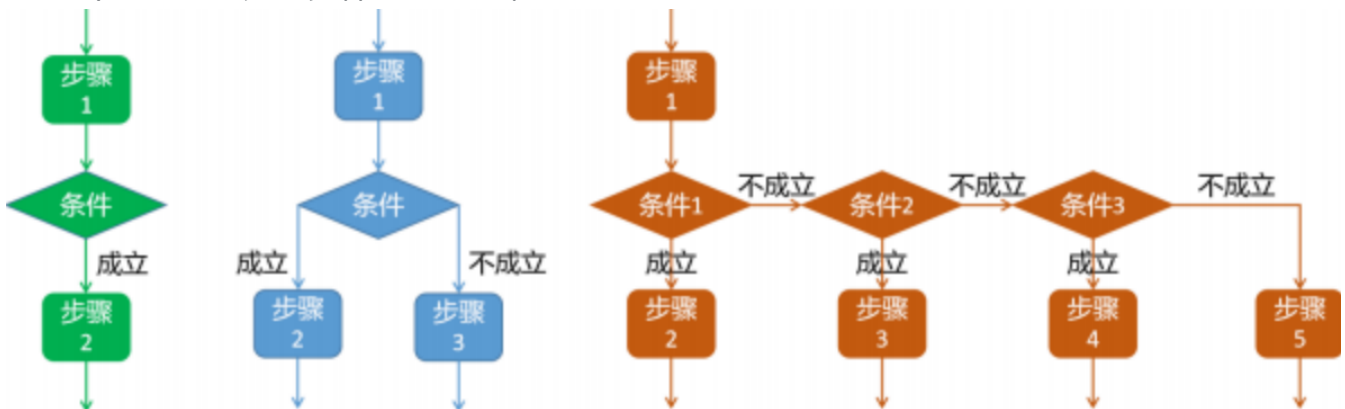
6.1 分支逻辑

6.1.1 if 分支逻辑

6.1.1.1 if 基础知识

条件结构能够根据某个特定的条件，结合内置的测试表达式功能，结合测试的结果状态值对于条件进行判断，然后选择执行合适的任务。在 bash 中，if 命令是条件结构最简单的形式。

shell 中的 if 语句支持多种条件的决策形式：



6.1.1.2 if 语法格式

6.1.1.2.1 单路模式

```
1 if [ condition ]; then
2     # 如果条件为真，执行此部分代码
3 fi
```

6.1.1.2.2 双路模式

```
1 if [ condition ]; then
2     # 如果条件为真，执行此部分代码
3 else
4     # 如果条件为假，执行此部分代码
5 fi
```

6.1.1.2.3 多路模式

```
1 if [ condition1 ]; then
2     # 如果条件1为真，执行此部分代码
3 elif [ condition2 ]; then
4     # 如果条件2为真，执行此部分代码
5 else
6     # 如果所有条件都为假，执行此部分代码
7 fi
```

6.1.1.3 条件判断格式

6.1.1.3.1 结合多个条件判断（&& 和 ||）

```
1 if [ condition1 ] && [ condition2 ]; then
2     # 如果条件1和条件2都为真，执行此部分代码
3 fi
4
5 if [ condition1 ] || [ condition2 ]; then
6     # 如果条件1或条件2为真，执行此部分代码
7 fi
```

6.1.1.3.2 if 与 [[...]] 条件表达式

使用 [[...]] 进行更强大的条件判断。

```
1 if [[ condition ]]; then
2     # 如果条件为真，执行此部分代码
3 fi
4 # 你可以将这些结构复制并在 Shell 脚本中应用，帮助你灵活地使用 `if` 语句进行分支控制。
```

6.1.1.3.3 ((cmd))

代替 let 命令来测试数值表达式

6.1.1.4 单路分支实践

```
1 13:12:52 root@redis02:~# cat test01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
```

```

10  #
    =====
    ====
11  # 获取用户输入的数字
12  echo "请输入一个数字:"
13  read num
14
15  # 判断数字是否大于10
16  if [ $num -gt 10 ]; then
17      echo "数字大于 10"
18  fi
19
20  13:12:53 root@redis02:~# bash test01.sh
21  请输入一个数字:
22  123
23  数字大于 10
24  13:12:59 root@redis02:~#

```

解释:

- `if` 语句判断 `$num` 是否大于 10。
- 如果条件为真，输出 "数字大于 10"；否则，不执行任何操作。

6.1.1.5 双路分支实践

```

1  13:13:41 root@redis02:~# cat test02.sh
2  #!/bin/bash
3  #
    =====
    ====
4  # 脚本基础信息
5  # filename: test02.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11
12 # 获取用户输入的数字
13 echo "请输入一个数字:"
14 read num
15
16 # 判断数字是否大于10
17 if [ $num -gt 10 ]; then
18     echo "数字大于 10"
19 else
20     echo "数字小于或等于 10"

```

```

21 fi
22
23 13:13:43 root@redis02:~# bash test02.sh
24 请输入一个数字:
25 4342432432
26 数字大于 10
27 13:13:48 root@redis02:~# bash test02.sh
28 请输入一个数字:
29 10
30 数字小于或等于 10
31 13:13:53 root@redis02:~#

```

解释:

- `if` 语句判断 `$num` 是否大于 10。
- 如果条件为真, 输出 "数字大于 10"。
- 如果条件为假, 输出 "数字小于或等于 10"。

6.1.1.6 多路分支实践

```

1 13:14:26 root@redis02:~# cat test03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16 # 获取用户输入的数字
17 echo "请输入一个数字:"
18 read num
19
20 # 多路分支判断数字的范围
21 if [ $num -gt 10 ]; then
22     echo "数字大于 10"
23 elif [ $num -eq 10 ]; then
24     echo "数字等于 10"
25 else
26     echo "数字小于 10"
27 fi
28
29 13:14:27 root@redis02:~# bash test03.sh
30 请输入一个数字:

```

```
26 3
27 数字小于 10
28 13:14:30 root@redis02:~#
```

解释:

- 使用 `if-elif-else` 语句来处理多个判断条件。
- 根据输入的数字判断其是大于、等于还是小于 10。

6.1.1.7 if 总结实践

需求:

我们有一组数字, 要求:

1. 判断每个数字是奇数还是偶数。
2. 判断数字是否大于 50。
3. 输出符合条件的数字。

```
1 13:18:00 root@redis02:~# cat test04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 定义一个数字数组
16 numbers=(12 25 48 53 76 33 17 8 91 60)
17
18 # 遍历数字数组
19 for num in "${numbers[@]}; do
20     # 判断是否大于 50
21     if [ $num -gt 50 ]; then
22         echo "$num 是大于 50 的数字"
23
24     # 判断是否是偶数
25     elif [ $((num % 2)) -eq 0 ]; then
26         echo "$num 是偶数"
27
28     # 判断是否是奇数
29     else
30         echo "$num 是奇数"
```



```
28     fi
29 done
30
31
32 13:18:01 root@redis02:~# bash test04.sh
33 12 是偶数
34 25 是奇数
35 48 是偶数
36 53 是大于 50 的数字
37 76 是大于 50 的数字
38 33 是奇数
39 17 是奇数
40 8 是偶数
41 91 是大于 50 的数字
42 60 是大于 50 的数字
43 13:18:02 root@redis02:~#
```

分支类型	语法结构	示例代码
单路分支	<code>if [condition]; then ... fi</code>	<code>if [\$num -gt 10]; then ... fi</code>
双路分支	<code>if [condition]; then ... else ... fi</code>	<code>if [\$num -gt 10]; then ... else ... fi</code>
多路分支	<code>if [condition]; then ... elif ... else ... fi</code>	<code>if [\$num -gt 10]; then ... elif ... fi</code>
总结实践	综合应用多个 <code>if</code> 条件判断	<code>if [\$age -ge 18]; then ... fi</code>

6.1.2 case

`case` 命令是一个多路分支的命令，它可以来代替 `if/elif` 相关的命令，在 `case` 语句中，它通过引入一个变量接收用户输入的数据，然后依次与相关的值进行匹配判断，一旦找到对应的匹配值后，就执行相关的语句。

6.1.2.1 case 语法

```

1 case "$variable" in
2     pattern1)
3         # 如果变量匹配 pattern1, 执行此部分代码
4         ;;
5     pattern2)
6         # 如果变量匹配 pattern2, 执行此部分代码
7         ;;
8     *)
9         # 如果没有匹配到任何模式, 执行此部分代码
10        ;;
11 esac

```

- `case` 语句用于根据变量的值进行模式匹配。
- `"$variable"` 是要检查的变量。
- `pattern1`, `pattern2`, 等是待匹配的模式, 可以是字符串、正则表达式等。
- `;;` 用来标记每个模式的结束。
- `*` 是通配符, 表示所有其他未匹配的情况

6.1.2.2 示例

```

1 13:23:31 root@redis02:~# cat test05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 服务名
16 SERVICE_NAME="myservice"
17
18 # 打印帮助信息
19 print_help() {
20     echo "Usage: $0 {start|stop|restart|status}"
21     echo "start    - 启动服务"
22     echo "stop      - 停止服务"
23     echo "restart   - 重启服务"
24     echo "status    - 查看服务状态"
25 }
26
27 # 检查服务是否正在运行

```

```
25 check_service_status() {
26     systemctl is-active --quiet $SERVICE_NAME
27     return $? # 0表示服务正在运行，非0表示服务没有运行
28 }
29
30 # 启动服务
31 start_service() {
32     echo "启动服务 $SERVICE_NAME ..."
33     systemctl start $SERVICE_NAME
34     if [ $? -eq 0 ]; then
35         echo "服务 $SERVICE_NAME 已启动"
36     else
37         echo "启动服务 $SERVICE_NAME 失败"
38     fi
39 }
40
41 # 停止服务
42 stop_service() {
43     echo "停止服务 $SERVICE_NAME ..."
44     systemctl stop $SERVICE_NAME
45     if [ $? -eq 0 ]; then
46         echo "服务 $SERVICE_NAME 已停止"
47     else
48         echo "停止服务 $SERVICE_NAME 失败"
49     fi
50 }
51
52 # 重启服务
53 restart_service() {
54     echo "重启服务 $SERVICE_NAME ..."
55     systemctl restart $SERVICE_NAME
56     if [ $? -eq 0 ]; then
57         echo "服务 $SERVICE_NAME 已重启"
58     else
59         echo "重启服务 $SERVICE_NAME 失败"
60     fi
61 }
62
63 # 查看服务状态
64 status_service() {
65     if check_service_status; then
66         echo "服务 $SERVICE_NAME 正在运行"
67     else
68         echo "服务 $SERVICE_NAME 没有运行"
69     fi
70 }
71
72 # 解析用户输入的命令
73 case "$1" in
74     start)
```

```

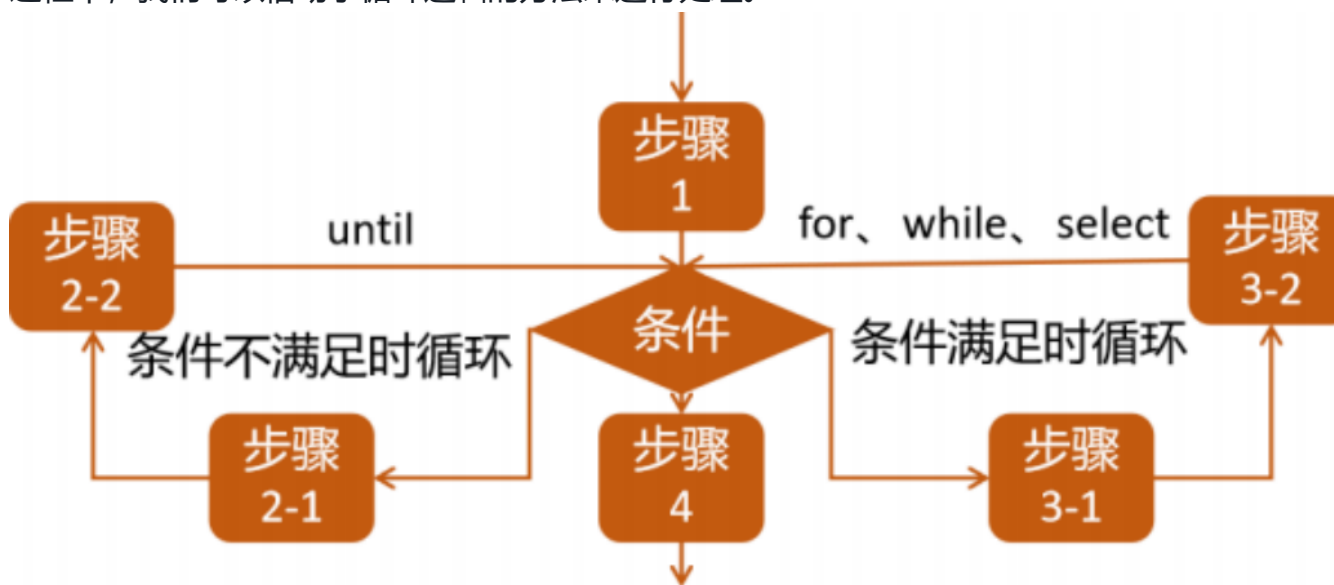
75     start_service
76     ;;
77 stop)
78     stop_service
79     ;;
80 restart)
81     restart_service
82     ;;
83 status)
84     status_service
85     ;;
86 *)
87     print_help
88     ;;
89 esac

```

6.2 循环逻辑

6.2.1 for 循环

生产工作中，我们有可能会遇到一种场景，需要重复性的执行相同的动作，我们在 shell 编程的过程中，我们可以借助于循环逻辑的方法来进行处理。



6.2.1.1 for 循环语法

```

1 for variable in value1 value2 value3 ...; do
2     # 执行的代码
3 done

```

- `variable` 是用来依次获取 `value1`, `value2`, `value3` 等的变量。
- `value1`, `value2`, `value3` 是循环中每次迭代时给 `variable` 赋的值。

```
1 for variable in {start..end}; do
2     # 执行的代码
3 done
```

- `{start..end}` 是一个数字范围，从 `start` 开始，到 `end` 结束（包括 `end`）。
- 这种语法适用于生成数字序列。

6.1.1.2 for 循环中列表生成

6.1.1.2.1 基本数字方式生成

6.1.1.2.1.1 使用花括号 `{start..end}` 创建数字范围

Shell 中使用花括号 `{start..end}` 可以轻松生成一个数字范围。生成的数字会按照升序顺序从 `start` 到 `end` 逐个列出。

```
1 for i in {start..end}; do
2     # 执行的代码
3 done
```

```
1 13:35:02 root@redis02:~# cat test06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 输出从 1 到 5 的数字
16 for num in {1..5}; do
17     echo "数字是: $num"
18 done
19
20 13:35:03 root@redis02:~# bash test06.sh
21 数字是: 1
22 数字是: 2
23 数字是: 3
24 数字是: 4
25 数字是: 5
26
27 13:35:04 root@redis02:~#
```

6.1.1.2.1.2 使用步长 {start..end..step} 创建带步长的数字范围

除了 `start` 和 `end`，你还可以指定一个步长（`step`），用来决定每次循环的数字增量。

```
1 for i in {start..end..step}; do
2     # 执行的代码
3 done
```

```
1 13:35:56 root@redis02:~# cat test07.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test07.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 输出从 1 到 10 的偶数
16 for num in {2..10..2}; do
17     echo "偶数是: $num"
18 done
19
20 13:35:57 root@redis02:~# bash test07.sh
21 偶数是: 2
22 偶数是: 4
23 偶数是: 6
24 偶数是: 8
25 偶数是: 10
26
27 13:35:59 root@redis02:~#
```

6.1.1.2.1.3 使用 seq 生成列表

`seq` 命令可以生成指定范围内的数字列表，它允许更灵活地控制步长，生成的列表可以直接在 `for` 循环中使用。

```
1 for i in $(seq start step end); do
2     # 执行的代码
3 done
```

- `seq start end` 用于生成从 `start` 到 `end` 的数字列表。
- `seq start step end` 用于生成带步长的数字列表。

```
1 13:40:45 root@redis02:~# cat test09.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test09.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 使用 seq 生成从 1 到 5 的数字列表
16 for num in $(seq 1 1 5); do
17     echo "数字是: $num"
18 done
19
20 13:40:46 root@redis02:~# bash test09.sh
21 数字是: 1
22 数字是: 2
23 数字是: 3
24 数字是: 4
25 数字是: 5
26
27 13:40:48 root@redis02:~# vim test09.sh
28 13:40:57 root@redis02:~# cat test09.sh
29 #!/bin/bash
30 #
31 =====
32 ====
33 # 脚本基础信息
34 # filename: test09.sh
35 # name: xuruizhao
36 # email: xuruizhao00@163.com
37 # v: LnxGuru
38 # GitHub: xuruizhao00-sys
39 #
40 =====
41 ====
42 # 使用 seq 生成从 1 到 5 的数字列表
43 for num in $(seq 1 2 5); do
44     echo "数字是: $num"
45 done
46
47 13:40:58 root@redis02:~# bash test09.sh
48 数字是: 1
49 数字是: 3
50 数字是: 5
51
52 13:41:00 root@redis02:~#
```

6.1.1.2.2 生成文件列表

Shell 中的通配符可以用来匹配指定模式的文件，`for` 循环可以用来遍历匹配的文件列表。

```
1 for file in /path/to/directory/*; do
2     # 执行的代码
3 done
```

```
1 13:37:21 root@redis02:~# cat test08.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test08.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 遍历当前目录下的所有文件
16 for file in *; do
17     echo "文件名: $file"
18 done
19
20 13:37:22 root@redis02:~# bash test08.sh
21 文件名: test01.sh
22 文件名: test02.sh
23 文件名: test03.sh
24 文件名: test04.sh
25 文件名: test05.sh
26 文件名: test06.sh
27 文件名: test07.sh
28 文件名: test08.sh
29
30 13:37:24 root@redis02:~#
```

6.1.1.2.3 生成数组列表

Shell 中的数组允许存储多个元素，`for` 循环可以用来遍历数组中的每个元素。

```
1 for item in "${array[@]}; do
2     # 执行的代码
3 done
```

```
1 13:43:00 root@redis02:~# cat test10.sh
```



```

2 #!/bin/bash
3 #
=====
====
4 # 脚本基础信息
5 # filename: test10.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
=====
====
11 # 定义一个数组
12 fruits=("apple" "banana" "cherry" "date")
13
14 # 遍历数组中的元素
15 for fruit in "${fruits[@]}; do
16     echo "水果是: $fruit"
17 done
18
19 13:43:01 root@redis02:~# bash test10.sh
20 水果是: apple
21 水果是: banana
22 水果是: cherry
23 水果是: date
24 13:43:04 root@redis02:~#

```

6.1.1.2.4 总结

生成方式	描述	示例代码
基本数字范围 <code>{start..end}</code>	生成从 <code>start</code> 到 <code>end</code> 的数字列表	<code>for i in {1..5}; do ... done</code>
数字范围 <code>{start..end..step}</code>	生成带步长的数字列表	<code>for i in {1..10..2}; do ... done</code>
<code>seq</code> 命令生成列表	使用 <code>seq</code> 命令生成从 <code>start</code> 到 <code>end</code> 的数字范围	<code>for num in \$(seq 1 1 5); do ... done</code>
文件列表 <code>*</code>	生成指定目录下的文件列表	<code>for file in *; do ... done</code>
数组生成列表	使用数组的元素生成列表	<code>for item in "\${arr[@]}; do ... done</code>
命令输出生成列表	使用命令输出的结果生成列表	<code>for file in \$(ls); do ... done</code>
<code>find</code> 命令生成列表	使用 <code>find</code> 命令查找符合条件的文件生成列表	<code>for file in \$(find . -name "*.txt"); do ... done</code>

6.1.1.3 生产案例

6.1.1.3.1 批量备份文件

定期备份某个目录下的所有文件，并将备份存储到另一个目录。

```
1 16:47:00 root@redis02:~# cat test11.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test11.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 源目录和目标备份目录
16 SOURCE_DIR="/var/www/html"
17 BACKUP_DIR="/backup/www"
18
19 # 创建备份目录（如果不存在）
20 mkdir -p $BACKUP_DIR
21
22 # 当前时间戳，用于备份文件夹命名
23 TIMESTAMP=$(date +%Y%m%d_%H%M%S)
24
25 # 遍历源目录中的所有文件并复制到备份目录
26 for file in $SOURCE_DIR/*; do
27     cp -r $file $BACKUP_DIR/${basename $file}_$TIMESTAMP
28     echo "已备份: $file"
29 done
30
31 echo "备份完成！"
```

说明：

- 使用 `for` 循环遍历源目录下的所有文件。
- 每个文件被复制到备份目录，并在文件名中加上时间戳。
- 可以根据需要调整文件或目录结构，定期执行该脚本进行备份。

6.1.1.3.2 批量处理日志文件

从多个日志文件中提取特定信息并生成汇总报告。

```
1 16:48:10 root@redis02:~# cat test12.sh
```

```

2 #!/bin/bash
3 #
=====
====
4 # 脚本基础信息
5 # filename: test12.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
=====
====
11 # 日志目录和汇总文件
12 LOG_DIR="/var/log"
13 REPORT_FILE="/var/reports/log_summary.txt"
14
15 # 创建汇总文件（如果不存在）
16 touch $REPORT_FILE
17 echo "日志汇总报告" > $REPORT_FILE
18 echo "===== " >> $REPORT_FILE
19
20 # 遍历日志目录中的所有日志文件
21 for log_file in $LOG_DIR/*.log; do
22     echo "正在处理日志: $log_file"
23
24     # 从日志文件中提取错误信息并添加到报告
25     grep "ERROR" $log_file >> $REPORT_FILE
26     echo "===== " >> $REPORT_FILE
27 done
28
29 echo "日志汇总完成, 报告已生成: $REPORT_FILE"

```

6.1.1.3.3 批量处理过期文件

定期清理过期的文件，例如删除日志目录下 30 天前的文件。

```

1 16:59:06 root@redis02:~# cat test13.sh
2 #!/bin/bash
3 #
=====
====
4 # 脚本基础信息
5 # filename: test13.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys

```

```

10 #
=====
====
11
12 # 日志目录
13 LOG_DIR="/var/log"
14
15 # 清理过期文件的时间（30 天）
16 DAYS=30
17
18 # 遍历日志目录中的文件
19 for file in $LOG_DIR/*; do
20     # 检查文件是否已超过指定天数
21     if [[ $(find $file -mtime +$DAYS -print) ]]; then
22         rm -f $file
23         echo "已删除过期文件: $file"
24     fi
25 done
26
27 echo "过期文件清理完成！"

```

6.1.1.3.4 批量处理数据库备份

定期备份多个数据库，并将备份文件保存到指定目录。

```

1 17:00:43 root@redis02:~# cat test14.sh
2 #!/bin/bash
3 #
=====
====
4 # 脚本基础信息
5 # filename: test14.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
=====
====
11 # 数据库信息
12 DBS=("db1" "db2" "db3")
13 BACKUP_DIR="/backups/db"
14
15 # 当前时间戳，用于备份文件命名
16 TIMESTAMP=$(date +%Y%m%d_%H%M%S)
17
18 # 遍历数据库列表进行备份
19 for db in "${DBS[@]}; do
20     echo "正在备份数据库: $db..."

```

```

21
22 # 使用 mysqldump 进行数据库备份
23 mysqldump -u root -p'password' $db > $BACKUP_DIR/$db-$TIMESTAMP.sql
24
25 if [ $? -eq 0 ]; then
26     echo "数据库 $db 备份成功"
27 else
28     echo "数据库 $db 备份失败"
29 fi
30 done
31
32 echo "所有数据库备份完成！"

```

说明：

- 使用 `mysqldump` 命令备份多个数据库，并将备份文件保存到指定目录。
- 每个备份文件使用当前时间戳来命名，避免文件覆盖。

6.1.1.4 for()

在 C 语言风格的 `for` 循环中，`(( ))` 并不是用来表示循环的语法，而是 **算术运算** 或 **条件判断** 的一部分。`(( ))` 是一种用于执行 **算术运算** 和 **测试条件** 的结构。这个语法通常出现在 `for` 循环的 **初始化**、**条件表达式** 和 **增量** 部分，用来进行算术操作或条件判断。

`(( ))` 语法是 **算术扩展** (Arithmetic Expansion)，它允许在其中进行 **数学运算** 或 **条件判断**。例如，你可以在 `for` 循环中使用 `((i++))` 来递增变量，或者使用 `((i < 10))` 来作为循环的条件判断。

6.1.1.4.1 基本语法

```

1 for ((initialization; condition; increment)); do
2     # 循环体
3 done

```

- `initialization`：初始化语句，通常用来设定循环变量的初始值。
- `condition`：条件判断，决定循环是否继续。通常是一个算术表达式。
- `increment`：增量操作，通常用来更新循环变量。

6.1.1.4.2 示例

在 `(( ))` 中，你可以进行 **算术运算**，例如加法、减法、乘法、除法、取模等。

```

1 for ((i = 1; i <= 10; i++)); do
2     echo $i
3 done
4
5 -----
6 sum=0

```

```

7  for ((i = 1; i <= 10; i++)); do
8      sum=$((sum + i)) # 逐步加到 sum 中
9  done
10
11 echo "总和: $sum"
12
13
14
15 -----
16 max=0
17 for ((i = 1; i <= 10; i++)); do
18     if ((i > max)); then
19         max=$i # 如果 i 大于 max, 则更新 max
20     fi
21 done
22
23 echo "最大值: $max"

```

6.2.2 while 循环

while 命令有点像 if/then 和 for 循环之间的结合，while 走循环之前会对输入的值进行条件判断，如果满足条件的话，才会进入到循环体中执行对应的语句，否则的话就退出循环。

6.2.2.1 while 语法解析

6.2.2.1.1 基本语法

```

1  while [ condition ]
2  do
3      # 执行的命令
4  done

```

- `condition` 是循环继续执行的条件表达式。
- 当 `condition` 为 **真** 时，循环继续执行 `do` 和 `done` 之间的命令。
- 一旦 `condition` 变为 **假**，循环终止。

6.2.2.1.2 结构说明

- `while`：定义循环的开始。
- `[ condition ]`：条件表达式，可以是数字比较、字符串比较或文件检查等。
- `do`：开始执行循环体。
- `done`：结束循环体。

6.2.2.2 while 循环条件

条件支持的样式 命令、[[字符串表达式]]、((数字表达式))
true 是一个特殊的条件，代表条件永远成立

6.2.2.3 while 实践

6.2.2.3.1 监控服务状态并自动重启

定期检查指定服务的状态，如果服务没有运行，则自动重启该服务。

```
1 17:07:36 root@redis02:~# cat test15.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test15.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 服务名称
16 SERVICE="apache2"
17
18 # 重启服务的函数
19 restart_service() {
20     echo "$(date): 服务 $SERVICE 未运行，正在重启..."
21     systemctl restart $SERVICE
22     if [ $? -eq 0 ]; then
23         echo "$(date): 服务 $SERVICE 重启成功"
24     else
25         echo "$(date): 服务 $SERVICE 重启失败"
26     fi
27 }
28
29 # 持续监控服务状态
30 while true; do
31     # 检查服务是否正在运行
32     systemctl is-active --quiet $SERVICE
33     if [ $? -ne 0 ]; then
34         # 服务没有运行，进行重启
35         restart_service
36     fi
37     # 每隔 60 秒检查一次
38     sleep 60
39 done
```

说明：

- `systemctl is-active --quiet $SERVICE` 用于检查服务是否运行。
- 如果服务没有运行，调用 `systemctl restart $SERVICE` 重启该服务。

- `while true` 创建一个无限循环，每隔 60 秒检查一次服务状态。

6.2.2.3.2 磁盘空间监控

持续监控服务器的磁盘空间使用情况，当某个磁盘分区使用率超过 90% 时，发送警告邮件给管理员。

```
1 17:09:21 root@redis02:~# cat test16.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test16.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 磁盘使用阈值
16 THRESHOLD=90
17
18 # 持续监控磁盘空间
19 while true; do
20     # 获取所有分区的磁盘使用百分比
21     df -h | awk 'NR>1 {print $1 " " $5}' | while read line; do
22         # 提取分区名称和使用百分比
23         PARTITION=$(echo $line | awk '{print $1}')
24         USAGE=$(echo $line | awk '{print $2}' | sed 's/%//')
25
26         # 如果使用率超过阈值，发送邮件通知
27         if [ $USAGE -ge $THRESHOLD ]; then
28             echo "$(date): 磁盘分区 $PARTITION 使用率 $USAGE%，发送警告邮件" |
29             mail -s "磁盘空间警告 - $PARTITION" admin@example.com
30         fi
31     done
32
33     # 每 10 分钟检查一次
34     sleep 600
35 done
36
37 17:09:22 root@redis02:~#
```

说明：

- 使用 `df -h` 命令获取磁盘使用信息，通过 `awk` 和 `sed` 提取分区和使用百分比。
- 如果某个分区的使用率超过设定阈值（如 90%），发送警告邮件给管理员。

- 每 10 分钟检查一次磁盘使用情况。

6.2.2.3.3 数据库备份监控

持续监控数据库备份的状态，并在备份失败时发送通知。每当备份任务完成时，记录备份日志。

```
1 17:10:41 root@redis02:~# cat test17.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test17.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 数据库相关信息
16 DB_NAME="my_database"
17 DB_USER="root"
18 DB_PASSWORD="password"
19 BACKUP_DIR="/backups"
20 LOG_FILE="/var/log/db_backup.log"
21
22 # 持续监控数据库备份任务
23 while true; do
24     echo "$(date): 开始备份数据库 $DB_NAME ..." >> $LOG_FILE
25
26     # 执行数据库备份
27     mysqldump -u $DB_USER -p$DB_PASSWORD $DB_NAME >
28     $BACKUP_DIR/$DB_NAME-$(date +%Y%m%d%H%M%S).sql
29
30     if [ $? -eq 0 ]; then
31         echo "$(date): 数据库备份成功" >> $LOG_FILE
32     else
33         echo "$(date): 数据库备份失败" >> $LOG_FILE
34         # 发送邮件或通知管理员
35         echo "$(date): 备份失败，发送通知给管理员" | mail -s "数据库备份失败"
36         admin@example.com
37     fi
38
39     # 每 24 小时备份一次
40     sleep 86400
41 done
```

说明：

- 使用 `mysqldump` 命令备份数据库。
- 如果备份成功，记录到日志文件。如果失败，发送邮件通知管理员。
- 每 24 小时执行一次备份任务

6.2.2.4 while read line

在 **Shell 脚本** 中，使用 `while read line` 是读取文件内容的常见方式。它可以逐行读取文件的内容并对每一行进行处理。

6.2.2.4.1 while read line 的语法

```
1 while read line; do
2     # 对每一行进行处理
3     echo "$line"
4 done < filename
```

- `read line`： `read` 命令读取一行输入并将其赋值给变量 `line`。
- `while ...; do ... done`： `while` 循环将持续执行，直到文件中的所有行都被读取完。
- `< filename`： 将文件内容传递给 `while` 循环进行处理。这里 `filename` 是你想要读取的文件。

6.2.2.4.2 示例

```
1 18:34:04 root@redis02:~# cat demo22.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo22.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 逐行读取文件 example.txt
16 while read line; do
17     # 输出每一行的内容
18     echo "读取到的行: $line"
19 done < example.txt
20 18:34:05 root@redis02:~# cat example.txt
21 apple
22 banana
23 cherry
```

```

20 date
21 18:34:09 root@redis02:~# bash demo22.sh
22 读取到的行: apple
23 读取到的行: banana
24 读取到的行: cherry
25 读取到的行: date
26 18:34:12 root@redis02:~#

```

6.2.2.4.3 处理空行

```

1 while read line; do
2     # 忽略空行
3     if [ -z "$line" ]; then
4         continue
5     fi
6     echo "读取到的行: $line"
7 done < example.txt

```

6.2.2.4.4 处理每行的字段

在很多情况下，我们读取的文件中的每一行不仅包含一个完整的字符串，而是由多个 **字段 (field)** 组成的。例如，CSV 文件、制表符分隔的文件，甚至是空格分隔的文件，每一行都有多个字段。为了从每一行中提取出不同的字段，Shell 提供了 **IFS** (Internal Field Separator) 来作为字段分隔符。

示例：读取 CSV 文件（以逗号分隔的文件）

假设你有一个 CSV 文件，内容如下：

```

1 apple,10,1.2
2 banana,20,0.8
3 cherry,30,2.5

```

可以使用 `while read line` 逐行读取文件，并根据逗号分隔每一行的数据。

```

1 18:39:55 root@redis02:~# cat demo23.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: demo23.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 # =====
12 #

```

```
11 # 设置 IFS 为逗号 (CSV 文件), 默认 IFS 是空格
12 IFS=', '
13
14 while read fruit quantity price; do
15     echo "水果: $fruit, 数量: $quantity, 价格: $price"
16 done < text.csv
17 18:39:57 root@redis02:~# cat text.csv
18 apple,10,1.2
19 banana,20,0.8
20 cherry,30,2.5
21
22 18:40:00 root@redis02:~# bash demo23.sh
23 水果: apple, 数量: 10, 价格: 1.2
24 水果: banana, 数量: 20, 价格: 0.8
25 水果: cherry, 数量: 30, 价格: 2.5
26 水果: , 数量: , 价格:
27 18:40:14 root@redis02:~#
```

6.2.3 until 循环

`until` 循环是另一种用于循环执行命令的控制结构, 它与 `while` 循环类似, 但有一个关键的区别: `until` 循环会在条件为 **假** 时继续执行, 直到条件变为 **真** 为止。

6.2.3.1 语法结构

```
1 until [ condition ]
2 do
3     # 执行的命令
4 done
```

- `condition` 是条件表达式。
- `until` 循环会一直执行循环体内的命令, 直到 `condition` 为 **真**。
- 只有在 `condition` 为 **假** 时, `until` 循环才会继续执行。

6.2.3.2 `until` 循环与 `while` 循环的对比

- `while` 循环: 在条件为 **真** 时, 执行循环体, 条件变为 **假** 时退出。
- `until` 循环: 在条件为 **假** 时, 执行循环体, 条件变为 **真** 时退出。

```

1 # while 循环示例
2 counter=1
3 while [ $counter -le 5 ]; do
4     echo "数字是: $counter"
5     ((counter++))
6 done
7
8 # until 循环示例
9 counter=1
10 until [ $counter -gt 5 ]; do
11     echo "数字是: $counter"
12     ((counter++))
13 done

```

在这两个例子中，`while` 和 `until` 循环的执行效果相同，但它们的条件判断方式不同。`while` 只要条件为真就执行，而 `until` 只要条件为假就执行。

6.2.3.3 `until` 循环的常见示例

6.2.3.3.1 打印从 1 到 5 的数字

打印从 1 到 5 的数字。

```

1 #!/bin/bash
2 #
3 #####
4 # 脚本基础信息
5 # filename: test17.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
11 #####
12 # 使用 until 循环
13 until [ $counter -gt 5 ]; do
14     if [ $counter -eq 3 ]; then
15         echo "达到了计数值 3，退出循环"
16         break
17     fi
18     echo "数字是: $counter"
19     ((counter++))
20 done

```

6.2.3.3.2 用户输入条件判断

不断读取用户输入，直到输入 "exit" 时退出。

```
1 #!/bin/bash
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: test17.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 # 提示用户输入
11 echo "请输入命令（输入 'exit' 退出）："
12
13 # 使用 until 循环等待用户输入
14 until [ "$input" == "exit" ]; do
15     read input
16     echo "你输入的是：$input"
17 done
18
19 echo "程序结束！"
```

6.2.3.3.3 监控系统磁盘空间

监控系统磁盘空间，直到某个分区的使用率低于指定阈值。

```
1 #!/bin/bash
2 #
   =====
   ====
3 # 脚本基础信息
4 # filename: test17.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys
9 #
   =====
   ====
10 # 提示用户输入
11 echo "请输入命令（输入 'exit' 退出）："
12
13 # 使用 until 循环等待用户输入
```

```
14 until [ "$input" == "exit" ]; do
15     read input
16     echo "你输入的是: $input"
17 done
18
19 echo "程序结束! "
```

6.2.4 循环控制 break continue

在 Shell 脚本中, `break` 和 `continue` 是两个用于控制循环流的关键字。它们通常与 `for`、`while` 或 `until` 循环一起使用, 能够帮助我们更精确地控制循环的执行。

6.2.4.1 break 语句详解

6.2.4.1.1 语法

```
1 break [n]
```

`n`: 是一个可选的数字, 表示跳出循环的层数。当没有指定时, `break` 会退出当前最内层的循环。如果有嵌套循环, 可以通过指定 `n` 来退出外层的循环。

6.2.4.1.2 功能

`break` 用于提前退出循环, 通常在满足某个条件时使用, 可以使循环在不需要继续迭代时提前结束。

6.2.4.1.3 示例

6.2.4.1.3.1 退出 for 循环

```
1 17:30:10 root@redis02:~# cat test18.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test18.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 打印从 1 到 10 的数字, 直到遇到 5 时退出
16 for i in {1..10}; do
17     if [ $i -eq 5 ]; then
18         echo "遇到数字 5, 退出循环"
19         break
20     fi
21 done
```

```

16     fi
17     echo "数字是: $i"
18 done
19
20 17:30:12 root@redis02:~# bash test18.sh
21 数字是: 1
22 数字是: 2
23 数字是: 3
24 数字是: 4
25 遇到数字 5, 退出循环

```

6.2.4.1.3.2 嵌套循环中使用 break

```

1 17:31:06 root@redis02:~# cat test19.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test19.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 外层循环
16 for i in {1..3}; do
17     echo "外层循环 $i"
18
19     # 内层循环
20     for j in {1..3}; do
21         if [ $j -eq 2 ]; then
22             echo "    内层循环遇到 2, 跳出外层循环"
23             break 2 # 跳出外层循环
24         fi
25         echo "    内层循环 $j"
26     done
27 done
28
29 17:31:07 root@redis02:~# bash test19.sh
30 外层循环 1
31     内层循环 1
32     内层循环遇到 2, 跳出外层循环
33
34 17:31:09 root@redis02:~# vim test19.sh
35 17:31:34 root@redis02:~# cat test19.sh
36 #!/bin/bash

```



```

32  #
=====
====
33  # 脚本基础信息
34  # filename: test19.sh
35  # name: xuruizhao
36  # email: xuruizhao00@163.com
37  # v: LnxGuru
38  # GitHub: xuruizhao00-sys
39  #
=====
====
40  # 外层循环
41  for i in {1..3}; do
42      echo "外层循环 $i"
43
44      # 内层循环
45      for j in {1..3}; do
46          if [ $j -eq 2 ]; then
47              echo "    内层循环遇到 2，跳出外层循环"
48              break 1 # 跳出外层循环
49          fi
50          echo "    内层循环 $j"
51      done
52  done
53
54  17:31:36 root@redis02:~# bash test19.sh
55  外层循环 1
56      内层循环 1
57      内层循环遇到 2，跳出外层循环
58  外层循环 2
59      内层循环 1
60      内层循环遇到 2，跳出外层循环
61  外层循环 3
62      内层循环 1
63      内层循环遇到 2，跳出外层循环
64  17:31:37 root@redis02:~#

```

6.2.4.2 `continue` 语句详解

6.2.4.2.1 语法

```
1 | continue [n]
```

`n`：表示跳过当前循环的哪一层。如果没有指定 `n`，则 `continue` 会跳过当前最内层循环的当前迭代。

6.2.4.2.2 功能

`continue` 用于跳过当前循环中的剩余部分，直接进行下一次循环的迭代。在某些条件下，可能希望跳过当前的循环步骤并继续处理后续的内容。

6.2.4.2.3 示例

6.2.4.2.3.1 跳过某些特定值

```
1 17:33:53 root@redis02:~# cat test20.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: test20.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 打印从 1 到 10 的数字，跳过 5 和 7
16 for i in {1..10}; do
17     if [ $i -eq 5 ] || [ $i -eq 7 ]; then
18         echo "跳过数字: $i"
19         continue
20     fi
21     echo "数字是: $i"
22 done
23
24 17:33:56 root@redis02:~# bash test20.sh
25 数字是: 1
26 数字是: 2
27 数字是: 3
28 数字是: 4
29 跳过数字: 5
30 数字是: 6
31 跳过数字: 7
32 数字是: 8
33 数字是: 9
34 数字是: 10
35
36 17:33:59 root@redis02:~#
```

6.2.4.2.3.2 嵌套循环中使用 `continue`

```
1 17:34:59 root@redis02:~# cat test21.sh
2 #!/bin/bash
```

```

3  #
  =====
  ====
4  # 脚本基础信息
5  # filename: test21.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 # 外层循环
12 for i in {1..3}; do
13     echo "外层循环 $i"
14
15     # 内层循环
16     for j in {1..3}; do
17         if [ $j -eq 2 ]; then
18             echo "    跳过内层循环 $j"
19             continue # 跳过内层循环的当前迭代
20         fi
21         echo "    内层循环 $j"
22     done
23 done
24
25 17:35:00 root@redis02:~# bash test21.sh
26 外层循环 1
27     内层循环 1
28     跳过内层循环 2
29     内层循环 3
30 外层循环 2
31     内层循环 1
32     跳过内层循环 2
33     内层循环 3
34 外层循环 3
35     内层循环 1
36     跳过内层循环 2
37     内层循环 3

```

6.2.4.3 exit 语句详解

6.2.4.3.1 语法

```
1 | exit [n]
```

n：表示退出脚本的状态码。默认情况下，`exit` 会返回状态码 `0`，表示脚本执行成功。非零状态码通常表示脚本执行出现错误，状态码的具体含义可以根据实际需求定义。

6.2.4.3.2 功能

`exit` 用于退出当前脚本的执行。通常在脚本的最后，或者在遇到错误时使用 `exit` 终止脚本执行。

6.2.4.3.3 示例

6.2.4.3.3.1 退出脚本

```
1 echo "开始执行脚本"
2
3 # 执行某些命令，假设命令失败
4 echo "命令执行失败"
5
6 # 使用 exit 退出脚本并返回非零状态码
7 exit 1
```

6.2.4.4 shift 语句详解

6.2.4.4.1 语法

```
1 shift [n]
```

`n`：是一个可选的数字，表示要移出的位置参数的个数。默认为 1，表示移出第一个位置参数（即 `$1`），并将剩余的参数左移。如果指定了 `n`，则移出前 `n` 个位置参数，剩下的参数会左移。

6.2.4.4.2 功能

`shift` 用于移动脚本中的位置参数。它可以让我们在脚本中按顺序访问位置参数，而不需要明确地引用每个参数（例如 `$1`，`$2`，`$3` 等）。当我们需要逐个处理传入的参数时，`shift` 是非常有用的。

6.2.4.4.3 示例

6.2.4.4.3.1 逐个访问命令行参数

```
1 17:40:18 root@redis02:~# cat test22.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: test22.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
```

```

10 #
    =====
    ====
11 # 打印所有传入的命令行参数
12 echo "传入的参数: $@"
13
14 # 使用 shift 逐个处理参数
15 while [ $# -gt 0 ]; do
16     echo "当前参数是: $1"
17     shift # 移出第一个位置参数, 处理下一个
18 done
19
20 17:40:51 root@redis02:~# bash test22.sh 1 2 3 4 5
21 传入的参数: 1 2 3 4 5
22 当前参数是: 1
23 当前参数是: 2
24 当前参数是: 3
25 当前参数是: 4
26 当前参数是: 5

```

6.2.4.4.3.2 结合 `shift` 和 `case` 处理不同的命令行选项

```

1 17:45:01 root@redis02:~# cat test23.sh
2 #!/bin/bash
3 #
    =====
    ====
4 # 脚本基础信息
5 # filename: test23.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11 while [ $# -gt 0 ]; do
12     case $1 in
13         -a)
14             echo "选项 -a 被指定"
15             ;;
16         -b)
17             echo "选项 -b 被指定"
18             ;;
19         *)
20             echo "无效选项: $1"
21             ;;
22     esac
23     shift # 移出当前选项

```

```

24 done
25
26 17:45:02 root@redis02:~# bash test23.sh -a -b
27 选项 -a 被指定
28 选项 -b 被指定
29 17:45:06 root@redis02:~# bash test23.sh -a -b -c
30 选项 -a 被指定
31 选项 -b 被指定
32 无效选项: -c
33 17:45:12 root@redis02:~#

```

6.2.4.5 总结

控制语句	功能	适用场景
<code>break</code>	提前退出循环	当满足某个条件时退出循环
<code>continue</code>	跳过当前循环的剩余部分，继续下一次迭代	跳过某些不需要处理的循环步骤
<code>exit</code>	退出脚本并返回状态码	当脚本执行成功或遇到错误时退出脚本
<code>shift</code>	移动位置参数，方便逐个处理命令行参数	逐个处理命令行传入的多个参数
使用场景总结：		

- `break`：用于在特定条件下退出循环。
- `continue`：用于跳过当前循环的某些步骤。
- `exit`：用于退出脚本，返回状态码，指示脚本的执行结果。
- `shift`：用于在脚本中处理位置参数，逐个访问命令行传入的参数。

七、函数

在 shell 脚本的编写过程中，我们经常会遇到一些功能代码场景：多条命令组合在一起，实现一个特定的功能场景逻辑、一些命令在脚本内部的多个位置频繁出现。在这些场景的代码量往往不多，但是频繁使用的话，会导致脚本的整体逻辑脉络比较松散和框架散乱。

所以我们需要一种脚本逻辑，不仅仅能够满足松散代码的功能目的，还能精简重复的代码。函数就是来满足这种场景的解决方案 -- **而函数，也是所谓的面向对象编程的一种表现样式。**

所谓的函数，本质上就是一段能够满足特定功能的代码块。一旦定义好函数代码后，我们就可以在脚本的很多位置随意的使用。定义功能代码块的动作叫 函数定义，使用函数代码的动作叫 函数调用。

函数的优势：

1. 代码模块化，调用方便，节省内存
2. 代码模块化，代码量少，排错简单

3. 代码模块化，可以改变代码的执行顺序

7.1 函数基本语法

7.1.1 定义函数

Shell 函数的定义方式有两种：

7.1.1.1 使用 `function` 关键字定义

```
1 function function_name {  
2     # 函数体  
3 }
```

7.1.1.2 使用直接定义的方式（不使用 `function` 关键字）

```
1 function_name() {  
2     # 函数体  
3 }
```

两种定义方式的功能完全相同，选择其一即可。

7.1.2 调用函数

函数定义后，可以通过函数名来调用：

```
1 function_name # 调用函数
```

7.1.3 函数返回值

Shell 函数通过 `return` 语句返回一个整数值（0-255）。如果需要返回更复杂的数据（如字符串或数组），可以使用输出重定向或通过命令的标准输出返回。

7.2 函数退出方式和退出状态

7.2.1 函数退出方式

7.2.1.1 使用 `return` 退出函数

在 Shell 中，`return` 语句用于退出函数并将一个状态码返回给调用者。函数的退出状态码通常是一个整数值（0 到 255）。如果没有指定状态码，`return` 默认返回 0，表示函数执行成功。

```
1 return [n]
```

`n`：是一个可选的整数，表示退出状态码。默认为 0，表示成功；非零值表示错误或失败。

```

1 20:32:13 root@redis02:~# cat demo01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 定义函数
16 function check_number {
17     if [ $1 -gt 10 ]; then
18         echo "数字大于 10"
19         return 0 # 返回 0 表示成功
20     else
21         echo "数字小于或等于 10"
22         return 1 # 返回 1 表示失败
23     fi
24 }
25
26 # 调用函数并检查退出状态
27 check_number 15
28 if [ $? -eq 0 ]; then
29     echo "函数执行成功"
30 else
31     echo "函数执行失败"
32 fi
33
34 20:32:14 root@redis02:~# bash demo01.sh
35 数字大于 10
36 函数执行成功
37 20:32:16 root@redis02:~# echo $?
38 0
39 20:32:19 root@redis02:~#
40

```

- `return 0` 表示函数成功执行。
- 使用 `$?` 可以获取函数的退出状态码。

7.2.1.2 使用 `exit` 退出整个脚本

如果函数内部执行了 `exit` 命令，则整个脚本会终止，函数及脚本的执行都停止。`exit` 用于脚本的完全终止，退出状态码为一个整数（0-255）。


```
1 | exit [n]
```

`n`：是一个可选的整数，表示退出状态码。如果不指定，默认为 0。

```
1 20:34:43 root@redis02:~# cat demo02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 #!/bin/bash
16
17 # 定义函数
18 function terminate_script {
19     echo "遇到错误，脚本终止"
20     exit 1 # 退出脚本，返回状态码 1
21 }
22
23 # 调用函数
24 terminate_script
25
26 echo "这行代码不会执行" # 因为脚本已退出
27
28 20:34:45 root@redis02:~# bash demo02.sh
29 遇到错误，脚本终止
30 20:34:48 root@redis02:~# echo $?
31 1
32 20:34:51 root@redis02:~#
```

使用 `exit` 终止了整个脚本的执行。后面的代码（如 `echo "这行代码不会执行"`）不会再执行。\\

7.2.2 函数的退出状态

在 Shell 中，每个命令和函数执行后都有一个退出状态码，表示命令或函数是否成功执行。退出状态码是一个整数，范围从 0 到 255。

- 0：表示成功。
- 非 0：表示失败，具体数值可以根据不同的错误类型进行定义。

Shell 中有一个特殊的变量 `$?`，它保存着上一条命令或函数执行后的退出状态码。

7.2.2.1 获取函数的退出状态

当函数执行完毕后，可以通过 `$?` 获取函数的退出状态码，进而判断函数是否执行成功。

```
1 20:41:10 root@redis02:~# cat demo03.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo03.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16 function my_function {
17     echo "执行一些操作"
18     return 2 # 返回 2, 表示某种错误
19 }
20
21 # 调用函数
22 my_function
23
24 # 获取并输出函数的退出状态码
25 echo "函数的退出状态码是: $?"
26
27 20:41:14 root@redis02:~# bash demo03.sh
28 执行一些操作
29 函数的退出状态码是: 2
30
31 20:43:55 root@redis02:~# echo $?
32 0
33
34 20:43:59 root@redis02:~#
```

7.2.2.2 使用退出状态进行条件判断

通过退出状态码，Shell 脚本可以根据命令或函数的执行结果进行不同的处理。常见的做法是使用 `if` 语句来判断退出状态。

```
1 20:45:07 root@redis02:~# cat demo04.sh
2 #!/bin/bash
3 #
4 =====
5 =====
6
7 # 脚本基础信息
```

```

5  # filename: demo04.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
    =====
    ====
11
12 # 定义函数，判断文件是否存在
13 function check_file {
14     if [ -f "$1" ]; then
15         return 0 # 文件存在，返回 0
16     else
17         return 1 # 文件不存在，返回 1
18     fi
19 }
20
21 # 调用函数并判断退出状态
22 check_file "/path/to/file.txt"
23
24 if [ $? -eq 0 ]; then
25     echo "文件存在"
26 else
27     echo "文件不存在"
28 fi
29
30 20:45:10 root@redis02:~# bash demo04.sh
31 文件不存在
32 20:45:12 root@redis02:~# echo $?
33 0
34 20:45:15 root@redis02:~#

```

7.2.3 return 与 exit 的区别

特性	return	exit
作用范围	退出当前函数，返回指定的退出状态码	退出整个脚本，返回指定的退出状态码
退出状态	返回一个整数（0-255），表示函数执行结果	返回一个整数（0-255），表示脚本执行结果
使用场景	用于退出函数，并返回执行状态	用于退出脚本，通常在错误或特殊条件下使用

7.3 函数参数

7.3.1 函数参数概述

Shell 中的函数参数是通过 **位置参数** 来传递的。位置参数是指传递给脚本或函数的具体值，在函数内部可以通过 `$1`, `$2`, `$3` 等访问。Shell 函数的参数是传递给函数的一组值，这些参数可以是数字、字符串、文件名等。

7.3.2 调用函数并传递参数

通过 `function_name arg1 arg2 ...` 的方式传递参数。

```
1 function_name arg1 arg2 arg3
```

```
1 20:48:44 root@redis02:~# cat demo05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 #!/bin/bash
16
17 # 定义一个函数，接受两个参数
18 greet() {
19     echo "Hello, $1! You are $2 years old."
20 }
21
22 # 调用函数并传递两个参数
23 greet "Alice" 30
24
25 20:48:45 root@redis02:~# bash demo05.sh
26 Hello, Alice! You are 30 years old.
27 20:48:49 root@redis02:~#
```

7.3.3 函数获取参数

7.3.3.1 获取单个函数

Shell 函数的参数可以通过 `$1`, `$2`, `$3` 等方式获取, 依次代表第一个、第二个、第三个参数。

```
1 #!/bin/bash
2
3 # 定义函数, 获取并输出第一个参数
4 print_param() {
5     echo "第一个参数是: $1"
6 }
7
8 # 调用函数并传递一个参数
9 print_param "Hello"
10
```

7.3.3.2 获取所有参数

示例: 使用 `$@` 获取所有参数

```
1 20:53:45 root@redis02:~# cat demo06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 #!/bin/bash
16
17 # 定义函数, 输出所有参数
18 print_all_params() {
19     echo "所有参数是: $@"
20     for item in "$@";do
21         echo $item
22     done
23 }
24
25 # 调用函数并传递多个参数
26 print_all_params "Alice" 30 "Engineer"
```

```
25 20:53:46 root@redis02:~# bash demo06.sh
26 所有参数是: Alice 30 Engineer
27 Alice
28 30
29 Engineer
```

示例：使用 `$*` 获取所有参数

```
1 20:53:22 root@redis02:~# cat demo07.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: demo06.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 # =====
13 # 定义函数，输出所有参数
14 print_all_params() {
15     echo "所有参数是: $*"
16     for item in "$*";do
17         echo $item
18     done
19 }
20
21 # 调用函数并传递多个参数
22 print_all_params "Alice" 30 "Engineer"
23
24
25 20:53:23 root@redis02:~# bash demo07.sh
26 所有参数是: Alice 30 Engineer
27 Alice 30 Engineer
```

7.3.3.3 获取参数的数量

Shell 提供了 `$#` 变量来获取传递给函数的参数个数

```
1 20:54:26 root@redis02:~# cat demo08.sh
2 #!/bin/bash
```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo08.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11
12 # 定义函数，输出参数个数
13 print_param_count() {
14     echo "参数个数是: $#"

```

7.3.4 默认值与参数检查

7.3.4.1 设置默认值

如果某些参数未传递，可以通过在函数内部设置默认值来避免错误。

```

1  20:55:40 root@redis02:~# cat demo09.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo09.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11
12 # 定义函数，设置默认值
13 greet() {
14     name=${1:-"Guest"} # 如果未传递参数，使用默认值 "Guest"
15     echo "Hello, $name!"

```

```

16 }
17
18 # 调用函数并传递参数
19 greet "Alice"
20
21 # 调用函数，不传递参数
22 greet
23
24
25 20:55:42 root@redis02:~# bash demo09.sh
26 Hello, Alice!
27 Hello, Guest!
28 20:55:44 root@redis02:~#

```

7.3.4.2 参数检查

可以使用 `if` 语句检查传递给函数的参数是否符合预期。

```

1 20:58:24 root@redis02:~# cat demo10.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo10.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 定义函数，检查参数个数
16 check_params() {
17     if [ $# -lt 2 ]; then
18         echo "错误：至少需要两个参数！"
19         return 1 # 返回错误状态码
20     fi
21     echo "参数个数正确，继续执行..."
22 }
23
24 # 调用函数，传递少于两个参数
25 check_params "Alice"
26
27 # 调用函数，传递两个参数
28 check_params "Alice" 30
29
30 20:58:25 root@redis02:~# bash demo10.sh

```



```
28 错误：至少需要两个参数！
29 参数个数正确，继续执行...
30 20:58:28 root@redis02:~#
```

7.4 函数递归调用

递归是指在函数内部调用自身的过程。在 Shell 脚本中，递归调用通常用于解决一些可以分解成更小子问题的问题，如计算阶乘、斐波那契数列等。

7.4.1 函数递归的基本语法

Shell 中函数的递归语法与其他编程语言类似。递归函数通常包含两个部分：

1. **递归基准条件**：当满足某个条件时，停止递归。
2. **递归调用**：函数在其内部调用自身，通常对某些参数进行改变，逐步逼近基准条件。

基本的递归函数结构如下：

```
1 function_name() {
2     # 基准条件
3     if [ some_condition ]; then
4         return # 或者其他结束递归的操作
5     fi
6
7     # 递归调用
8     function_name # 传递新的参数或条件
9 }
```

7.4.2 递归计算阶乘

阶乘是一个经典的递归例子。 $n!$ （ n 的阶乘）是从 1 到 n 所有整数的乘积。例如：

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $4! = 4 \times 3 \times 2 \times 1 = 24$

```
1 21:47:38 root@redis02:~# cat demo24.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo24.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
```

```

10 #
=====
11
12 # 计算阶乘的递归函数
13 factorial() {
14     # 基准条件: 0! 和 1! 都是 1
15     if [ $1 -le 1 ]; then
16         echo 1
17     else
18         # 递归调用: n! = n * (n-1)!
19         local result=$(( $1 - 1 ))
20         local recursive_result=$(factorial $result)
21         echo $(( $1 * recursive_result ))
22     fi
23 }
24
25 # 调用函数计算阶乘
26 echo "5! = $(factorial 5)"
27
28 21:47:39 root@redis02:~# bash demo24.sh
29 5! = 120
30 21:47:41 root@redis02:~#

```

- **递归基准条件:** 当 $n \leq 1$ 时, 直接返回 1 ($0! = 1$ 和 $1! = 1$)。
- **递归调用:** `factorial $(( $1 - 1 ))`, 递归计算 $n!$, 通过 $n * (n-1)!$ 。

7.4.3 递归计算斐波那契数列

斐波那契数列是一个经典的递归问题, 数列中的每一项是前两项的和。斐波那契数列的前几项是:

```

1 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

```

公式是: $F(n) = F(n-1) + F(n-2)$, 基准条件是 $F(0) = 0$ 和 $F(1) = 1$ 。

Shell 脚本: 递归计算斐波那契数列

```

1 21:49:17 root@redis02:~# cat demo25.sh
2 #!/bin/bash
3 #
=====
4 # 脚本基础信息
5 # filename: demo25.sh
6 # name: xuruizhao
7 # email: xuruizhao00@163.com
8 # v: LnxGuru
9 # GitHub: xuruizhao00-sys

```

```

10  #
    =====
    ====
11
12  # 计算斐波那契数列的递归函数
13  fibonacci() {
14      # 基准条件: F(0) = 0, F(1) = 1
15      if [ $1 -eq 0 ]; then
16          echo 0
17      elif [ $1 -eq 1 ]; then
18          echo 1
19      else
20          # 递归调用: F(n) = F(n-1) + F(n-2)
21          local result1=$(fibonacci $(( $1 - 1 )))
22          local result2=$(fibonacci $(( $1 - 2 )))
23          echo $(( result1 + result2 ))
24      fi
25  }
26
27  # 调用函数计算第 6 项斐波那契数列
28  echo "F(6) = $(fibonacci 6)"
29
30
31  21:49:18 root@redis02:~# bash demo25.sh
32  F(6) = 8
33  21:49:21 root@redis02:~#

```

- **递归基准条件**: 当 $n = 0$ 时返回 0, 当 $n = 1$ 时返回 1。
- **递归调用**: `fibonacci $(( $1 - 1 ))` 和 `fibonacci $(( $1 - 2 ))` 分别计算 $F(n-1)$ 和 $F(n-2)$, 然后返回它们的和。

7.4.4 递归深度限制

在一些系统中, 默认的递归深度限制可能会导致程序因为递归过深而报错。如果你遇到递归深度的问题, 可以使用 `ulimit` 命令增加栈大小限制, 或者考虑 **尾递归优化**, 或者将递归改为循环。

```

1  ulimit -s unlimited  # 增加栈的大小限制

```

八、Shell 脚本高阶部分

8.1 信号

8.1.1 信号基础知识

当我们在构建一些更高级的脚本的时候，就会涉及到如何在 linux 系统上来更好的运行和控制它们，到目前为止，我们运行脚本的方式都是以实时的模式，在命令行来运行它。但是这并不是脚本唯一的运行方式，我们可以在 linux 系统中以更丰富的方式来运行它们，甚至在脚本遇到不可查的异常中止时候，以关闭 linux 终端界面的方式终止脚本。这些能力都是基于信号的机制来实现了

linux 使用信号与系统上运行的进程进行通信，想要对 shell 的脚本控制，只需要传递相关信号给 shell 脚本即可

信号值	信号名	信号作用	如何生成
1	SIGHUP	挂起信号，通常在终端关闭时发送，用于通知进程终止或重新加载配置文件	<code>kill -SIGHUP <pid></code> 或关闭终端连接时自动发送
2	SIGINT	中断信号，通常由 <code>Ctrl+C</code> 触发，用户请求终止进程	<code>Ctrl+C</code> 或 <code>kill -SIGINT <pid></code>
3	SIGQUIT	退出信号，通常由 <code>Ctrl+\</code> 触发，进程退出并生成核心转储文件	<code>Ctrl+\</code> 或 <code>kill -SIGQUIT <pid></code>
9	SIGKILL	强制终止信号，无法捕获或忽略，直接终止进程	<code>kill -SIGKILL <pid></code>
15	SIGTERM	终止信号，要求进程正常终止，能够捕获并执行清理工作	<code>kill -SIGTERM <pid></code> 或默认 <code>kill <pid></code>
19	SIGSTOP	暂停信号，停止进程的执行	<code>kill -SIGSTOP <pid></code>
18	SIGCONT	恢复信号，恢复被暂停的进程	<code>kill -SIGCONT <pid></code>

信号值	信号名	信号作用	如何生成
20	SIGTSTP	停止信号，通常由 Ctrl+Z 触发，用于将进程放到后台	Ctrl+Z 或 kill - SIGTSTP <pid>
17	SIGCHLD	子进程终止信号，父进程收到子进程结束的通知	子进程终止时，自动发送
10	SIGUSR1	用户自定义信号，用于进程间的自定义通信	kill - SIGUSR1 <pid>
12	SIGUSR2	用户自定义信号，用于进程间的自定义通信	kill - SIGUSR2 <pid>
14	SIGALRM	定时信号，通常用于定时器到期后触发	kill - SIGALRM <pid> 或使用 alarm() 系统调用触发
11	SIGSEGV	段错误信号，进程访问非法内存时触发	由程序访问无效内存地址时自动触发
28	SIGPIPE	管道破裂信号，写入一个无读进程的管道时触发	写入关闭的管道时自动触发
22	SIGTTOU	后台进程尝试读取终端时发送的信号	通过后台进程读取终端时自动发送
23	SIGTTIN	后台进程尝试写入终端时发送的信号	通过后台进程写入终端时自动发送
24	SIGBUS	总线错误信号，通常是硬件相关错误	程序访问未映射的内存地址时触发

信号值	信号名	信号作用	如何生成
默认情况下，bash shell 会忽略收到的任何 SIGQUIT(3) 和 SIGTERM(15) 信号（正因为这样交互式 shell 才不会被意外终止）。但是 bash shell 会处理收到的 SIGHUP(1) 和 SIGINT(2) 信号。			

如果 bash shell 收到 SIGHUP 信号，它会退出。但在退出之前，它会将信号传给 shell 启动的所有进程（比如 shell 脚本）。通过 SIGINT 信号，可以中断 shell，Linux 内核停止将 CPU 的处理时间分配给 shell，当这种情况发生时，shell 会将 SIGINT 信号传给 shell 启动的所有进程。

8.1.2 生成信号

```

1 终止进程：
2  ctrl+c,
3
4 暂停进程：
5  ctrl+z, 停止的进程继续保留在内存中，并能从停止的位置继续运行
6
7 恢复进程：
8  jobs 查看运行任务，fg num 重新执行
9
10 杀死进程：
11 kill -9 pid

```

8.1.3 终止和挂起

终止进程

```

1 20:58:28 root@redis02:~# sleep 1000
2 ^C
3 21:13:09 root@redis02:~#

```

挂起进程

```

1 21:13:09 root@redis02:~# sleep 1000
2 ^Z
3 [1]+  Stopped                  sleep 1000
4 21:13:49 root@redis02:~# ps -aux | grep sleep
5 root      15212  0.1  0.0  5684  1920 pts/0    T   21:13   0:00 sleep
6          1000
7 root      15214 18.7  0.1  6544  2304 pts/0    S+  21:13   0:00 grep --
8          color=auto sleep
9 21:13:58 root@redis02:~# jobs -l
10 [1]+  15212 Stopped                  sleep 1000
11 21:14:02 root@redis02:~# fg 1
12 sleep 1000
13 ^C
14 21:14:06 root@redis02:~#

```

8.2 信号捕获

在 **Shell 编程** 中，`trap` 命令是一种强大的工具，用于捕获并处理进程在运行过程中收到的信号。通过 `trap`，可以让脚本在接收到特定信号时执行预定的操作，而不是让默认的信号处理机制（如终止脚本、暂停进程等）进行处理。

`trap` 命令可以帮助我们优雅地终止脚本、清理资源、处理错误等，在处理信号时具有很高的灵活性。

8.2.1 `trap` 命令概述

8.2.1.1 基本语法

```
1 trap "command" SIGNALS
```

- `'commands'`：当指定的信号发生时要执行的命令（可以是多个命令，用分号分隔）。
- `SIGNALS`：要捕获的信号列表，可以是一个或多个信号，多个信号用空格分隔。

8.2.1.2 作用

`trap` 允许你在脚本中捕获信号并执行自定义操作，防止默认行为的发生。常见的用途包括：

- 捕获并处理错误信号。
- 在脚本终止时执行清理工作。
- 响应用户的中断请求。
- 处理后台进程的异常。

8.2.2 `trap` 使用示例

示例 1: 捕获 SIGINT (Ctrl+C) 信号

```
1 17:10:09 root@redis02:~# cat demo01.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo01.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 捕获 SIGINT 信号 (Ctrl+C) 并执行清理操作
16 trap 'echo "捕获到 SIGINT 信号, 正在退出..."; exit' SIGINT
17 # 模拟脚本长时间运行
18 echo "脚本正在运行...按 Ctrl+C 退出"
19 while true; do
20     sleep 1
21 done
22 17:10:11 root@redis02:~# bash demo01.sh
23 脚本正在运行...按 Ctrl+C 退出
24 ^C捕获到 SIGINT 信号, 正在退出...
25 17:10:15 root@redis02:~#
```

说明:

- 当用户按下 `Ctrl+C` 时, 脚本会捕获到 `SIGINT` 信号并执行自定义的退出清理操作 (打印提示信息并退出)

示例 2: 捕获多个信号并执行相同操作

```
1 17:11:16 root@redis02:~# cat demo02.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo02.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
```



```

9  # GitHub: xuruizhao00-sys
10 #
=====
====
11
12 # 捕获 SIGINT 和 SIGTERM 信号，执行相同的清理操作
13 trap 'echo "脚本终止，清理资源..."; exit' SIGINT SIGTERM
14
15 # 模拟脚本长时间运行
16 echo "脚本正在运行...发送 SIGINT 或 SIGTERM 终止脚本"
17 while true; do
18     sleep 1
19 done
20
21
22 17:11:17 root@redis02:~# bash demo02.sh
23 脚本正在运行...发送 SIGINT 或 SIGTERM 终止脚本
24 ^C脚本终止，清理资源...

```

说明：

- 捕获 `SIGINT` (Ctrl+C) 和 `SIGTERM` (终止信号)，并执行相同的操作：输出清理信息并退出。

示例 3：在脚本退出时执行清理操作

```

1  17:16:19 root@redis02:~# cat demo03.sh
2  #!/bin/bash
3  #
=====
====
4  # 脚本基础信息
5  # filename: demo03.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
=====
====
11
12 # 定义清理函数
13 cleanup() {
14     echo "清理临时文件..."
15     rm -f /tmp/tempfile
16 }
17
18 # 捕获 SIGINT 和 SIGTERM 信号并执行清理
19 trap cleanup SIGINT SIGTERM

```

```

20
21 # 模拟脚本运行并创建临时文件
22 echo "创建临时文件..."
23 touch /tmp/tempfile
24 echo "临时文件已创建"
25
26 # 模拟脚本的长时间运行
27 while true; do
28     sleep 1
29 done
30
31
32 17:16:20 root@redis02:~# bash demo03.sh
33 创建临时文件...
34 临时文件已创建
35 ^C清理临时文件...

```

说明:

- 通过 `trap` 捕获 `SIGINT` 和 `SIGTERM` 信号，调用 `cleanup` 函数清理临时文件 `/tmp/tempfile`。

示例 4: 捕获 `EXIT` 信号

`EXIT` 是一个特殊的信号，表示脚本退出时触发。它通常用于在脚本执行完毕时进行清理工作。

```

1 17:17:20 root@redis02:~# cat demo04.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo04.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16 # 捕获 EXIT 信号，脚本结束时执行清理操作
17 trap 'echo "脚本执行完毕，进行清理操作..."; rm -f /tmp/tempfile' EXIT
18
19 # 模拟脚本运行并创建临时文件
20 echo "创建临时文件..."
21 touch /tmp/tempfile
22 echo "临时文件已创建"
23
24

```

```

20 # 模拟脚本执行
21 echo "脚本正在执行，退出时会清理临时文件"
22
23 17:17:21 root@redis02:~# bash demo04.sh
24 创建临时文件...
25 临时文件已创建
26 脚本正在执行，退出时会清理临时文件
27 脚本执行完毕，进行清理操作...
28 17:17:23 root@redis02:~#

```

说明：

- 当脚本执行完毕退出时，`EXIT` 信号被捕获并执行清理操作，删除临时文件。

8.2.3 捕获信号并忽略

有时我们希望在脚本运行时忽略某些信号。可以使用 `trap` 命令来捕获信号并什么都不做，这样就会忽略该信号。

示例：忽略 SIGINT 信号 (Ctrl+C)

```

1 17:20:20 root@redis02:~# cat demo05.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo05.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 忽略 SIGINT 信号 (Ctrl+C)
16 trap '' SIGINT
17
18 # 模拟脚本长时间运行
19 echo "脚本正在运行...按 Ctrl+C 也不会终止脚本"
20 while true; do
21     sleep 1
22 done
23
24 17:21:01 root@redis02:~# bash demo05.sh
25 脚本正在运行...按 Ctrl+C 也不会终止脚本
26 ^C
27 ^C^C^C^C^C^C

```

8.2.4 trap 命令中的信号操作

信号	说明	示例命令	操作类型
SIGINT	中断信号，通常由 <code>Ctrl+C</code> 触发	<code>trap 'commands' SIGINT</code>	捕获中断信号，执行自定义操作
SIGTERM	终止信号，要求进程终止	<code>trap 'commands' SIGTERM</code>	捕获终止信号，执行清理或退出
SIGKILL	强制终止信号，无法捕获或忽略	无法捕获，直接终止进程	强制退出进程（无法捕获或处理）
SIGQUIT	退出信号，通常由 <code>Ctrl+\</code> 触发	<code>trap 'commands' SIGQUIT</code>	捕获退出信号，生成 core dump
SIGSTOP	停止信号，暂停进程执行	<code>trap 'commands' SIGSTOP</code>	捕获停止信号，暂停进程执行
SIGCONT	恢复信号，恢复被暂停的进程	<code>trap 'commands' SIGCONT</code>	捕获恢复信号，恢复进程执行
SIGPIPE	管道破裂信号，当进程尝试向管道写入时	<code>trap 'commands' SIGPIPE</code>	捕获管道破裂信号，防止脚本退出
SIGALRM	定时信号，通常用于定时器触发	<code>trap 'commands' SIGALRM</code>	捕获定时信号，执行定时任务
EXIT	脚本退出时触发	<code>trap 'commands' EXIT</code>	捕获脚本退出时执行清理操作

8.2.5 trap 命令的高级用法

8.2.5.1 在函数内使用 trap

我们可以在函数内部使用 `trap` 来捕获并处理信号，这样可以让函数在特定信号发生时执行自定义操作。

```
1 17:25:10 root@redis02:~# cat demo06.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo06.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
```

```

10 #
=====
====
11
12 # 定义函数，捕获 SIGINT 信号
13 my_function() {
14     trap 'echo "捕获到 SIGINT 信号"; exit' SIGINT
15     while true; do
16         echo "函数正在运行...按 Ctrl+C 终止"
17         sleep 1
18     done
19 }
20
21 # 调用函数
22 my_function
23
24
25 17:25:11 root@redis02:~# bash demo06.sh
26 函数正在运行...按 Ctrl+C 终止
27 ^C捕获到 SIGINT 信号

```

8.2.6 trap 命令注意事项

8.2.6.1 trap 命令本身不具备推出脚本的能力

拿 8.2.2 小节中的示例3举例

- **trap 捕获信号并执行清理：**
 - 当你按下 `Ctrl+C` 时，`SIGINT` 信号会被触发。
 - `trap` 捕获到 `SIGINT` 后，调用 `cleanup` 函数，并执行清理操作（删除 `/tmp/tempfile`）。
 - 但是 `trap` 只是捕获信号并执行 `cleanup`，然后脚本继续执行后面的代码。
 - **trap 不会自动退出脚本：**
 - `trap` 仅仅是执行捕获信号时的操作，但它不会自动结束脚本。如果希望脚本在捕获信号后退出，必须显式调用 `exit` 命令。
 - 在 `cleanup` 函数中添加 `exit` 语句来确保脚本终止。
- 修改后的脚本如下：

```

1 #!/bin/bash
2 #
=====
====
3 # 脚本基础信息
4 # filename: demo03.sh
5 # name: xuruizhao
6 # email: xuruizhao00@163.com
7 # v: LnxGuru
8 # GitHub: xuruizhao00-sys

```

```

9  #
  =====
  ====
10
11 # 定义清理函数
12 cleanup() {
13     echo "清理临时文件..."
14     rm -f /tmp/tempfile
15     exit
16 }
17
18 # 捕获 SIGINT 和 SIGTERM 信号并执行清理
19 trap cleanup SIGINT SIGTERM
20
21 # 模拟脚本运行并创建临时文件
22 echo "创建临时文件..."
23 touch /tmp/tempfile
24 echo "临时文件已创建"
25
26 # 模拟脚本的长时间运行
27 while true; do
28     sleep 1
29 done

```

说明：

- 当 `Ctrl+C` 触发 `SIGINT` 信号时，`trap` 会捕获该信号，执行 `cleanup` 函数。
- 在 `cleanup` 函数中，添加了 `exit` 命令，这会导致脚本在执行清理操作后退出。

8.3 expect

在 **Shell 编程** 中，`expect` 是一个非常有用的工具，通常用于自动化交互式任务的执行。

`expect` 允许脚本自动处理那些需要用户交互的程序，模拟用户的输入并自动进行操作。它常用于 SSH 登录、FTP 操作、数据库管理、安装脚本等需要用户输入密码或选择的场景。

8.3.1 什么是 expect

`expect` 是一个自动化工具，提供了一种处理交互式会话的方式。在传统的 Shell 脚本中，我们通常会等待用户输入或交互，而 `expect` 允许我们模拟这些输入，自动执行一些交互任务。它的主要功能是：

- 自动处理程序的提示信息。
- 模拟用户输入。
- 自动响应密码提示、确认提示等交互式问题。

`expect` 本身并不是一个 Shell 内建命令，而是一个独立的程序。可以通过命令行或者在 Shell 脚本中调用它。

8.3.2 expect 安装

```
1 # Centos
2 [root rockylinux-1 ~] WORK 0 # yum install -y expect
3
4 # Ubuntu
5 17:33:25 root@redis02:~# apt install -y expect
6
7
8 [root rockylinux-1 ~] WORK 0 # expect -v
9 expect version 5.45.4
10 17:35:27 root@redis02:~# expect -v
11 expect version 5.45.4
```

8.3.3 expect 符号标识

符号	含义	示例
.	匹配任何单个字符（不包括换行符）	a.c 匹配 "abc", "adc" 等
*	匹配零个或多个字符	ab*c 匹配 "ac", "abc", "abbbbbc" 等
?	匹配零个或一个字符	a?b 匹配 "ab" 或 "b"
[]	匹配字符集中的任何一个字符	[a-z] 匹配任何小写字母
[^]	匹配不在字符集中的任何一个字符	[^0-9] 匹配任何非数字字符
{n,m}	匹配前一个字符或子表达式至少 n 次，最多 m 次	a{3,5} 匹配 "aaa", "aaaa", "aaaaa"
()	用于分组，通常与 一起使用	
\	转义特殊字符，使其具有字面意义	\. 匹配字符 "."
\r	匹配回车符	\r 在 send 命令中用于模拟回车键
\n	匹配换行符	\n 用于模拟换行
eof	表示进程输出结束	expect eof 用于等待进程结束

8.3.4 expect 常用命令

命令	说明
spawn	启动一个新的进程（如 SSH、FTP、MySQL 等）。
expect	捕获进程的输出，根据匹配的模式执行操作（可以使用正则表达式）。
send	发送输入到启动的进程中，模拟用户输入。
timeout	设置等待输出的最大时间，如果超时会进行相应处理。

命令	说明
<code>exp_continue</code>	继续匹配下一个 <code>expect</code> 。

8.3.5 expect 基础用法

```
1 #!/usr/bin/expect
2
3 # 设置一个超时时间（可选）
4 set timeout 20
5
6 # 启动一个命令
7 spawn command
8
9 # 捕获期望的输出并进行响应
10 expect "pattern1"
11 send "response1\n"
12
13 expect "pattern2"
14 send "response2\n"
15
16 # 退出 expect
17 expect eof
```

语法说明：

- `spawn command`：启动一个新进程（如 SSH 登录、FTP 上传等）。
- `expect "pattern"`：等待特定的字符串或输出。
- `send "response"`：向进程发送指定的响应。
- `set timeout`：设置 `expect` 的超时时间，单位为秒，默认为 10 秒。
- `expect eof`：表示处理完所有的交互后，脚本结束。

8.3.6 expect 使用场景

8.3.6.1 自动化 ssh 登录

在通过 SSH 远程登录时，通常会要求输入密码。使用 `expect` 可以自动化这个过程，避免手动输入密码。

```
1 [root rockylinux-1 ~] WORK 127 # cat demo02.sh
2 #!/usr/bin/expect
3
4 # 设置超时时间为 10 秒
5 set timeout 10
6
7 # 远程服务器的登录信息
8 set user "root"
9 set host "192.168.121.132"
```



```
10 set password "123"
11
12 # 使用 spawn 命令启动 SSH 登录
13 spawn ssh $user@$host
14
15 # 捕获密码提示并自动发送密码
16 expect "password:"
17 send "$password\r"
18
19 # 捕获登录后显示的 shell 提示符
20 expect "$ "
21
22 # 执行命令
23 send "ls -l\r"
24
25 # 等待命令执行完毕
26 expect "$ "
27
28 # 退出 SSH 会话
29 send "exit\r"
30
31 # 等待 SSH 进程结束
32 expect eof
33
34 [root rockylinux-1 ~] WORK 0 # expect demo02.sh
35 spawn ssh root@192.168.121.132
36 root@192.168.121.132's password:
37 Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-88-generic x86_64)
38
39 * Documentation:  https://help.ubuntu.com
40 * Management:    https://landscape.canonical.com
41 * Support:       https://ubuntu.com/pro
42
43 System information as of Mon Dec 22 10:27:46 PM CST 2025
44
45 System load: 0.42          Processes:                225
46 Usage of /:  15.2% of 47.93GB  Users logged in:        1
47 Memory usage: 42%          IPv4 address for ens33: 192.168.121.132
48 Swap usage:  0%
49
50 * Strictly confined Kubernetes makes edge and IoT secure. Learn how
  MicroK8s
51 just raised the bar for easy, resilient and secure K8s cluster
  deployment.
52
53 https://ubuntu.com/engage/secure-kubernetes-at-the-edge
54
55 Expanded Security Maintenance for Applications is not enabled.
56
57 58 updates can be applied immediately.
```

```

58 To see these additional updates run: apt list --upgradable
59
60 Enable ESM Apps to receive additional future security updates.
61 See https://ubuntu.com/esm or run: sudo pro status
62
63
64 *** System restart required ***
65
66 Last login: Mon Dec 22 22:27:56 2025 from 192.168.121.180
67 22:28:24 root@redis02:~# ls -l
68 total 32
69 -rw-r--r-- 1 root root 540 Dec 22 17:10 demo01.sh
70 -rw-r--r-- 1 root root 567 Dec 22 17:11 demo02.sh
71 -rw-r--r-- 1 root root 673 Dec 22 17:16 demo03.sh
72 -rw-r--r-- 1 root root 647 Dec 22 17:17 demo04.sh
73 -rw-r--r-- 1 root root 480 Dec 22 17:20 demo05.sh
74 -rw-r--r-- 1 root root 543 Dec 22 17:25 demo06.sh
75 -rw-r--r-- 1 root root 6520 Dec 19 15:20 libaio1_0.3.113-4_amd64.deb
76 22:28:29 root@redis02:~# exit
77 logout
78 Connection to 192.168.121.132 closed.
79 [root rockylinux-1 ~] WORK 0 #

```

说明:

- `spawn` 启动 SSH 连接。
- `expect "password:"` 捕获密码提示，并用 `send` 发送密码。
- 在登录后，脚本自动执行 `ls -l` 命令，并最终退出 SSH 会话。

8.3.6.2 自动化 FTP 登录与文件上传

`expect` 可以用于自动化 FTP 登录、上传文件或执行其他交互命令。

```

1 [root rockylinux-1 ~] WORK 0 # cat demo03.sh
2 #!/usr/bin/expect
3
4 # 设置超时时间为 10 秒
5 set timeout 10
6
7 # FTP 登录信息
8 set host "ftp_host"
9 set user "your_username"
10 set password "your_password"
11 set file_path "/path/to/your/file"
12 set remote_path "/remote/directory"
13
14 # 使用 spawn 启动 FTP 会话
15 spawn ftp $host
16

```

```

17 # 捕获用户名提示并发送用户名
18 expect "Name *:"
19 send "$user\r"
20
21 # 捕获密码提示并发送密码
22 expect "Password:"
23 send "$password\r"
24
25 # 等待 FTP 提示符
26 expect "ftp>"
27
28 # 发送文件上传命令
29 send "put $file_path $remote_path\r"
30
31 # 等待文件上传完成
32 expect "ftp>"
33
34 # 退出 FTP 会话
35 send "bye\r"
36
37 # 等待 FTP
38

```

说明：

- `spawn ftp $host` 启动 FTP 会话。
- `expect "Name *:"` 和 `expect "Password:"` 捕获用户名和密码提示。
- `send "put ..."` 上传指定文件。

8.3.6.3 自动化数据库登录

`expect` 还可以用于自动化数据库登录，例如自动登录 MySQL 数据库并执行查询。

```

1 #!/usr/bin/expect
2
3 # 设置超时时间为 10 秒
4 set timeout 10
5
6 # MySQL 登录信息
7 set user "root"
8 set password "your_password"
9
10 # 使用 spawn 启动 MySQL 客户端
11 spawn mysql -u $user -p
12
13 # 捕获密码提示并发送密码
14 expect "Enter password:"
15 send "$password\r"
16

```

```

17 # 执行 SQL 查询
18 expect "mysql>"
19 send "SHOW DATABASES;\r"
20
21 # 等待查询结果
22 expect "mysql>"
23
24 # 退出 MySQL
25 send "exit\r"
26
27 # 等待 MySQL 进程结束
28 expect eof
29

```

说明：

- `spawn mysql -u $user -p` 启动 MySQL 客户端。
- `expect "Enter password:"` 捕获密码提示，并发送密码。
- 执行 SQL 查询并退出 MySQL。

8.3.7 expect 高级用法

8.3.7.1 使用 expect 自动处理不同的提示

`expect` 可以根据不同的提示执行不同的操作，例如根据不同的错误码进行处理
 示例：自动处理错误提示

```

1 [root rockylinux-1 ~] WORK 0 # cat demo04.sh
2 #!/usr/bin/expect
3
4 set timeout 10
5 spawn ssh user@remote_host
6
7 expect {
8     "password:" {
9         send "your_password\r"
10     }
11     "yes/no" {
12         send "yes\r"
13         exp_continue
14     }
15     timeout {
16         send_user "连接超时\n"
17         exit 1
18     }
19 }
20
21 expect "$ "
22 send "ls -l\r"

```

```
23 expect "$ "
24 send "exit\r"
25 expect eof
```

说明:

- `expect` 支持多种情况的处理, 可以使用大括号 `expect {}` 来定义多个匹配条件。
- `exp_continue` 允许在匹配后继续执行后续的 `expect`。

8.3.7.2 使用正则表达式匹配输出

`expect` 支持使用正则表达式来匹配程序输出, 这可以在处理复杂的交互式任务时非常有用。

示例: 使用正则表达式捕获输出

```
1 [root rockylinux-1 ~] WORK 0 # cat demo05.sh
2 #!/usr/bin/expect
3
4 set timeout 10
5 spawn ssh user@remote_host
6
7 expect {
8     "password:" {
9         send "your_password\r"
10     }
11     "yes/no" {
12         send "yes\r"
13         exp_continue
14     }
15     timeout {
16         send_user "连接超时\n"
17         exit 1
18     }
19 }
20
21 expect {
22     "Welcome" { send "echo Welcome\r" }
23     "Permission denied" { send "echo 权限拒绝\r" }
24 }
25
26 expect "$ "
27 send "exit\r"
28 expect eof
```

说明:

- `expect` 可以通过正则表达式捕获不同的字符串 (如 "Welcome" 或 "Permission denied") 。

8.4 嵌套变量（变量的变量/间接变量）

8.4.1 什么是嵌套变量

- 普通变量: `name="tom" → $name` 得到 `tom`
- 嵌套变量 / 间接引用: `a="name"`, 你希望通过 `$a` 找到变量 `name`, 再取出 `name` 的值。也就是:

变量 A 存的是变量 B 的名字; 我们想取的是变量 B 的值。

8.4.2 嵌套变量的使用方式

8.4.2.1 一层嵌套

```
1 name="tom"
2 a="name"
3
4 # 想得到 tom (即 $name 的值)
```

你会发现:

- `$a` 得到的是 `"name"`
- 但你想要的是 `$name` 的值 `"tom"`

8.4.2.2 多层嵌套

```
1 v1="v2"
2 v2="v3"
3 v3="final"
4
5 # 希望从 v1 一路解析到 final
```

8.4.3 eval 命令详解

8.4.3.1 eval 是什么

`eval` 会把它的参数拼成一条命令字符串, 然后让当前 shell 再解析执行一次。

```
1 11:21:38 root@redis02:~# echo hello
2 hello
3 11:23:25 root@redis02:~# eval "echo hello"
4 hello
5 11:23:31 root@redis02:~#
```

8.4.3.2 基本语法

```
1 eval COMMAND_STRING
```

8.4.3.3 eval 常见用法总结表

用法	目的	示例
执行拼接命令	动态执行字符串命令	<code>cmd="ls -l"; eval "\$cmd"</code>
间接引用变量	变量的变量	<code>eval echo \\$\$var</code>
动态赋值	把拼出来的变量名赋值	<code>eval "x_\$i=100"</code>
动态导出环境变量	变量名动态生成	<code>eval "export \$k=\$v"</code>

8.4.4 如何实现嵌套变量

`eval` 的作用：把“拼出来的一段命令字符串”，再当成命令执行一遍。

8.4.4.1 使用 eval 实现一层嵌套变量

示例：从 `a="name"` 取到 `$name`

```
1 11:18:26 root@redis02:~# cat demo12.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo12.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 name="tom"
16 a="name"
17 eval echo \$$a
18
19 11:18:27 root@redis02:~# bash demo12.sh
20 tom
21 11:18:30 root@redis02:~#
```

解释：

- `\$$a`：先得到字面上的 `$name`

- `eval echo $name`: 再执行一次, 输出 `tom`

8.4.4.2 把结果赋值出来

```
1 11:20:11 root@redis02:~# cat demo13.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo13.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 name="tom"
16 a="name"
17
18 eval value=\$$a
19 echo $value
20
21 11:20:13 root@redis02:~# bash demo13.sh
22 tom
```

8.4.4.3 多层嵌套解析 (循环 + eval)

```
1 11:21:35 root@redis02:~# cat demo14.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo14.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 #!/bin/bash
16 v1="v2"
17 v2="v3"
18 v3="final"
19
20 cur="v1"
```



```

17 for i in {1..3}; do
18     eval cur=\${$cur}
19 done
20
21 echo "$cur"    # final
22
23
24 11:21:36 root@redis02:~# bash demo14.sh
25 final

```

8.4.5 eval 实践案例

8.4.5.1 动态变量名赋值（批量生成）

```

1 11:26:50 root@redis02:~# cat demo15.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo15.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16 for i in {1..3}; do
17     eval "user_${i}=tom_${i}"
18 done
19
20 echo "$user_1"
21 echo "$user_2"
22 echo "$user_3"
23
24 11:26:52 root@redis02:~# bash demo15.sh
25 tom_1
26 tom_2
27 tom_3
28
29 11:26:54 root@redis02:~#

```

8.4.5.2 动态读取变量名（通过索引取值）

```

1 17:21:04 root@redis02:~# cat demo16.sh
2 #!/bin/bash

```

```

3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo16.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11 user_1="alice"
12 user_2="bob"
13 user_3="cindy"
14
15 idx=2
16 var="user_${idx}"
17
18 eval echo \$$var    # bob
19
20 17:21:05 root@redis02:~# bash demo16.sh
21 bob
22 17:21:06 root@redis02:~#

```

8.4.5.3 写一个通用函数：传入变量名，返回变量值

```

1  17:24:23 root@redis02:~# cat demo17.sh
2  #!/bin/bash
3  #
   =====
   ====
4  # 脚本基础信息
5  # filename: demo17.sh
6  # name: xuruizhao
7  # email: xuruizhao00@163.com
8  # v: LnxGuru
9  # GitHub: xuruizhao00-sys
10 #
   =====
   ====
11
12 get_var() {
13     local varname="$1"
14     eval echo \$$varname
15 }
16
17 name="tom"
18 echo "$(get_var name)"    # tom

```

```
19
20
21 17:24:24 root@redis02:~# bash demo17.sh
22 tom
```

8.4.5.4 eval + case：动态执行运维命令

```
1 17:25:30 root@redis02:~# cat demo18.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo18.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 action="$1"
16
17 cmd_start="systemctl start nginx"
18 cmd_stop="systemctl stop nginx"
19 cmd_restart="systemctl restart nginx"
20
21 case "$action" in
22     start) eval "$cmd_start" ;;
23     stop) eval "$cmd_stop" ;;
24     restart) eval "$cmd_restart" ;;
25     *) echo "Usage: $0 {start|stop|restart}" ;;
26 esac
```

8.4.6 注意事项

8.4.6.1 eval 的风险：命令注入

如果 eval 的内容来自用户输入或不可信数据，容易被注入恶意命令。

× 危险示例：

```
1 user_input='name; rm -rf /'
2 eval "echo $user_input"
```

8.4.6.2 安全建议

建议	说明
不对不可信输入使用 eval	用户输入、外部文件内容要谨慎
变量名做白名单校验	只允许 <code>[a-zA-Z_][a-zA-Z0-9_]*</code>
优先用 <code>\${!var}</code> (bash 特性) 替代 eval	更安全、更清晰 (如果你的环境允许)

这部分我们是在讲 `eval` 命令，但在真实生产中能不用 `eval` 就尽量不用。

8.5 嵌套变量进阶

不使用 `eval` 的实现方式 (强烈推荐)

在 Bash 中，**嵌套变量** ≈ **间接引用** (Indirect Expansion)

8.5.1 为什么要“避免 eval”

虽然 `eval` 能实现嵌套变量，但它有 **三大问题**：

问题	说明
× 安全风险	极易命令注入 (生产事故重灾区)
× 可读性差	调试困难，逻辑不直观
× 维护成本高	稍复杂就很难看懂

👉 Bash 已经提供了“官方安全方案”

8.5.2 方式一：`${!var}` —— Bash 间接展开 (最常用)

8.5.2.1 什么是 `${!var}`

```
1 ${!var} === “把 var 的值当成变量名，再取它的值”
```

8.5.2.2 基础示例 (一层嵌套)

```
1 17:25:31 root@redis02:~# name="jerry"
2 17:31:43 root@redis02:~# b="name"
3 17:31:50 root@redis02:~# echo ${!b}
4 jerry
```

```

5 17:32:01 root@redis02:~#
6
7 ##### 执行过程
8 17:32:49 root@redis02:~# cat demo19.sh
9 #!/bin/bash
10 #
11 =====
12 ====
13 # 脚本基础信息
14 # filename: demo19.sh
15 # name: xuruizhao
16 # email: xuruizhao00@163.com
17 # v: LnxGuru
18 # GitHub: xuruizhao00-sys
19 #
20 =====
21 ====
22 name="jack"
23 c="name"
24 echo ${!c}
25
26 17:32:51 root@redis02:~# bash -x demo19.sh
27 + name=jack
28 + c=name
29 + echo jack
30 jack
31
32 #####
33 a="name"
34 ${!a}
35 → ${name}
36 → jack

```

8.5.2.3 对比 eval 的写法

方式	写法
eval	<code>eval echo \\$\$a</code>
推荐	<code>echo \${!a}</code>

- ✓ 更安全
- ✓ 更直观
- ✓ Bash 原生支持

8.5.2.4 多层嵌套（循环解析）

```
1 17:34:24 root@redis02:~# cat demo20.sh
2 #!/bin/bash
3 #
4 =====
5 # 脚本基础信息
6 # filename: demo20.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 #
12 =====
13 #
14 v1="v2"
15 v2="v3"
16 v3="final"
17
18 cur="v1"
19 for i in {1..3}; do
20     cur="${!cur}"
21 done
22 echo "$cur"
23
24 17:34:25 root@redis02:~# bash demo20.sh
25 final
```

8.5.3 方式二：declare -n（命名引用，最优雅）

Bash 4.3+ 支持

类似“指针 / 引用变量”

8.5.3.1 什么是 declare -n

```
1 declare -n ref=var
```

含义：

ref 不再是普通变量

👉 它“指向”变量 var

8.5.3.2 基础示例

```
1 17:34:27 root@redis02:~# name="lucy"
2 17:36:49 root@redis02:~# declare -n ref=name
3 17:36:58 root@redis02:~# echo $ref
4 lucy
5 17:37:03 root@redis02:~#
```

8.5.3.3 修改引用变量（双向）

```
1 17:34:27 root@redis02:~# name="lucy"
2 17:36:49 root@redis02:~# declare -n ref=name
3 17:36:58 root@redis02:~# echo $ref
4 lucy
5 17:37:03 root@redis02:~# ref="zhangsan"
6 17:37:36 root@redis02:~# echo $name
7 zhangsan
8 17:37:39 root@redis02:~#
```

👉 ref 就像 name 的别名

8.5.3.4 动态变量名（非常适合函数）

```
1 17:38:32 root@redis02:~# cat demo21.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo21.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 get_value() {
16     local varname="$1"
17     declare -n ref="$varname"
18     echo "$ref"
19 }
20
21 user="alice"
22 get_value user
```

8.5.4 `${!prefix*}`：批量嵌套变量（高级）

8.5.4.1 用途

根据变量名前缀，批量取变量名

8.5.4.2 示例

```
1 cfg_db_host="127.0.0.1"
2 cfg_db_port="3306"
3 cfg_db_user="root"
4
5 echo ${!cfg_db_*}
6
7
8 # 输出
9 cfg_db_host cfg_db_port cfg_db_user
```

8.5.4.3 配合循环读取值

```
1 for var in ${!cfg_db_*}; do
2     echo "$var = ${!var}"
3 done
4
5 # 输出
6 cfg_db_host = 127.0.0.1
7 cfg_db_port = 3306
8 cfg_db_user = root
```

8.5.5 对比

方式	是否推荐	安全性	可读性	适用场景
<code>eval</code>	× 不推荐	× 低	× 差	老脚本、被迫兼容
<code>\${!var}</code>	✓ 推荐	✓ 高	✓ 好	变量的变量
<code>declare -n</code>	★ 强烈推荐	★★★★	★★★★	函数 / 复杂逻辑

8.5.6 生产环境最佳实践

- 能用 `${!var}` 就不用 `eval`
 - 90% 的嵌套变量都能解决
- 函数里优先用 `declare -n`
- `eval` 只在“你 100% 控制输入”时使用
- 永远不要 `eval` 用户输入

8.6 二维数组

在 Shell 脚本中，**二维数组** 是一种特殊类型的数组，它将 **一维数组** 作为元素，这样可以存储多维数据。由于 Shell 本身并不直接支持传统的二维数组（类似于其他编程语言中的数组），我们通常使用 **关联数组** 或 **数组的数组** 来模拟二维数组的效果。

- **一维数组**：数组的元素是单个数据，可以通过索引访问，如 `array[0]`。
- **二维数组**：数组的元素是 **一维数组** 或 **关联数组**，可以通过 **两个索引** 来访问，如 `array[0][1]`

Shell 并没有内建的 **二维数组** 语法，但可以通过两种方法来实现：

- **方法一：使用关联数组（`declare -A`）**：用数组模拟二维数组。
- **方法二：数组的数组**：利用一维数组的索引表示二维结构

8.6.1 使用关联数组模拟二维数组

关联数组允许使用 **字符串** 作为索引，因此可以模拟二维数组。每个索引可以表示二维数组中的行和列。

```
1 22:02:01 root@redis02:~# cat demo26.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo26.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15 # 声明一个关联数组
16 declare -A matrix
17 # 给二维数组赋值
18 matrix[0,0]=1
19 matrix[0,1]=2
20 matrix[0,2]=3
21 matrix[1,0]=4
22 matrix[1,1]=5
23 matrix[1,2]=6
24 matrix[2,0]=7
25 matrix[2,1]=8
26 matrix[2,2]=9
27 # 打印二维数组内容
```

```

27 for ((i=0; i<3; i++)); do
28     for ((j=0; j<3; j++)); do
29         echo -n "${matrix[$i,$j]} "
30     done
31     echo
32 done
33
34
35 22:02:03 root@redis02:~# bash demo26.sh
36 1 2 3
37 4 5 6
38 7 8 9
39 22:02:05 root@redis02:~#

```

- `declare -A matrix`：声明一个关联数组 `matrix`，用来模拟二维数组。
- `matrix[0,0]=1`：给二维数组的第一个元素赋值。
- **循环打印**：通过双重循环访问二维数组的每个元素，并打印它们。

8.6.2 数组的数组

另一种实现二维数组的方法是通过**数组的数组**。在这种方法中，我们将每行作为一个单独的数组，使用普通数组的索引来表示行，然后通过进一步索引来访问列。

```

1 22:03:33 root@redis02:~# cat demo27.sh
2 #!/bin/bash
3 #
4 =====
5 ====
6 # 脚本基础信息
7 # filename: demo27.sh
8 # name: xuruizhao
9 # email: xuruizhao00@163.com
10 # v: LnxGuru
11 # GitHub: xuruizhao00-sys
12 #
13 =====
14 ====
15
16 # 定义数组的数组
17 matrix=(
18     "1 2 3"
19     "4 5 6"
20     "7 8 9"
21 )
22
23 # 打印二维数组内容
24 for row in "${matrix[@]}"; do
25     echo $row
26 done

```

```
23
24
25 22:03:34 root@redis02:~# bash demo27.sh
26 1 2 3
27 4 5 6
28 7 8 9
```

特性	关联数组模拟二维数组	数组的数组模拟二维数组
数据结构	使用一维数组模拟二维结构（使用逗号做索引）	每个元素是一行的字符串数组
访问方式	通过两个索引访问： <code>matrix[0,0]</code>	通过索引和行号访问： <code>matrix[0]</code>
优势	更加灵活，可以处理更复杂的结构和类型	实现简单，易于理解
限制	依赖于 Bash 4.x 或更高版本的关联数组支持	不适用于非简单的多维结构

8.6.3 杨辉三角

杨辉三角是一个数字三角形，其中每一行的数字由前一行的相邻两个数字相加得到。它的生成规则是：

- 第一行是 1；
 - 第二行是 1 1；
 - 第三行是 1 2 1；
 - 第四行是 1 3 3 1；
 - 以此类推。
- 每个数字的值是其上方两个数字之和。

```
1 22:07:28 root@redis02:~# cat demo28.sh
2 #!/bin/bash
3 #
4 # =====
5 # 脚本基础信息
6 # filename: demo28.sh
7 # name: xuruizhao
8 # email: xuruizhao00@163.com
9 # v: LnxGuru
10 # GitHub: xuruizhao00-sys
11 # =====
12 # 读取用户输入的行数
13 read -p "请输入杨辉三角的行数: " rows
14
15 # 初始化一个二维数组，用来存储杨辉三角的值
16 declare -A triangle
```

```

17
18 # 生成杨辉三角
19 for ((i = 0; i < rows; i++)); do
20     # 第一列和最后一列的值为 1
21     triangle[$i,0]=1
22     triangle[$i,$i]=1
23
24     # 计算每一行的中间值
25     for ((j = 1; j < i; j++)); do
26         triangle[$i,$j]=$((triangle[$((i-1)),$((j-1))]] + triangle[$((i-
27         1)),$j]))
28     done
29 done
30
31 # 打印杨辉三角
32 for ((i = 0; i < rows; i++)); do
33     # 打印每一行前面的空格（用于格式化输出）
34     for ((j = i; j < rows - 1; j++)); do
35         echo -n " "
36     done
37
38     # 打印当前行的数字
39     for ((j = 0; j <= i; j++)); do
40         echo -n "${triangle[$i,$j]} "
41     done
42     echo # 换行
43 done
44
45 22:07:31 root@redis02:~# bash demo28.sh
46 请输入杨辉三角的行数: 10
47     1
48     1 1
49     1 2 1
50     1 3 3 1
51     1 4 6 4 1
52     1 5 10 10 5 1
53     1 6 15 20 15 6 1
54     1 7 21 35 35 21 7 1
55     1 8 28 56 70 56 28 8 1
56     1 9 36 84 126 126 84 36 9 1
57
58 22:07:33 root@redis02:~#

```

结束语

1. 至此，Shell 脚本核心知识点的梳理就告一段落了。理论结合实践，多敲多练才能真正掌握。愿这份笔记能成为你工作学习中的小帮手！

2. 笔记的收尾，不是学习的终点。Shell 脚本的魅力在于它能灵活解决各类自动化问题，后续不妨试着结合实际场景编写脚本，探索更多高级用法。希望你可以写出更高效的代码！
3. 这份笔记记录了 Shell 脚本的核心要点，但技术的边界远不止于此。建议多查阅官方文档、分析开源脚本案例，不断拓宽自己的技术视野。
4. 以上就是本次 Shell 脚本笔记的全部内容。从基础语法到实战技巧，希望能帮你打通 Shell 脚本的任督二脉。动手尝试，你会发现更多乐趣！